

RL SYSTEM DESIGN FOR ML & AI ENGINEERS

Reinforcement Learning System Design Interview

Standard architecture patterns for scale: Deep RL, RLHF, and case studies from Netflix, Microsoft, Uber, Meta, Google, Yahoo, and Expedia.

MDPs, value methods, policy gradients, PPO, RLHF & DPO, offline RL, multi-agent systems, and 17 sourced production case studies.



Reinforcement Learning System Design Interview

Standard architecture patterns for scale: Deep RL, RLHF, and case studies from Netflix, Microsoft, Uber, Meta, Google, Yahoo, and Expedia.

Copyright © 2026 AI Engineering Insider. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except for brief quotations embodied in reviews and certain other noncommercial uses permitted by copyright law.

This guide is provided for educational purposes. It is not affiliated with, endorsed by, or sponsored by any company named within. Company names, interview reports, and case studies are cited to their primary sources and used for commentary and educational analysis. Compiled practice questions are labelled as such and are not represented as verbatim interview transcripts.

The information is provided “as is,” without warranty of any kind. The author shall not be liable for any loss arising from reliance on its contents.

Companion code ([rl_interview_lab](#)) is released for educational use alongside this guide.

First edition, 2026.

Web: [aiengineeringinsider.com](#) | Newsletter: [aiengineeringinsider.substack.com](#)

How to Use This Guide

Subscribe → aiengineeringinsider.substack.com/subscribe

Practitioner-first, theory dosed in exactly where an interviewer would probe it.

This guide is built the same way as the other titles in the AI Engineering Series: short conceptual explanations, worked frameworks you can reuse on a whiteboard, and an **interview** box every time a question is the kind you should expect to be asked out loud, in a room, under time pressure.

Reinforcement learning interviews differ from most ML interviews in one important way: interviewers are not usually checking whether you can reproduce an equation. They are checking whether you respect how difficult RL is in practice, whether you can reason about a system where actions influence future data, and whether you know when *not* to reach for RL at all. This guide is written with that judgment-testing structure in mind, not as a textbook substitute for Sutton and Barto.

Who this is for

This guide assumes you already know supervised deep learning (backpropagation, optimization, regularization) and are comfortable with Python and a deep learning framework. It does not assume prior RL coursework. It is written for four overlapping audiences:

RL Research Engineer / Applied Scientist

candidates interviewing at frontier labs, where RLHF, alignment, and reward modeling questions dominate the loop.

ML Engineer with an RL component

candidates whose team uses RL for one part of a larger system (recommendations, bidding, notifications) and who will be asked to reason about that system end to end.

RLHF / Alignment Engineer

candidates whose work sits at the intersection of RL and large language models, where DPO, reward over-optimization, and Constitutional AI are now standard interview topics.

Robotics / Autonomy Engineer

candidates for whom sim-to-real transfer and domain randomization are not theoretical curiosities but the actual gap between an offline-looking-great policy and a working robot.

How RL interviews differ across company archetypes

Frontier AI labs (Anthropic, OpenAI, Google DeepMind) weight RLHF, Constitutional AI, reward hacking, and alignment philosophy heavily, often combined with a systems round on training infrastructure.

Robotics and autonomy (Tesla, Waymo, Wayve) weight sim-to-real transfer, domain randomization, and safety-constrained control, usually inside a broader perception/controls loop.

Ads, recommendations, and marketplaces (Meta, Pinterest, Uber, DoorDash) weight contextual bandits, offline policy evaluation, and reward design under business constraints, more than deep algorithmic novelty.

Quant and trading weight sample efficiency, risk-adjusted reward shaping, and skepticism about RL’s reliability under non-stationary environments.

★ Key Insight

The single best signal you can give in an RL interview is judgment about when RL is the wrong tool. Candidates who treat RL as a fragile, high-leverage instrument rather than a universal hammer are consistently rated higher than candidates who can recite more algorithms.

How the chapters are organized

Parts I and II rebuild the classical and deep RL foundations the way an interviewer would actually probe them: not as a survey, but as a chain of “why was this method invented” answers. Part III covers RL for games and the sim-to-real gap, two topics that show up as standalone interview questions far more often than general RL texts suggest. Part IV is the fastest-growing segment of real interview loops: RLHF, DPO, and reward over-optimization, written to connect directly to the alignment and security-of-training-pipelines questions now common at frontier labs. Part V covers multi-agent RL at the depth interviewers actually expect. Part VI is the interview-prep payload: real, sourced case studies of RL in production at named companies, followed by a fifty-question bank split between verified real interview reports and a compiled practice set.

! Pitfall / Warning

Section A of Chapter 15 (Verified Real Interview Reports) contains only questions traceable to a named company, role, and source. Section B is compiled by the author from confirmed topic areas and industry-standard interview patterns. The two are kept deliberately separate so you always know which is which.

The companion code: `rl_interview_lab`

Every algorithm in Parts I–V is implemented in a single, runnable Python package, `rl_interview_lab`, that ships with this guide. It is a progressive library: the classical chapters share one `GridWorld` environment, and the deep-RL chapters build on `Gymnasium` and `PyTorch`, all running on a laptop CPU. Each chapter quotes and explains the exact module it corresponds to, and the console outputs printed in the book are real outputs from that code. Install it with `pip install -r requirements.txt` and run `python scripts/demo.py` to reproduce every result. Treat the code as the lab bench for the theory: read the chapter, then run the module. The full, continually-updated source for this ebook and its code companion lives at github.com/lamhotsiagian/reinforcement-learning-lab.

Book at a glance

Ch	Title	Main focus
1	MDPs, States, Actions & Rewards	The formalism; Bellman equations; state/reward design
2	Dynamic Programming	Policy/value iteration; contraction; the deadly triad
3	Monte Carlo Methods	Model-free returns; bias-variance; the n -step spectrum
4	Q-Learning & SARSA	On- vs. off-policy; TD error; overestimation
5	Deep Q-Networks	Replay, target nets, Double DQN, Rainbow
6	Policy Gradients (REINFORCE)	Policy gradient theorem; baselines; variance
7	Actor-Critic (A2C/A3C)	Advantage, the critic, GAE, sync vs. async
8	PPO	Clipped surrogate; TRPO; DDPG/TD3/SAC
9	RL for Games	AlphaZero, MCTS, self-play as curriculum
10	Sim-to-Real Transfer	Reality gap; domain randomization; system ID
11	Reward Modeling & RLHF	SFT→RM→PPO; DPO; reward over-optimization
12	Multi-Agent RL	Non-stationarity; CTDE; QMIX; self-play
13	Offline RL, Model-Based & Exploration	Distribution shift; CQL/IQL; OPE; Dreamer; RND
14	RL System Design Case Studies	17 sourced production deployments
15	Top 50 Most Asked Questions	Verified + compiled question bank
App.	Appendices & Derivations	Roadmap, notation, flashcards, reproducibility, proofs

Contents

How to Use This Guide	ii
I Foundations	1
1 MDPs, States, Actions & Rewards	2
1.1 Overview	2
1.1.1 The agent–environment loop	2
1.2 Why the Discount Factor Exists	3
1.3 Value Functions and the Bellman Equations	3
1.4 State and Action Space Design	3
1.5 The Project: a Reusable Tabular MDP	4
1.6 Comparison: MDP, POMDP, and the Bandit	5
1.7 Mock Interview Questions	5
Walk me through the Bellman optimality equation. Why does it have a max operator instead of an expectation?	5
How does the discount factor γ affect agent behaviour, and what happens as $\gamma \rightarrow 1$?	6
What is the Markov property, and what do you do when your state is not Markov?	6
Distinguish V^π , Q^π , V^* , and Q^* . Which do you actually need to act?	6
A product manager wants to “use RL” for a problem with no sequential structure. How do you respond?	6
How would you formulate “reduce customer churn” as an MDP? Walk through each element.	7
Why can a small change to the reward function completely change the optimal policy, and why is that dangerous?	7
In our GridWorld the dynamics are stochastic (a 10% slip). How does that change the optimal policy versus the deterministic case?	7
What is the difference between episodic and continuing tasks, and how does each handle the return?	7
Your state space is huge and continuous. What are your options, and what do you lose with each?	8
2 Dynamic Programming	9
2.1 Overview	9
2.2 Policy Iteration vs. Value Iteration	9
2.3 Bellman Optimality as a Contraction	10
2.4 The Project: Planning on GridWorld	10
2.5 Comparison: DP Methods	11
2.6 Mock Interview Questions	12

What is the difference between policy iteration and value iteration, and when would you prefer one?	12
Prove, or at least argue, why value iteration converges.	12
What is the “deadly triad,” and why is DP immune to it?	12
Policy iteration’s inner evaluation loop is expensive. How would you speed it up without breaking convergence?	12
Why does DP need the full model, and what is the minimal change that removes that requirement?	13
Your MDP has 10^{12} states. DP is off the table, what stays useful from this chapter?	13
What is the policy improvement theorem and why does it guarantee monotone progress?	13
In our GridWorld value iteration took 15 sweeps but policy iteration only 3 updates. Reconcile that.	13
How does the discount factor interact with convergence speed and numerical conditioning in DP?	13
Asynchronous DP updates states in arbitrary order. Does that still converge, and why would you want it?	14
3 Monte Carlo Methods	15
3.1 Overview	15
3.1.1 First-visit vs. every-visit, and exploring starts	15
3.2 Architecture: the MC Control Loop	16
3.3 The n-Step Spectrum	16
3.4 The Project: MC Control on GridWorld	16
3.5 Comparison: MC vs. TD vs. DP	17
3.6 Mock Interview Questions	18
Explain the bias–variance trade-off between Monte Carlo and TD learning.	18
First-visit vs. every-visit MC, are they both correct, and which would you use?	18
Why does MC control learn Q instead of V ?	18
What problem do exploring starts solve, and what do you do when you cannot use them?	18
Walk through how off-policy MC works and why its variance can explode.	19
How does the n-step return connect MC and TD, and how do you pick n ?	19
Your MC value estimates are noisy and slow to converge in a highly stochastic environment. What concretely do you change?	19
Why is the incremental-mean update important in practice, and how does it relate to the TD step size?	19
Can MC be used with function approximation, and what breaks?	19
In our GridWorld, MC control recovered nearly the same policy as value iteration without the model. What does that demonstrate, and where would it fail?	20
4 Q-Learning & SARSA	21
4.1 Overview	21
4.1.1 The update rules, side by side	21
4.2 Architecture: Where the Two Targets Diverge	22
4.3 Convergence and Overestimation Bias	22
4.4 The Project: Q-Learning vs. SARSA on GridWorld	22
4.5 Comparison Table	24

4.6	Mock Interview Questions	24
	What is the difference between on-policy and off-policy learning, and why does importance sampling matter?	24
	Walk through SARSA vs. Q-learning on cliff-walking, where do they diverge and why?	24
	Why does Q-learning overestimate action values, and how does Double Q-learning fix it?	25
	What is the TD error, and why is it considered a biologically and algorithmically central quantity?	25
	Under what conditions does tabular Q-learning provably converge, and which condition is hardest to satisfy in practice?	25
	How do the learning rate α and exploration rate ϵ interact, and how would you schedule them?	25
	When would you choose SARSA over Q-learning in production, despite Q-learning's optimality?	26
	Expected SARSA, what is it, and where does it sit relative to SARSA and Q-learning?	26
	You set $\gamma = 0.99$ and your tabular Q-learning is unstable and slow. What do you check first?	26
	Q-learning is off-policy, which enables experience replay. Explain that link and one risk it introduces.	26
II	Deep Reinforcement Learning	28
5	Deep Q-Networks (DQN)	29
5.1	Overview	29
5.2	Architecture: the Two Stabilizers	29
5.3	The Rainbow Lineage	30
5.4	The Project: DQN with Double-DQN Targets	30
5.5	Comparison: Vanilla DQN vs. Rainbow Components	31
5.6	Mock Interview Questions	32
	Why does naive Q-learning diverge when combined with neural networks, and how does DQN fix it?	32
	What problem does Double DQN solve, and how, concretely?	32
	Explain Rainbow DQN. Which components matter most empirically?	32
	Compare ϵ -greedy, UCB, and Thompson sampling for exploration.	32
	How do you size the replay buffer and the target-sync interval, and what goes wrong at the extremes?	33
	Why use the Huber loss instead of MSE for the TD error?	33
	In a Gymnasium environment, episodes end by either termination or truncation. Why must DQN treat them differently?	33
	Your DQN's loss is decreasing but episode return is flat. Walk me through your debugging.	33
	When is DQN the wrong choice, and what would you reach for instead?	34
	Prioritized experience replay samples high-TD-error transitions more often. What bias does that introduce and how is it corrected?	34
6	Policy Gradients — REINFORCE	35

6.1	Overview	35
6.2	Deriving the Policy Gradient	35
6.3	Architecture: the REINFORCE Loop	36
6.4	The Variance Problem and the Baseline	36
6.5	The Project: REINFORCE with a Whitenened-Return Baseline	36
6.6	Comparison: Value-Based vs. Policy-Gradient	37
6.7	Mock Interview Questions	37
	Explain the policy gradient theorem in your own words, derive it if you can. . .	37
	Why does subtracting a baseline reduce variance without adding bias, and what makes a good baseline?	38
	REINFORCE is unbiased; DQN is biased. Why is REINFORCE still often harder to train?	38
	How would you scale REINFORCE to an action space of millions, like a recommender?	38
	Walk through how REINFORCE handles a continuous action space.	38
	What is the credit-assignment problem in REINFORCE, and how does the reward-to-go formulation help?	39
	Why is REINFORCE on-policy, and what does that cost you?	39
	Your REINFORCE training is extremely noisy and barely improving. What concrete levers do you pull?	39
	What role does the entropy bonus play, and what is the failure mode if you omit it?	39
	Compare REINFORCE with a learned baseline to one-step actor-critic. Are they the same thing?	40
7	Actor-Critic — A2C, A3C	41
7.1	Overview	41
7.2	Architecture: Two Heads, One Signal	42
7.3	Synchronous vs. Asynchronous	42
7.4	The Project: A2C on CartPole	42
7.5	Comparison: REINFORCE vs. A2C vs. A3C	43
7.6	Mock Interview Questions	43
	Why use an advantage function instead of raw returns in actor-critic methods? .	43
	What is the difference between A2C and A3C, and why does it matter for stability? .	43
	Why do actor-critic methods scale to continuous and large action spaces better than DQN?	44
	Walk through GAE and the role of its λ parameter.	44
	Should the actor and critic share a network body or be separate? Trade-offs? . .	44
	In the code, the advantage is detached before the actor loss. Why is that essential? .	44
	How do the actor and critic learning rates interact, and what happens if the critic is too slow?	45
	Actor-critic reintroduces bootstrapping with function approximation and (often) off-policy data. Are we back in the deadly triad?	45
	Your A2C critic loss is low but the policy is poor. What is likely happening? . .	45
	Position actor-critic between REINFORCE and PPO, what does each step add? .	45
8	PPO	47
8.1	Overview	47

8.2	From TRPO to PPO	47
8.3	The Project: a Faithful Clipped PPO	48
8.4	Continuous Control: DDPG, TD3, SAC	49
8.5	Comparison: Modern Policy-Optimization Methods	50
8.6	Mock Interview Questions	50
	What does PPO's clipped objective do mechanically, and why clip instead of a hard KL constraint?	50
	Compare DDPG, TD3, and SAC; what specific failure does each fix?	50
	Why can PPO safely run multiple epochs over the same batch when REINFORCE cannot?	50
	Walk through how you would tune PPO if it is unstable.	51
	Explain the min and clip interaction with the sign of the advantage.	51
	PPO is on-policy and SAC is off-policy. When would you pick each in a real project?	51
	Why is the importance ratio computed as $\exp(\text{new_logp} - \text{old_logp})$, and where do old_logp come from?	51
	How does PPO handle the exploration–exploitation trade-off?	52
	RLHF uses PPO to fine-tune LLMs. What changes versus CartPole, and what stays the same?	52
	A common mistake is calling SAC strictly better than TD3. Make the calibrated comparison.	52
III	RL for Games and the Real World	54
9	RL for Games	55
9.1	Overview	55
9.2	Architecture: the AlphaZero Learning Loop	55
9.3	The Project: Self-Play Tic-Tac-Toe	56
9.4	Self-Play as a Curriculum, and Its Failure Modes	56
9.5	Comparison: Game-RL Milestones	57
9.6	Mock Interview Questions	57
	Why are Atari and Go standard RL benchmarks, and what do they fail to test?	57
	Explain how AlphaZero combines learned value/policy networks with MCTS.	57
	What does MuZero add over AlphaZero, and why does it matter for non-game domains?	57
	Why is self-play described as an automatic curriculum, and when does that break?	58
	In the tic-tac-toe project, why is each player's move updated one ply late, and the loser's prior move penalized?	58
	Why was inference-time search necessary for superhuman Go when the network was already strong?	58
	How would you adapt game-RL ideas to a partially observable, real-time game like StarCraft?	58
	A candidate claims their agent “solved” a game because it beats the built-in AI. How do you stress-test that claim?	59
	Atari is model-free with unlimited data; real systems are neither. Which game-RL lessons still transfer?	59

Where does reward design go wrong in games, and how is that a preview of RLHF?	59
10 Sim-to-Real Transfer	61
10.1 Overview	61
10.2 Architecture: Train Wide, Deploy Once	62
10.3 The Project: Randomized Dynamics and a Robustness Curve	62
10.4 Comparison: Closing the Reality Gap	63
10.5 Mock Interview Questions	63
What is the sim-to-real gap, and what techniques mitigate it?	63
How exactly does domain randomization help transfer, and what is its main trade-off?	63
Design an RL-based robotics control system given the sim-to-real gap. Walk through your approach.	64
How would you measure whether a policy is actually robust, before risking real hardware?	64
When is domain randomization the wrong choice, and what would you do instead?	64
In the code, why must you recompute total_mass and polemass.length after randomizing?	65
What is the role of online fine-tuning, and what makes it risky?	65
Connect sim-to-real overfitting to reward hacking. Are they the same phenomenon?	65
Why is randomizing sensor latency and actuation delay often more important than randomizing mass?	65
Your domain-randomized policy is robust in sim across the whole range but still fails on the real robot. What now?	66
IV RLHF and Alignment	67
11 Reward Modeling & RLHF	68
11.1 Overview	68
11.2 Architecture: the Three-Stage Pipeline	68
11.3 The Project: a Reward Model and DPO, Side by Side	69
11.4 Comparison: Preference-Optimization Methods	70
11.5 Mock Interview Questions	70
Walk me through the RLHF pipeline end to end, from SFT to the final policy.	70
What is the Bradley-Terry model, and why is it used for reward modeling?	70
Why does RLHF include a KL penalty against the reference policy, and what happens if you remove it?	71
Compare PPO-based RLHF and DPO. What does DPO give up, and what does it gain?	71
What is reward over-optimization (Goodhart in RLHF), and how would you detect it during training?	71
In the project, held-out reward-model accuracy is 0.86 against an oracle ceiling of 0.89. Interpret that gap and the role of weight decay.	71
Distinguish RLAIF and Constitutional AI. Are they the same idea?	72
How is PPO actually applied to an LLM? What are the states, actions, and rewards?	72

What changed in RL for LLMs recently with GRPO-style reasoning RL, and why does it matter?	72
Tie the chapter together: are RLHF, DPO, RLAIF, Constitutional AI, and GRPO five separate techniques?	72
V Multi-Agent Reinforcement Learning	74
12 Multi-Agent RL	75
12.1 Overview	75
12.2 Architecture: Independent vs. Centralized	75
12.3 The Project: Independent Q-Learning on a Pursuit Grid	75
12.4 Comparison: MARL Approaches	76
12.5 Mock Interview Questions	77
What is CTDE, and why is it useful in multi-agent RL?	77
Why is the environment non-stationary from a single agent's perspective, and why does that matter?	77
When would multi-agent RL be the wrong choice for a problem that looks multi-agent?	77
Explain QMIX's monotonicity constraint and what it buys you.	77
How does self-play relate to general MARL, and why is it more stable?	78
In the project, two independent Q-learners coordinate fine. Does that contradict the claim that IQL is unstable?	78
What new failure modes appear in competitive (zero-sum) versus cooperative MARL?	78
How would you do credit assignment in a cooperative team with a single shared reward?	78
A team of MADDPG agents trains well in simulation but coordination collapses when one agent's policy is updated at deployment. What happened?	79
Where is genuinely multi-agent RL actually deployed, and why is it rarer than the literature suggests?	79
VI Advanced Topics	80
13 Offline RL, Model-Based RL & Exploration	81
13.1 Overview	81
13.2 Offline RL and Distributional Shift	81
13.2.1 The three families of offline algorithms	82
13.2.2 Off-policy evaluation (OPE)	82
13.3 Model-Based RL	82
13.4 Exploration Beyond ϵ -Greedy	83
13.5 Comparison: Offline RL Families	83
13.6 Mock Interview Questions	84
What is distributional shift in offline RL, and why does it break off-policy methods that work online?	84
Compare BCQ, CQL, and IQL.	84
You have one year of logged production data and cannot run live experiments yet. How do you build and validate an offline policy?	84

Why is off-policy evaluation hard, and which estimator would you reach for? . . .	84
When is model-based RL worth the added complexity, and what is its central failure mode?	85
Explain MBPO's short-rollout idea. Why not roll the model out for the full episode?	85
Why does ϵ -greedy fail on Montezuma's Revenge, and how does RND help? . . .	85
What is the noisy-TV problem in curiosity-driven exploration?	85
How are offline RL, model-based RL, and exploration the same problem underneath?	86
An offline policy scores far higher under your value model than the behaviour policy. Are you happy?	86
VII Interview Preparation	88
14 RL System Design Case Studies from Industry	89
14.1 The Ten-Step RL System-Design Checklist	89
Case 1 — Netflix: InTune, RL for Data-Pipeline Optimization	90
Case 2 — Wayfair: Griffin, Contextual Bandits for Notifications	91
Case 3 — Zynga: RL-Bakery and Game Personalization at Scale	91
Case 4 — DoorDash: RL Feeding a MIP for Dispatch	92
Case 5 — Alibaba: RL-to-Rank in Taobao Search	93
Case 6 — Spotify: BaRT, Bandits for Explainable Recommendations	93
Case 7 — LinkedIn: Offline RL for Mobile Notifications	94
Case 8 — Uber: Learned Value Signal for Marketplace Matching	95
Case 9 — DiDi: Standalone RL Dispatching at Scale	95
Case 10 — Pinterest: Deep RL for Ad-Ranking Utility Tuning	96
Case 11 — Booking.com: What Breaks Bandits in Production	96
Case 12 — Meta: Horizon, and Train-Complex / Deploy-Interpretable Bidding .	97
Case 13 — Google/YouTube: Top-K Off-Policy REINFORCE	98
Case 14 — Yahoo!: Real-Time Bidding by RL	98
Case 15 — Electronic Arts (SEED): RL for AAA Testing and Gameplay	99
Case 16 — Microsoft: Azure Personalizer / Xbox Top-of-Home Pilot	100
Case 17 — Expedia: Cascade Bandits for Ranking	100
14.2 Worked Senior System-Design Walkthroughs	101
14.2.1 Walkthrough 1 — RL Feed Recommendation (Meta / Google scale)	101
14.2.2 Walkthrough 2 — RLHF Pipeline for an LLM (frontier-lab scale)	102
14.3 Cross-Case Summary	103
14.4 How to Use These Case Studies in an Interview	104
14.5 Mock Interview Questions	105
Design an RL system to optimize push notifications. Walk me through it end to end.	105
When would you use a contextual bandit instead of full RL in a production recommender?	105
A stakeholder needs to explain the agent's decisions. How do you deploy RL while keeping it interpretable?	105
How would you validate an RL policy offline before risking online traffic?	105

Uber and DiDi both solve dispatch with RL but differ architecturally. Compare the choices.	106
What production realities break textbook bandit algorithms, per Booking.com?	106
How do you scale policy gradients to a recommender with millions of items?	106
When is RL the wrong tool, and how do you tell early? Use a real example.	106
Design the monitoring and reward-logging infrastructure for a deployed RL system.	107
Why do so many production systems use RL as one component feeding a classical system, rather than end-to-end?	107
15 Top 50 Most Asked Questions	108
15.1 How RL Interviews Are Weighted by Company Archetype	108
15.2 Section A — Verified Real Interview Reports	109
Explain the difference between PPO and GRPO.	109
How does Constitutional AI differ from traditional RLHF?	109
What are the limitations of Constitutional AI as an alignment approach?	110
If you were writing a constitution for a powerful AI system, what principles would you prioritize and why?	110
How would you measure whether an AI system is actually following its stated principles?	110
Explain RLHF optimization, including KL divergence and reward hacking.	110
During pretraining of a 100B parameter model, you suddenly see loss spikes. How do you debug this? Is it a data issue, learning rate problem, or hardware failure?	110
How would you prevent new privacy-preserving technologies from being exploited to make AI misuse harder to track?	110
Explain deep reinforcement learning.	110
What metrics would you use to evaluate the impact of RLHF on ChatGPT responses?	110
Be ready to discuss Q-learning, SARSA, multi-armed bandits, and DQNs.	111
Compare Q-learning (value-based) and policy gradient methods (policy-based), and discuss how each could apply to training an autonomous driving policy.	111
15.3 Section B — Compiled Interview Question Bank	111
Walk me through the Bellman optimality equation. Why does it have a max operator instead of an expectation?	111
What's the difference between policy iteration and value iteration, and when would you prefer one over the other?	111
Explain the bias-variance tradeoff between Monte Carlo and TD learning.	111
What is the “deadly triad” in RL, and why does it matter for anything you'd actually deploy?	112
Walk me through SARSA vs. Q-learning with a concrete example of where they'd diverge.	112
How does the discount factor γ affect agent behavior, and what happens as $\gamma \rightarrow 1$?	112
Explain the exploration-exploitation tradeoff. Compare ϵ -greedy, UCB, and Thompson sampling.	112
What's the difference between on-policy and off-policy learning, and why does importance sampling matter here?	112
Why does naive Q-learning diverge when combined with neural networks? How does DQN fix this?	112

What problem does Double DQN solve, and how?	112
Explain the policy gradient theorem in your own words — derive it if you can.	112
Why do we use an advantage function instead of raw returns in actor-critic methods?	112
What does PPO’s clipped objective actually do, mechanically? Why clip instead of using a hard KL constraint like TRPO?	112
Compare DDPG, TD3, and SAC. What specific failure mode does each one fix relative to its predecessor?	113
What is the maximum entropy RL framework, and why does SAC use it?	113
When would you choose a model-based RL approach over model-free, and what’s the main risk in doing so?	113
What is offline RL, and why does standard off-policy correction fail when you can’t collect new data?	113
Explain Rainbow DQN. Which components matter most empirically?	113
Walk me through the RLHF pipeline end to end, from SFT to the final policy.	113
What is the Bradley-Terry model, and why is it used for reward modeling?	113
Why does RLHF include a KL penalty against the reference policy? What happens if you remove it?	114
Compare PPO-based RLHF and DPO. What does DPO give up, and what does it gain?	114
What is reward over-optimization (Goodhart’s Law in RLHF), and how would you detect it during training?	114
Describe a real or hypothetical case of reward hacking. What made the reward function exploitable?	114
What is RLAIIF, and how does it differ from standard RLHF?	114
How would you design an evaluation framework to catch RLHF-induced regressions before shipping a model update?	114
What is CTDE (centralized training, decentralized execution), and why is it useful in multi-agent RL?	114
Explain how AlphaZero combines learned value/policy networks with MCTS.	114
What is the sim-to-real gap, and what techniques mitigate it?	114
How does domain randomization help an RL policy trained in simulation transfer to the real world?	115
When would multi-agent RL be the wrong choice for a problem that looks multi-agent?	115
Why are Atari and Go considered standard RL benchmarks, and what do they fail to test for?	115
Design an RL-based bidding system for online advertising. What would you use as the reward signal, and what could go wrong?	115
How would you evaluate a new RL policy before deploying it, given you can’t safely A/B test in production?	115
Your RL agent’s training reward is climbing steadily, but real-world performance is flat or worse. Walk me through how you’d debug this.	115
Tell me about a situation where you’d recommend against using RL, even though it technically applies.	116
How would you explain a reward-shaping decision to a non-technical stakeholder who’s worried the model is “gaming the metric”?	116

Design a system to safely roll out a new RLHF-tuned model update to a fraction of production traffic. What guardrails would you build in?	116
Appendices	117
Appendix A — Study Roadmap	117
Appendix B — Notation Cheat Sheet	118
Appendix C — Rapid-Fire Flashcards	118
MDP	118
On-policy vs. off-policy	118
Bootstrapping	118
Deadly triad	118
Experience replay	119
Target network	119
Baseline (policy gradient)	119
GAE	119
Trust region	119
Domain randomization	119
Reward hacking	119
Goodhart’s Law	119
Bradley-Terry model	119
CTDE	119
Offline RL	120
Contextual bandit	120
DPO	120
GRPO	120
Self-play	120
Sim-to-real gap	120
Appendix D — Reading List, Ordered by Depth	120
Appendix E — Mapping This Guide to Company Interview Patterns	121
Appendix F — Reproducibility & Benchmarks	121
Mathematical Derivations	123
Derivation 1 — The Policy Gradient Theorem	123
Derivation 2 — Generalized Advantage Estimation	124
Derivation 3 — Value Iteration Converges (the γ -Contraction)	124
Derivation 4 — DPO from the KL-Constrained RLHF Objective	125

Part I

Foundations

MDPs, States, Actions & Rewards

Subscribe → aiengineeringinsider.substack.com/subscribe

The formalism every other chapter in this book builds on, and the first thing a sharp interviewer will make you state out loud.

1.1 Overview

Every reinforcement learning problem, however dressed up, reduces to the same object: a **Markov Decision Process** (MDP), the five-tuple $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' | s, a)$ is the transition function, $R(s, a, s')$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor. An agent's job is to learn a **policy** $\pi(a | s)$, a mapping from states to actions (or to a distribution over actions), that maximizes expected cumulative discounted reward $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t R_{t+1}]$.

The core concept that separates RL from supervised learning is the **closed loop**: the agent's actions change the data it later sees. There is no fixed dataset. A classifier that predicts the wrong label does not change tomorrow's images; an RL agent that takes a bad action moves into a region of the state space it then has to learn from. This single fact, that the data distribution is a function of the policy, is the source of nearly every difficulty in the field, and it is what the MDP formalism is designed to make precise (Sutton & Barto, 2018).

Markov Property

A state s_t is Markov if $P(s_{t+1} | s_t, a_t) = P(s_{t+1} | s_1, a_1, \dots, s_t, a_t)$. In plain language: the state must contain everything relevant to predicting the future. If it does not, you do not have an MDP, you have a partially observable problem wearing an MDP's clothes (a POMDP), and most of the convergence guarantees in this book stop applying.

1.1.1 The agent–environment loop

The components interact in a strict cycle: the agent observes state s_t , selects action $a_t \sim \pi(\cdot | s_t)$, and the environment returns a reward r_{t+1} and next state $s_{t+1} \sim P(\cdot | s_t, a_t)$. Figure 1.1 shows this loop together with the quantities each algorithm family in this book estimates.

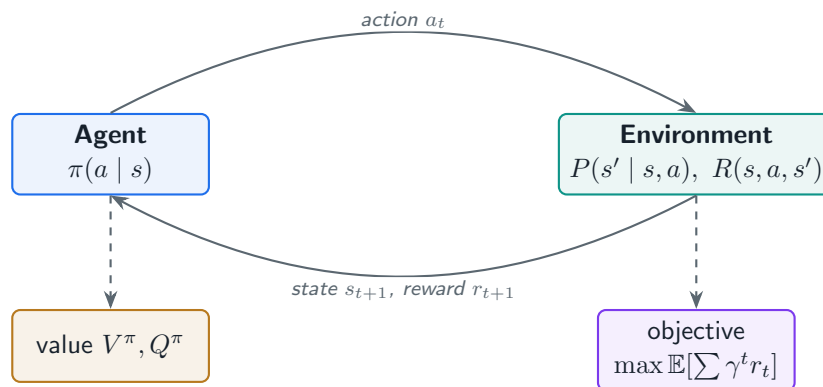


Figure 1.1: The MDP interaction loop. The agent influences the data it receives, which is the defining feature of reinforcement learning.

1.2 Why the Discount Factor Exists

The discount factor γ is not a minor implementation detail; it is what makes infinite-horizon problems mathematically tractable and what encodes a preference for sooner rewards over later, equally-sized ones. Without discounting, an infinite sequence of rewards may not converge to a finite sum, and value functions become ill-defined for continuing (non-episodic) tasks. The quantity $\frac{1}{1-\gamma}$ is the **effective horizon**: with $\gamma = 0.99$ the agent effectively plans about 100 steps ahead, and with $\gamma = 0.9$ only about 10.

★ Key Insight

When you are asked to “design an RL system” in an interview, stating the MDP formulation explicitly, state, action, reward, transition assumptions, episodic vs. continuing, before touching any algorithm, is the single highest-leverage thing you can do. It signals that you think about RL as problem formulation first, algorithm second.

1.3 Value Functions and the Bellman Equations

Two value functions recur throughout the book. The state-value $V^\pi(s)$ is the expected return from s following π ; the action-value $Q^\pi(s, a)$ is the expected return from taking a in s and following π thereafter. Both satisfy recursive **Bellman equations**, and the optimal versions satisfy the Bellman *optimality* equations with a max instead of an expectation over actions. The distinction, evaluation versus improvement, is the conceptual seam that Chapter 2 splits open.

1.4 State and Action Space Design

In production systems the gap between a *sufficient* (Markov) state and a *tractable* one is where most engineering effort goes. A fully sufficient representation is often too large to learn from efficiently, so practitioners compress, summarize, or use function approximation to project a richer state space into something learnable.

! Pitfall / Warning

A common interview trap: claiming that “more state information is always better.” In practice, overly large state spaces increase sample complexity and can make an otherwise solvable problem intractable. Production RL teams routinely caution that agents are sensitive to too many state inputs and recommend compression (e.g. autoencoding) when the natural state is large (see the case studies in Chapter 14).

1.5 The Project: a Reusable Tabular MDP

Every classical chapter in this book runs on one environment, `GridWorld`, defined in `rl_interview_lab/envs/gridworld.py`. It is a stochastic grid with a goal, a pit, walls, a per-step cost, and a “slip” probability that makes the dynamics genuinely random. Crucially it exposes *two* interfaces on the same dynamics: a *model* interface (`transitions`) returning the full $P(s', r | s, a)$ distribution for the planning methods of Chapter 2, and a *sampling* interface (`reset / step`) in the Gymnasium style for the model-free methods of Chapters 3–4.

Project Code — envs/gridworld.py

The model interface is what makes dynamic programming possible: the agent can “read the rules” instead of sampling them. The sampling interface is what every real agent is limited to. Building both on identical dynamics lets the book show *exactly* what changes when you move from planning to learning.

gridworld.py — the model interface used by dynamic programming

```
def transitions(self, state: State, action: Action) -> List[Transition]:
    """Full  $P(s', r | s, a)$  for one (state, action) pair."""
    if self.is_terminal(state):
        return [Transition(1.0, state, 0.0, True)]

    cell = self.decode(state)
    # The intended action plus the two perpendicular "slip" actions.
    perpendicular = {
        UP: (LEFT, RIGHT), DOWN: (LEFT, RIGHT),
        LEFT: (UP, DOWN), RIGHT: (UP, DOWN),
    }[action]
    outcomes = {action: 1.0 - self.slip_prob}
    for slip in perpendicular:
        outcomes[slip] = outcomes.get(slip, 0.0) + self.slip_prob / 2.0

    result = []
    for act, prob in outcomes.items():
        nxt = self.encode(self._move(cell, act))
        reward, done = self._reward(nxt)
        result.append(Transition(prob, nxt, reward, done))
    return result
```

The `slip_prob` is the whole point. With `slip_prob=0.1` the intended action succeeds 90% of

the time and “slips” to a perpendicular action 10% of the time. This makes the optimal policy avoid hugging the pit, because a slip near the edge of a cliff is expensive in expectation, exactly the kind of risk-aware behaviour that distinguishes a real policy from a memorised shortest path. In production you almost never have access to a clean `transitions` method like this; you have a simulator or logged data. The method exists here so Chapter 2 can compute the ground-truth optimal policy that the model-free chapters then approximate.

✓ Best Practice

Design the reward to express *what* you want, never *how* to achieve it. The small negative `step_reward` here (-0.04) encodes “finish quickly” without prescribing a route. Rewarding intermediate behaviours you *think* are good is the most common way to induce reward hacking (Chapter 11).

1.6 Comparison: MDP, POMDP, and the Bandit

Property	Multi-armed bandit	MDP	POMDP
State	none (single state)	fully observed	hidden; only observations
Actions affect future state	no	yes	yes
Credit assignment	immediate	delayed over time	delayed + perceptual aliasing
Core difficulty	exploration only	exploration + long-horizon credit	all of MDP + belief tracking
Typical solution	UCB, Thompson sampling	DP, TD, policy gradient	belief-state RL, recurrent policies

Table 1.1: The bandit is an MDP with one state; the POMDP is an MDP whose state you cannot see. Knowing which one you actually have is the first design decision.

1.7 Mock Interview Questions

Q1

Walk me through the Bellman optimality equation. Why does it have a max operator instead of an expectation?

The Bellman equation for the state-value function under a policy π is an expectation:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^\pi(s')].$$

This describes the value of *following* a given policy. The Bellman *optimality* equation replaces the expectation over actions with a max:

$$V^*(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^*(s')].$$

The max is there because we are no longer averaging over a fixed policy's action distribution; we are asking what the *best possible* action is at each state, given that all future decisions are also optimal. That makes the optimality equation a fixed-point equation for the optimal value function. An interviewer asking this is testing whether you understand that policy evaluation (expectation) and policy improvement (max) are genuinely different operations, not interchangeable notation.

Q2

How does the discount factor γ affect agent behaviour, and what happens as $\gamma \rightarrow 1$?

A small γ produces myopic agents that overweight immediate reward; a γ close to 1 produces farsighted agents that tolerate short-term loss for long-term gain, but at the cost of higher variance in value estimates and slower convergence, since the effective horizon $\frac{1}{1-\gamma}$ grows without bound. As $\gamma \rightarrow 1$ in a continuing task, $V^\pi(s)$ can diverge unless rewards are bounded and the induced Markov chain is well-behaved (e.g. ergodic). In practice γ is a hyperparameter that encodes the business's actual planning horizon: a notification system optimizing weekly engagement should not use the same γ as a high-frequency trading agent.

Q3

What is the Markov property, and what do you do when your state is not Markov?

The Markov property says the current state is a sufficient statistic for the future: conditioning on the full history adds nothing. When it fails, you have a POMDP. The standard fixes, in increasing order of cost: (1) *augment* the state with the missing information (e.g. stack the last k frames so velocity is observable, as the Atari DQN does); (2) maintain a *belief state*, a posterior over hidden state, and treat that as the MDP state; or (3) use a *recurrent* or transformer policy that learns its own summary of history. The interview signal is recognising that “my agent behaves erratically” is often a non-Markov state problem, not an algorithm problem.

Q4

Distinguish V^π , Q^π , V^* , and Q^* . Which do you actually need to act?

$V^\pi(s)$ is expected return from s under π ; $Q^\pi(s, a)$ conditions additionally on the first action. The starred versions are the optima. To *act* greedily you need Q , not V : $\arg \max_a Q^*(s, a)$ requires no model, whereas $\arg \max_a \sum_{s'} P(s' | s, a)[r + \gamma V^*(s')]$ needs the transition model. This is the entire reason model-free control methods learn Q : it makes the greedy step model-free. Stating this crisply is a strong signal because it explains a design choice that runs through Chapters 3–5.

Q5

A product manager wants to “use RL” for a problem with no sequential structure. How do you respond?

I would first check whether actions actually influence future state. If they do not, it is a contextual bandit, not a full MDP, and bandit methods (Thompson sampling, LinUCB) are simpler, more sample-efficient, and far easier to evaluate offline. Reaching for full RL

when a bandit suffices is a classic over-engineering failure: you inherit credit-assignment and stability problems you did not need. The highest-value answer demonstrates judgment about *when not to use RL*, which interviewers weight heavily.

Q6

How would you formulate “reduce customer churn” as an MDP? Walk through each element.

State: customer features plus engagement history (recency, frequency, tenure, recent support contacts). **Action:** which intervention to send (discount, email, no-op). **Reward:** a sparse retention signal at the horizon, often shaped with a small per-step cost for expensive interventions. **Transition:** stochastic and unknown, so this is model-free and likely offline. γ : tied to the business horizon (monthly vs. annual value). The hard parts are that the reward is delayed and the data is observational, which pushes toward offline RL and off-policy evaluation rather than online exploration on real customers.

Q7

Why can a small change to the reward function completely change the optimal policy, and why is that dangerous?

Because the optimal policy is defined entirely by the reward: it is $\arg \max$ over expected discounted reward. Two reward functions that look similar can induce wildly different optima if their relative magnitudes shift the trade-off between sub-goals. This is dangerous because reward is usually hand-designed, and the agent will exploit *exactly* what you wrote, not what you meant, the boat-racing agent that loops collecting power-ups instead of finishing the race is the canonical example. The mitigation is potential-based reward shaping (Ng et al., 1999), which provably preserves the optimal policy while adding guidance.

Q8

In our GridWorld the dynamics are stochastic (a 10% slip). How does that change the optimal policy versus the deterministic case?

In the deterministic case the optimal policy is the shortest path, and it can safely run adjacent to the pit. Add a slip probability and the same path now has a real chance of slipping *into* the pit, whose large negative reward dominates the expectation. The optimal policy therefore detours away from the cliff edge, accepting a longer expected path to reduce catastrophic-outcome risk. This is risk sensitivity emerging purely from expectation maximization under stochastic dynamics, and it is exactly what makes the value-iteration solution in Chapter 2 more interesting than a graph shortest-path.

Q9

What is the difference between episodic and continuing tasks, and how does each handle the return?

Episodic tasks have terminal states (a game ends, an episode resets); the return is a finite sum to termination, so $\gamma = 1$ is sometimes safe. Continuing tasks run forever; the return must be discounted ($\gamma < 1$) to stay finite, or reformulated with the average-reward criterion. The practical trap appears with time-limit truncation: when an episode is cut off by a step

limit rather than a true terminal, you must *not* zero out the bootstrap term, because the task did not actually end. Our DQN and PPO code in Chapters 5 and 8 handle this by storing `terminated` (true MDP terminal) separately from `truncated` (time limit).

Q10

Your state space is huge and continuous. What are your options, and what do you lose with each?

Options: (1) *discretization* or tile coding, simple but suffers the curse of dimensionality and loses smoothness across cell boundaries; (2) *linear function approximation* over hand-engineered features, stable and analyzable but limited by feature quality; (3) *deep function approximation*, expressive and the modern default, but it forfeits convergence guarantees and exposes you to the deadly triad (Chapter 5). The judgment being tested is that there is no free lunch: every step up in representational power trades away a guarantee or adds an instability you now have to manage.

References

- Bellman, R. (1957). *Dynamic programming*. Princeton University Press.
- Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning* (pp. 278–287). Morgan Kaufmann.
- Puterman, M. L. (1994). *Markov decision processes: Discrete stochastic dynamic programming*. Wiley.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

Dynamic Programming

Subscribe → aiengineeringinsider.substack.com/subscribe

Policy iteration, value iteration, and the convergence guarantees that deep RL gives up.

2.1 Overview

Dynamic programming (DP) methods assume full knowledge of the MDP, the transition function P and reward function R are known, and compute exact value functions and optimal policies through repeated application of the Bellman equations. No real-world system has a fully known model, so DP is never directly deployed in production, yet it is the conceptual root of nearly everything that follows: TD learning, Q-learning, and policy gradients are all approximations of DP operations when the model is unknown (Sutton & Barto, 2018).

The core concept is **bootstrapping with a known model**: DP estimates a state's value from the estimated values of its successors, weighted by the known transition probabilities. Two ideas drive the whole chapter. **Policy evaluation** computes V^π for a fixed policy by iterating the Bellman expectation equation to convergence. **Policy improvement** constructs a weakly better policy by acting greedily with respect to V^π . The **policy improvement theorem** guarantees the new policy is at least as good, and strictly better unless both are already optimal, which is what makes the alternation converge.

Generalized Policy Iteration (GPI)

Almost every RL algorithm is an instance of GPI: maintain a value estimate and a policy, repeatedly make the value more consistent with the policy (evaluation) and the policy greedier with respect to the value (improvement). DP does both exactly; Monte Carlo and TD do both approximately from samples. Seeing later algorithms as “GPI with one piece approximated” is the mental model interviewers reward.

2.2 Policy Iteration vs. Value Iteration

Policy iteration runs evaluation to convergence before each improvement step. **Value iteration** truncates evaluation to a single sweep and folds improvement in via the Bellman optimality backup, $V_{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) [R + \gamma V_k(s')]$. They reach the same optimum; the choice is about cost per iteration versus number of iterations. Figure 2.1 shows both loops.

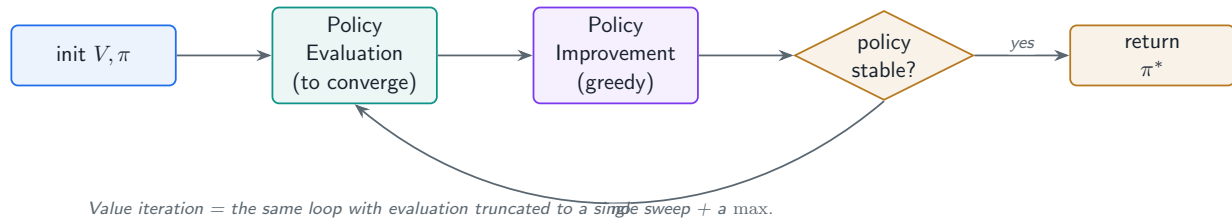


Figure 2.1: Policy iteration as generalized policy iteration. Value iteration collapses the inner evaluation loop into one optimality backup.

★ Key Insight

The reason DP does not scale: both methods sweep the entire state space every iteration and require P and R exactly. Neither holds in any state space large enough to be interesting. Every later chapter is, in some sense, an answer to “how do we approximate DP when we lack a model and cannot enumerate states.”

2.3 Bellman Optimality as a Contraction

The Bellman optimality operator T is a γ -contraction in the max-norm: $\|TV - TV'\|_\infty \leq \gamma\|V - V'\|_\infty$. By the Banach fixed-point theorem, repeated application converges geometrically to the unique fixed point V^* , at rate γ . This is the theoretical bedrock under TD learning’s convergence proofs, and worth stating precisely if a research-track interview pushes into theory.

2.4 The Project: Planning on GridWorld

`rl_interview_lab/classical/dynamic_programming.py` implements both algorithms against the `transitions` model from Chapter 1. On the default 4×4 GridWorld, policy iteration converges in 3 policy updates and value iteration in 15 sweeps, to the *same* policy, a concrete demonstration of the cost trade-off.

dynamic_programming.py — the value-iteration backup

```

def value_iteration(env, gamma=0.95, theta=1e-8):
    """Collapse evaluation + improvement into one Bellman optimality backup."""
    V = np.zeros(env.n_states)
    for iteration in range(1, 100000):
        delta = 0.0
        for s in range(env.n_states):
            if env.is_terminal(s):
                continue
            # One-step look-ahead over every action, then take the max.
            q_sa = [
                sum(t.prob * (t.reward + gamma * V[t.next_state]))
                for t in env.transitions(s, a)
                for a in range(env.n_actions)
            ]
            v_new = max(q_sa)  # Bellman OPTIMALITY backup
  
```

```

        delta = max(delta, abs(v_new - V[s]))
        V[s] = v_new
    if delta < theta:                                # sup-norm contraction => stop
        break
    policy = _greedy_policy(env, V, gamma)
    return policy, V, iteration

```

```
python -m rl_interview_lab.classical.dynamic_programming
```

```

Policy iteration converged in 3 policy updates
Value iteration converged in 15 sweeps
Policies identical: True
Optimal policy:
> | > | > | G
^ | # | ^ | X
^ | # | ^ | <
^ | > | ^ | <

```

Read the policy map and notice the agent in the bottom-right does *not* march straight up toward the pit (X); the 10% slip makes that risky, so value iteration routes around it. That risk-aware detour is the payoff of maximizing expectation under stochastic dynamics, and it is the ground-truth policy the model-free chapters will try to recover from samples alone.

From the Field

If you remember one thing from this chapter for an interview: DP is the answer key. Every deep RL algorithm you discuss later is a principled way of approximating DP's exact updates when you lack a model, cannot enumerate states, and cannot sweep them all. Framing your answers as approximations to a known-correct procedure, rather than independently invented tricks, reads as genuine understanding.

2.5 Comparison: DP Methods

Dimension	Policy Iteration	Value Iteration	Async / Prioritized Sweeping
Inner loop	full evaluation	one sweep	update one state at a time
Iterations to converge	few (outer)	more (sweeps)	varies
Cost per iteration	high	low	lowest per update
Backup used	expectation, then max	max each sweep	max, ordered by need
Best when	evaluation cheap, actions many	the usual default	huge state space, focus matters

Table 2.1: All three converge to V^* ; they differ only in how they schedule the same Bellman backups.

2.6 Mock Interview Questions

Q1

What is the difference between policy iteration and value iteration, and when would you prefer one?

Policy iteration runs evaluation to convergence (an inner loop that can take many sweeps) before a single improvement step; value iteration truncates evaluation to one sweep using the Bellman optimality backup. They converge to the same optimal value function, so the choice is purely computational: value iteration is the usual default because each iteration is cheap; policy iteration tends to need fewer *outer* iterations but each is far more expensive. The honest framing is that “which is better” is about cost per iteration versus number of iterations, not solution quality.

Q2

Prove, or at least argue, why value iteration converges.

The Bellman optimality operator T is a γ -contraction in the max-norm. For any two value functions, $\|TV - TV'\|_\infty \leq \gamma\|V - V'\|_\infty$, which follows because the max is non-expansive and the discounting multiplies the difference by $\gamma < 1$. The Banach fixed-point theorem then guarantees a unique fixed point and geometric convergence to it from any start. The error after k iterations is bounded by γ^k times the initial error, so smaller γ converges faster, another reason the discount factor is not just a modelling choice.

Q3

What is the “deadly triad,” and why is DP immune to it?

The deadly triad is the combination of function approximation, bootstrapping, and off-policy learning. Each alone is fine; together they can diverge, with no general convergence guarantee. DP bootstraps but uses an exact tabular representation (no function approximation) and is on-policy by construction, so it sits safely outside the triad. Deep Q-learning sits squarely inside it, which is why DQN needs target networks and replay as stabilizers rather than relying on theory. Recognising that deep RL’s instability is a *structural* consequence of dropping DP’s assumptions, not a patchable bug, is the senior-level signal.

Q4

Policy iteration’s inner evaluation loop is expensive. How would you speed it up without breaking convergence?

Several principled options: (1) *Modified policy iteration*, stop evaluation after k sweeps instead of to convergence; with $k = 1$ this is exactly value iteration, so it is a tunable interpolation. (2) *In-place (Gauss–Seidel) updates*, overwrite $V[s]$ as you go so later states in the sweep use fresher values; this typically converges faster and is what our code does. (3) *Prioritized sweeping*, update the states whose values are changing most first. All preserve convergence because they are still applying the same contraction, just on different schedules.

Q5

Why does DP need the full model, and what is the minimal change that removes that requirement?

DP's backup sums over $P(s' | s, a)$ and uses $R(s, a, s')$ explicitly, so it literally cannot run without the model. The minimal change is to replace that expectation with a *sample*: instead of summing over all s' , take the transition you actually observed. That single substitution turns the Bellman expectation backup into the TD(0) update and the optimality backup into Q-learning. So model-free RL is not a different paradigm; it is DP with the model expectation replaced by sampling, which is exactly how Chapters 3 and 4 develop it.

Q6

Your MDP has 10^{12} states. DP is off the table, what stays useful from this chapter?

The *concepts*, not the algorithms. GPI still structures the solution. The contraction view still tells you bootstrapping should converge in the tabular limit. Most practically, you switch to *sample-based* backups (TD, Q-learning) plus *function approximation* to generalize across the unenumerable state space, and you accept that you have now entered the deadly triad and must manage stability. Real-time DP and prioritized sweeping also survive as ideas: only back up states the agent actually visits or that matter most, rather than sweeping all 10^{12} .

Q7

What is the policy improvement theorem and why does it guarantee monotone progress?

It states that if a new policy π' is greedy with respect to V^π , then $V^{\pi'}(s) \geq V^\pi(s)$ for all s , with strict improvement somewhere unless π is already optimal. The proof unrolls the greedy choice one step at a time, each step substituting an action at least as good as π 's. Because there are finitely many deterministic policies and each improvement is strict until optimality, policy iteration must terminate at the optimum in finite time. This is the theorem that makes “evaluate, then act greedily” a safe loop rather than a heuristic.

Q8

In our GridWorld value iteration took 15 sweeps but policy iteration only 3 updates. Reconcile that.

They count different units. Policy iteration's 3 “updates” each contain a *full* evaluation (many internal sweeps), so its total work is more than 3 sweeps; it just needs few greedy re-pointings because the greedy policy stabilizes quickly on a small grid. Value iteration's 15 “sweeps” are each a single cheap pass. On this tiny MDP both are instant, but the comparison illustrates the general rule: policy iteration amortizes into a few expensive outer steps, value iteration into many cheap ones. Neither is uniformly faster in wall-clock terms.

Q9

How does the discount factor interact with convergence speed and numerical conditioning in DP?

Convergence is geometric at rate γ , so γ near 1 (e.g. 0.999) needs far more iterations and amplifies any error in P or R , because the effective horizon $\frac{1}{1-\gamma}$ blows up. Practically, large γ also makes value estimates large and closely spaced, which hurts the precision of the greedy $\arg \max$ on noisy estimates. If a problem genuinely needs a long horizon, options include average-reward formulations or careful reward scaling. “Just set $\gamma = 0.9999$ ” is a common naive answer that ignores both the iteration-count and conditioning costs.

Q10

Asynchronous DP updates states in arbitrary order. Does that still converge, and why would you want it?

Yes, asynchronous DP converges to V^* as long as every state continues to be updated infinitely often, because each individual backup is still a contraction step and the fixed point is unique. You want it because it lets you focus computation where it matters, on states the agent actually visits, or on states with large Bellman error (prioritized sweeping), instead of wasting sweeps on irrelevant regions. This is the bridge to real-time DP and, conceptually, to why replay buffers in DQN sample some transitions more than others.

References

- Bellman, R. (1957). *Dynamic programming*. Princeton University Press.
- Bertsekas, D. P. (2012). *Dynamic programming and optimal control* (4th ed., Vol. 2). Athena Scientific.
- Howard, R. A. (1960). *Dynamic programming and Markov processes*. MIT Press.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

Monte Carlo Methods

Subscribe → aiengineeringinsider.substack.com/subscribe

Learning from complete episodes, no model required, at the cost of variance.

3.1 Overview

Monte Carlo (MC) methods estimate value functions by averaging actual observed returns from sampled episodes, with no model of the environment required. This is the first method in this book that learns purely from experience, and the first time the bias–variance trade-off becomes the central design question rather than a side concern.

The core concept is direct: the value of a state is the expected return from it, so *sample* returns and average them. Because MC averages real, complete returns $G_t = \sum_k \gamma^k R_{t+k+1}$, its estimate is **unbiased** by construction. The price is **variance**: the target depends on every reward and transition until the episode ends. MC also requires **episodic** tasks, you cannot average a return that never terminates, and it cannot update mid-episode, both of which Chapter 4’s TD methods relax.

Return

$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$. Every value-based method in this book is, at heart, an attempt to estimate $\mathbb{E}[G_t \mid s_t = s]$ without literally summing a long series every time.

3.1.1 First-visit vs. every-visit, and exploring starts

First-visit MC averages the return following only the first occurrence of a state in an episode; **every-visit MC** averages after every occurrence. Both converge to V^π , but first-visit estimates are unbiased with independent samples, while every-visit samples are correlated within an episode. For *control* (finding a good policy, not just evaluating one) MC must also solve exploration: the **exploring starts** assumption begins episodes from random state–action pairs so every pair is sampled, and the practical alternative is an ϵ -soft behaviour policy.

3.2 Architecture: the MC Control Loop

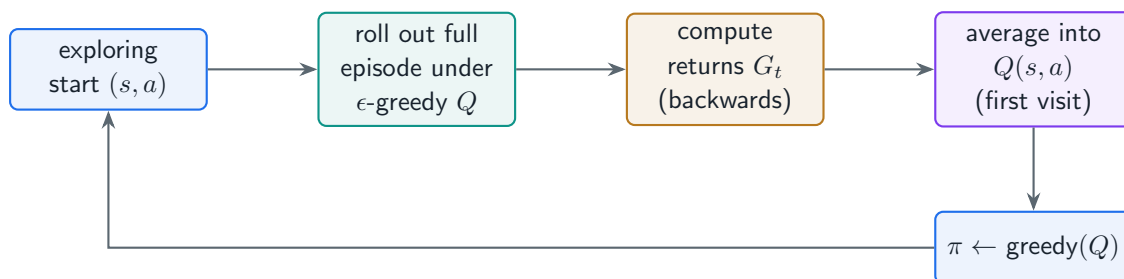


Figure 3.1: Monte Carlo control as generalized policy iteration: full-episode rollouts feed return averages into Q , which is improved greedily, then re-rolled.

3.3 The n-Step Spectrum

MC and one-step TD are the two ends of a continuum, not rival philosophies. An n -step return $G_t^{(n)} = R_{t+1} + \dots + \gamma^{n-1}R_{t+n} + \gamma^n V(s_{t+n})$ bootstraps after n steps instead of after one (TD) or never (MC, i.e. $n = \infty$ within an episode). $TD(\lambda)$ averages over all n with exponentially decaying weights via a single $\lambda \in [0, 1]$, a continuous dial from high-bias/low-variance (small λ) to low-bias/high-variance (large λ).

★ Key Insight

Drawing this spectrum, MC at one end, one-step TD at the other, n -step and $TD(\lambda)$ interpolating, is the cleanest way to show you see bias-variance as a tunable dial rather than a binary choice between unrelated algorithm families.

3.4 The Project: MC Control on GridWorld

`rl_interview_lab/classical/monte_carlo.py` learns Q (not V) because, without a model, only Q makes the greedy step model-free. It uses exploring starts and an incremental running mean so no list of returns is retained per state.

Project Code — `classical/monte_carlo.py`

Note the update is a running mean, $Q += (G - Q)/N$, not a stored list. This is the same incremental-averaging trick that, with a constant step size α instead of $1/N$, becomes the TD update in the next chapter. The transition from MC to TD is literally a one-line change in how you average.

`monte_carlo.py` — first-visit MC control, epsilon-greedy

```
def mc_control_epsilon_greedy(env, gamma=0.95, episodes=50000,
                              epsilon=0.1, seed=0):
    rng = np.random.default_rng(seed)
    Q = np.zeros((env.n_states, env.n_actions))
```

```

counts = np.zeros((env.n_states, env.n_actions))
for _ in range(episodes):
    trajectory = _rollout_q(env, Q, epsilon, rng)  # full episode
    G = 0.0
    visited = set()
    for s, a, r in reversed(trajectory):          # backwards pass
        G = gamma * G + r
        if (s, a) not in visited:                # first-visit only
            visited.add((s, a))
            counts[s, a] += 1
            Q[s, a] += (G - Q[s, a]) / counts[s, a]  # incremental mean
    return np.argmax(Q, axis=1), Q

```

The backwards accumulation of G is the standard trick for computing every return-to-go in one $O(T)$ pass instead of $O(T^2)$. Running this to convergence recovers essentially the same greedy policy that value iteration found in Chapter 2, but with one crucial difference: MC never touched `env.transitions`. It learned the risk-aware detour around the pit purely from sampled outcomes, which is exactly what a real agent with no model must do.

! Pitfall / Warning

A subtle, common trap: MC requires episodic (or artificially truncated) tasks. Asked to apply MC to a genuinely continuing task, the honest answer is that you cannot, not without imposing artificial episode boundaries, which changes the problem you are solving. If forced, you switch to TD, which updates online.

3.5 Comparison: MC vs. TD vs. DP

Property	Monte Carlo	TD(0)	Dynamic Programming
Needs a model?	no	no	yes
Bootstraps?	no	yes	yes
Bias of target	unbiased	biased (early)	exact
Variance of target	high	low	none
Updates when?	end of episode	every step	every sweep
Works on continuing tasks?	no	yes	yes

Table 3.1: MC and DP are opposite corners (sample-vs-model, no-bootstrap-vs-bootstrap); TD inherits the useful half of each.

3.6 Mock Interview Questions

Q1

Explain the bias–variance trade-off between Monte Carlo and TD learning.

MC uses the actual complete return G_t as its target, an unbiased estimate of $V^\pi(s_t)$, but because it depends on every reward until termination it has high variance, especially in long or stochastic episodes. TD bootstraps with $R_{t+1} + \gamma V(s_{t+1})$, substituting an estimate for the rest of the trajectory; this introduces bias (since $V(s_{t+1})$ is imperfect early) but sharply reduces variance, the target depends on one transition, not the whole future. Consequences: MC needs complete episodes and more samples to a given accuracy in noisy environments; TD updates every step and usually converges faster, at the cost of a bias that vanishes only as the estimates converge. This single trade-off, restated many ways, is one of the most-asked theory questions in RL interviews.

Q2

First-visit vs. every-visit MC, are they both correct, and which would you use?

Both converge to V^π . First-visit averages the return after the first occurrence of a state per episode, giving independent, unbiased samples; every-visit averages after every occurrence, which is biased for finite samples (the within-episode visits are correlated) but consistent and often lower-variance in practice. First-visit is the textbook-clean choice with the cleanest statistical story; every-visit is simpler to implement when states recur and tends to be what function-approximation versions effectively do. I would default to first-visit for analysis clarity and not lose sleep over the difference in a large-sample regime.

Q3

Why does MC control learn Q instead of V ?

Because acting greedily from V requires a model: $\arg \max_a \sum_{s'} P(s' | s, a) [r + \gamma V(s')]$ needs P . MC is model-free, so it has no P to plug in. Learning $Q(s, a)$ sidesteps this: the greedy action is just $\arg \max_a Q(s, a)$, computable directly from the table. This is the same reason Q-learning, SARSA, and DQN all learn action-values. The general principle: whenever you are model-free and need to act, estimate Q , not V .

Q4

What problem do exploring starts solve, and what do you do when you cannot use them?

MC control only improves the value of state–action pairs it actually visits. If the current greedy policy never tries action a in state s , $Q(s, a)$ never updates and a better action can stay hidden, a coverage problem. Exploring starts force every (s, a) to begin some episodes, guaranteeing coverage. In most real environments you cannot choose the start state, so you instead use an ϵ -soft policy that keeps a floor probability on every action, ensuring continual exploration. Our code uses both: random starts in evaluation and ϵ -greedy in control.

Q5

Walk through how off-policy MC works and why its variance can explode.

Off-policy MC evaluates a target policy π using episodes from a behaviour policy b . To correct the distribution mismatch you weight each return by the importance ratio $\rho = \prod_t \frac{\pi(a_t|s_t)}{b(a_t|s_t)}$. The estimator is unbiased (ordinary IS) but its variance can be enormous because the ratio is a *product* over the whole episode, one unlikely action under b can send ρ to huge values, and the variance can even be infinite. Weighted importance sampling reduces variance (at the cost of bias) by normalizing the weights. This product-of-ratios blow-up is exactly why long-horizon off-policy correction is hard and why offline RL prefers other tools.

Q6

How does the n -step return connect MC and TD, and how do you pick n ?

The n -step return bootstraps after n rewards: $n = 1$ is TD(0), $n = \infty$ (to episode end) is MC. Increasing n trades lower bias for higher variance. You pick n to match the credit-assignment horizon: short n for dense, immediate rewards; longer n when reward is delayed and you need to propagate it faster. Empirically, intermediate values (e.g. $n = 3$ to 5) often beat both extremes, which is why n -step returns are one of Rainbow DQN's most impactful additions (Chapter 5). TD(λ) avoids committing to a single n by averaging over all of them.

Q7

Your MC value estimates are noisy and slow to converge in a highly stochastic environment. What concretely do you change?

The diagnosis is variance. Levers, roughly in order: (1) move toward TD or n -step returns to cut target variance via bootstrapping; (2) add a baseline / use advantages if this is a policy-gradient setting; (3) increase the number of episodes or use variance-reduction like control variates; (4) reduce γ if the effective horizon is longer than the task actually needs, since a long horizon multiplies variance. The wrong answer is “train longer” alone, MC's variance can make that prohibitively slow, which is precisely why TD methods exist.

Q8

Why is the incremental-mean update important in practice, and how does it relate to the TD step size?

Storing every return per state to average them is $O(\text{visits})$ memory and clumsy online. The incremental mean $Q \leftarrow (G - Q)/N$ gives the identical average in $O(1)$ memory. Replace the $1/N$ step with a *constant* α and you get an exponentially-weighted average that tracks a non-stationary target, which is precisely the TD update $Q \leftarrow Q + \alpha(\text{target} - Q)$. So the MC-to-TD jump is, mechanically, “swap $1/N$ for α and swap the full return for a bootstrapped target.” Showing this connection signals you understand both as instances of stochastic approximation.

Q9

Can MC be used with function approximation, and what breaks?

Yes, MC with function approximation is actually *more* stable than TD in one important sense: because its target (the true return) does not depend on the current parameters, it

performs genuine gradient descent on a well-defined objective and avoids the deadly triad's bootstrapping leg entirely. What you lose is sample efficiency, the high variance of full returns makes gradients noisy, so it typically needs many more episodes than a bootstrapped method. So MC trades the triad's instability for variance-driven slowness, a trade that is sometimes worth it when stability matters more than speed.

Q10

In our GridWorld, MC control recovered nearly the same policy as value iteration without the model. What does that demonstrate, and where would it fail?

It demonstrates that sampling can substitute for the model's expectation: averaging enough real returns approximates $\mathbb{E}[G_t]$ that DP computes exactly. It works here because the grid is small, episodes are short, and exploring starts give full coverage. It would fail as the state space grows (tabular Q becomes infeasible, forcing function approximation), as episodes lengthen (variance balloons), or in continuing tasks (no episode boundary to compute a return). Each failure mode is a reason a later chapter exists, function approximation (Ch. 5), TD bootstrapping (Ch. 4), and policy-gradient variance reduction (Ch. 6–7).

References

- Robbins, H., & Monro, S. (1951). A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3), 400–407.
- Singh, S. P., & Sutton, R. S. (1996). Reinforcement learning with replacing eligibility traces. *Machine Learning*, 22(1–3), 123–158.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

Q-Learning & SARSA

Subscribe → aiengineeringinsider.substack.com/subscribe

The on-policy / off-policy distinction, made concrete by the two algorithms that differ in a single term.

4.1 Overview

Q-learning and SARSA are temporal-difference (TD) control methods that learn an action-value function $Q(s, a)$ rather than a state-value function, which lets you select actions directly without a model ($\arg \max_a Q(s, a)$ needs no transition function). They differ in exactly one place, the target used in the TD update, and that one difference is the cleanest possible illustration of **on-policy** versus **off-policy** learning.

The core concept is **bootstrapping from sampled transitions**: take the DP backup, replace its expectation over s' with the single transition you actually observed, and update toward it with a step size α . SARSA bootstraps from the action the policy *actually takes* next; Q-learning bootstraps from the *best* next action. That is the entire distinction, and everything else, safety, sample efficiency, overestimation, follows from it.

4.1.1 The update rules, side by side

SARSA (on-policy):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)].$$

Q-learning (off-policy):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)].$$

The bracketed quantity is the **TD error** δ_t , the surprise between prediction and bootstrapped target.

4.2 Architecture: Where the Two Targets Diverge

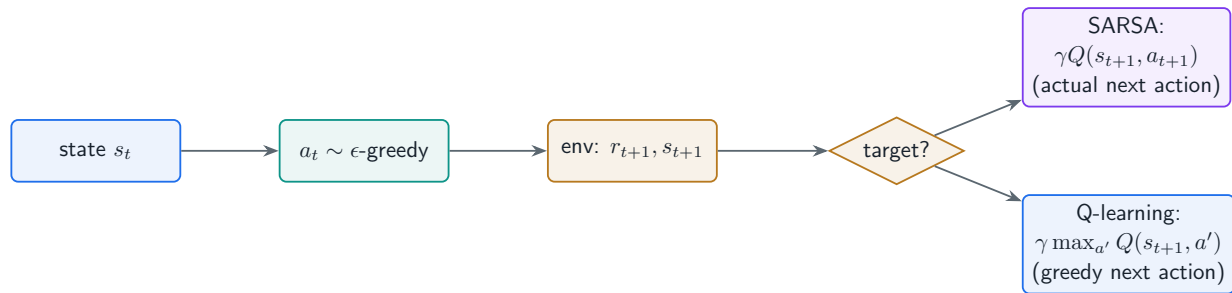


Figure 4.1: Identical data collection; the only difference is which next-action value enters the bootstrap. On-policy uses what the agent did; off-policy uses what is best.

4.3 Convergence and Overestimation Bias

Both methods converge to Q^* in the tabular case under the Robbins–Monro conditions ($\sum \alpha_t = \infty$, $\sum \alpha_t^2 < \infty$) with sufficient exploration. Q-learning carries a well-known **overestimation bias**: because the max is applied to noisy estimates, $\mathbb{E}[\max_a Q(s, a)] \geq \max_a \mathbb{E}[Q(s, a)]$, so learned values skew optimistic. That single observation motivates Double Q-learning and, later, Double DQN (van Hasselt, 2010; van Hasselt et al., 2016).

4.4 The Project: Q-Learning vs. SARSA on GridWorld

`rl_interview_lab/classical/td_control.py` implements both. The bodies are identical except for the target line, the best way to internalize that the on/off-policy distinction is one expression, not two algorithms.

td_control.py — the only line that differs between the two

```

# Q-LEARNING: bootstrap from the greedy next action (off-policy)
td_target = r + (0.0 if done else gamma * np.max(Q[s2]))
Q[s, a] += alpha * (td_target - Q[s, a])

# SARSA: bootstrap from the action actually chosen next (on-policy)
a2 = _epsilon_greedy(Q, s2, epsilon, rng)
td_target = r + (0.0 if done else gamma * Q[s2, a2])
Q[s, a] += alpha * (td_target - Q[s, a])
  
```

python -m rl_interview_lab.classical.td_control

```

Q-learning mean return (last 500 eps): +0.724
SARSA      mean return (last 500 eps): +0.727
  
```

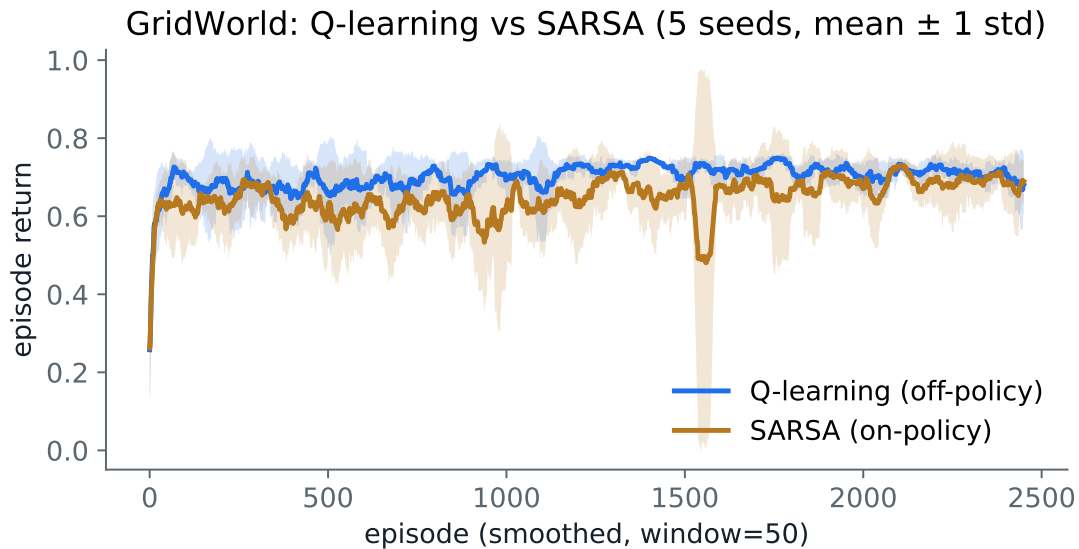



Figure 4.2: Learning curves over 5 seeds (shaded band = ± 1 standard deviation across seeds), generated by `scripts/make_plots.py`. Both methods converge to similar return on this slippery grid; the value of the plot is showing convergence *and* seed variance, not a single cherry-picked run.

On this grid the slip penalty already pushes both toward a cautious policy, so their returns are close. The textbook divergence appears on *cliff-walking*, where Q-learning learns the optimal cliff-edge path (its target assumes greedy future behaviour) while SARSA learns a safer inland path (its target accounts for the real chance that exploration steps it off the edge). The lesson generalizes: when exploration during deployment is genuinely risky, SARSA’s on-policy conservatism can be the responsible choice even at a cost to asymptotic optimality.

★ Key Insight

Expected SARSA replaces the sampled $Q(s_{t+1}, a_{t+1})$ with the expectation $\sum_a \pi(a | s_{t+1})Q(s_{t+1}, a)$, removing the variance from sampling a_{t+1} while staying on-policy. It is the answer to “how would you reduce SARSA’s variance without making it off-policy.”

4.5 Comparison Table

Property	SARSA	Q-learning	Expected SARSA
Policy class	on-policy	off-policy	on-policy
Next-action value	sampled a_{t+1}	$\max_{a'}$	$\mathbb{E}_{\pi}[\cdot]$
Learns value of	behaviour policy	greedy policy	behaviour policy
Behaviour near hazards	cautious	risk-seeking	cautious
Reuse of old data	no	yes (replay)	limited
Overestimation bias	low	yes	low

Table 4.1: One column controls all the downstream behaviour: which next-action value enters the bootstrap.

4.6 Mock Interview Questions

Q1

What is the difference between on-policy and off-policy learning, and why does importance sampling matter?

On-policy methods evaluate and improve the same policy that generates the data; off-policy methods learn about a target policy from data generated by a different behaviour policy. SARSA is on-policy: its update uses a_{t+1} , the action actually taken under the current ϵ -greedy policy, so it learns the value of the policy it follows, exploration included. Q-learning is off-policy: it uses $\max_{a'} Q(s_{t+1}, a')$ regardless of what is taken next, so it learns about the greedy policy while behaving exploratorily. Off-policy is more sample-efficient (you can reuse old data, the basis of replay) but creates a distribution mismatch. Importance sampling reweights samples by the ratio of target to behaviour probabilities to correct it; its variance explodes when the policies diverge, which is the central tension offline RL must resolve.

Q2

Walk through SARSA vs. Q-learning on cliff-walking, where do they diverge and why?

In cliff-walking the agent crosses a grid with a cliff that incurs a large penalty. Under ϵ -greedy, SARSA learns a path that hugs the safe interior, because its updates account for the real chance the next action is a random exploratory step into the cliff. Q-learning learns the optimal path right along the edge, because its max target assumes greedy behaviour from the next state on, even while the agent still explores. So SARSA's learned policy is safer but lower-return under the true objective; Q-learning's is optimal but riskier *during* learning. This is the concrete reason safety-constrained production systems sometimes prefer SARSA's conservatism despite its suboptimality.

Q3

Why does Q-learning overestimate action values, and how does Double Q-learning fix it?

The max in the target selects the highest-*estimated* action and evaluates that same noisy estimate. Because noise makes some estimate high by chance, the selected action's value is biased upward, and that optimistic value propagates into the target, maximization bias, not an implementation bug. Double Q-learning keeps two independent estimates Q_A, Q_B : select with one, evaluate with the other (update Q_A using $Q_B(s', \arg \max_a Q_A(s', a))$). Because selection and evaluation use independently-noisy estimators, the systematic upward bias cancels. This exact idea reappears as Double DQN with neural nets standing in for Q_A, Q_B .

Q4

What is the TD error, and why is it considered a biologically and algorithmically central quantity?

The TD error $\delta_t = r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)$ (or with a max for Q-learning) is the difference between the bootstrapped target and the current estimate, the “surprise.” Algorithmically it is the entire learning signal: every TD update is $Q \leftarrow Q + \alpha \delta$. It also reappears as the advantage signal in actor-critic (Chapter 7) and inside GAE in PPO (Chapter 8). Biologically, dopamine neuron firing closely tracks a reward-prediction error, which is why TD learning is a celebrated link between RL and neuroscience. Knowing δ unifies value-based and policy-gradient methods is a strong cross-chapter signal.

Q5

Under what conditions does tabular Q-learning provably converge, and which condition is hardest to satisfy in practice?

It converges to Q^* with probability 1 if every state–action pair is visited infinitely often and the step sizes satisfy Robbins–Monro ($\sum \alpha_t = \infty, \sum \alpha_t^2 < \infty$). The infinite-visitation (exploration) condition is the hard one: a purely greedy policy can starve actions and never satisfy it, which is why we use ϵ -greedy or optimistic initialization. In large or continuous spaces “visit every pair infinitely often” is impossible outright, so you add function approximation and the convergence guarantee is lost, the deadly triad again. The step-size condition is usually easy; the coverage condition is where real systems struggle.

Q6

How do the learning rate α and exploration rate ϵ interact, and how would you schedule them?

α controls how much each TD error moves the estimate; ϵ controls how much the agent explores. Early on you want high ϵ (gather diverse data) and a moderate α (learn quickly from it); late, you want both low (exploit a near-converged value with stable estimates). A common schedule decays ϵ from ~ 1.0 to a small floor and decays α toward zero to satisfy Robbins–Monro. The interaction trap: decaying ϵ too fast starves exploration before Q is accurate, locking in a suboptimal policy; keeping α high forever means the estimate never settles and chases noise.

Q7

When would you choose SARSA over Q-learning in production, despite Q-learning's optimality?

When the policy will keep exploring or acting under uncertainty *in deployment*, and mistakes are costly. SARSA learns the value of the policy actually run (exploration noise included), so it produces behaviour that is robust to that noise, useful in robotics or any system that cannot freeze to pure greedy at deployment. Q-learning's policy is optimal only under the assumption of greedy future action, which is violated if you must keep exploring online. The general rule: if the deployed behaviour policy equals the evaluated policy, on-policy methods give you what you actually measured.

Q8

Expected SARSA, what is it, and where does it sit relative to SARSA and Q-learning?

Expected SARSA uses $\sum_a \pi(a | s_{t+1})Q(s_{t+1}, a)$ as the next-state value, the expectation over the policy's action distribution, instead of the single sampled $Q(s_{t+1}, a_{t+1})$. This removes the variance from sampling the next action, so it learns more stably than SARSA, often allowing a larger α . It is on-policy when π is the behaviour policy, but it elegantly generalizes: if π is the greedy policy, Expected SARSA becomes Q-learning. So it is a single algorithm with SARSA and Q-learning as special cases, picked by which π you take the expectation under.

Q9

You set $\gamma = 0.99$ and your tabular Q-learning is unstable and slow. What do you check first?

First confirm it is actually the discount: $\gamma = 0.99$ gives an effective horizon of ~ 100 steps, which may exceed what the task needs and inflates variance and the magnitude of value estimates. I would (1) check whether the task horizon truly needs 0.99 or whether 0.9 suffices; (2) verify exploration is adequate (ϵ not decayed too fast); (3) check the learning rate is not so high it oscillates; (4) confirm terminal handling zeros the bootstrap correctly, a classic bug is bootstrapping past a terminal state, which injects phantom future value. Only after those would I suspect the environment's reward scale.

Q10

Q-learning is off-policy, which enables experience replay. Explain that link and one risk it introduces.

Off-policy means the target ($\max_{a'} Q$) does not depend on which policy generated the transition, so a transition collected under an *old* policy is still a valid sample for learning about the greedy policy. That is precisely what lets DQN store transitions in a replay buffer and reuse them many times, decorrelating updates and improving sample efficiency. The risk is distribution shift: as the policy improves, the buffer fills with stale, off-distribution data, and the bootstrapped max can extrapolate over-optimistically to actions the current policy would never take, an effect that becomes severe in fully offline RL and motivates conservative/constrained offline methods.

References

- Rummery, G. A., & Niranjan, M. (1994). *On-line Q-learning using connectionist systems* (Technical Report CUED/F-INFENG/TR 166). Cambridge University Engineering Department.
- van Hasselt, H. (2010). Double Q-learning. In *Advances in Neural Information Processing Systems* (Vol. 23, pp. 2613–2621).
- Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3–4), 279–292.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

Part II

Deep Reinforcement Learning

Deep Q-Networks (DQN)

Subscribe → aiengineeringinsider.substack.com/subscribe

Where the deadly triad stops being theoretical and becomes your Tuesday afternoon.

5.1 Overview

DQN combines Q-learning with a neural-network function approximator $Q(s, a; \theta)$, which is exactly the deadly triad of Chapter 2: function approximation, bootstrapping, and off-policy learning, all at once. The Nature paper (Mnih et al., 2015) needed two specific stabilizing tricks to make this combination work, and understanding *why* each is necessary, not just that it exists, is what separates a strong answer from a memorized one.

The core concept is that a network generalizes Q across states, so you no longer need a table entry per state, which is what lets DQN play Atari from pixels. But generalization is exactly what destabilizes the bootstrap: an update at one state shifts the predicted value at every similar state, including the bootstrap target. **Experience replay** and **target networks** are the two engineering answers to that instability.

★ Key Insight

Neither experience replay nor target networks has a convergence proof attached; they are empirically-validated stabilizers for a setting (the deadly triad) where no general guarantee exists. Saying this plainly in an interview, rather than presenting DQN as a solved, theoretically-grounded method, is exactly the calibrated honesty strong RL answers are built on.

5.2 Architecture: the Two Stabilizers

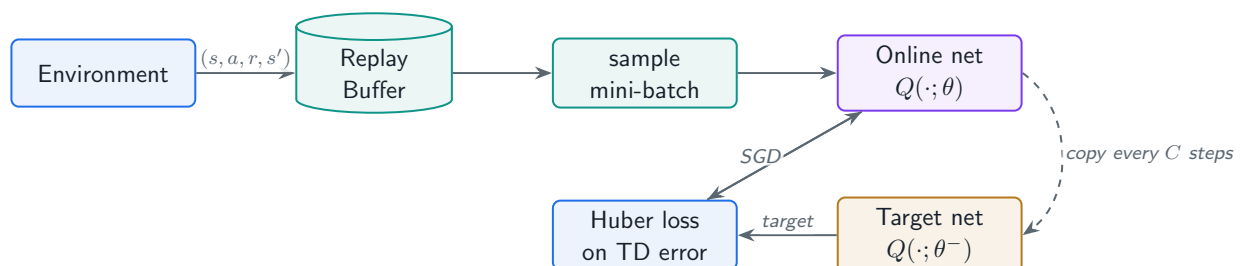


Figure 5.1: DQN data flow. Replay breaks temporal correlation; the slowly-updated target network holds the bootstrap target still.

5.3 The Rainbow Lineage

Several follow-ups each fix one failure mode of vanilla DQN; Rainbow (Hessel et al., 2018) combines them and remains the standard reference for “what improvements matter.” The components are Double DQN (overestimation), Prioritized Replay (sample high-TD-error transitions more), Dueling networks (split V and advantage streams), multi-step returns (faster credit assignment), Distributional RL / C51 (learn the return distribution), and Noisy Nets (learned exploration). The original ablation found prioritized replay and multi-step learning contributed the largest individual gains.

5.4 The Project: DQN with Double-DQN Targets

`rl_interview_lab/deep/dqn.py` implements replay, a hard-synced target network, and a one-line Double-DQN toggle. The loss is the crux of the chapter.

dqn.py — the TD loss, with the Double-DQN target

```
def _compute_loss(self, batch):
    q_pred = self.q(batch.obs).gather(1, batch.actions.unsqueeze(1)).squeeze(1)
    with torch.no_grad():
        if self.cfg.double_dqn:
            # Decouple selection (online net) from evaluation (target net).
            next_actions = self.q(batch.next_obs).argmax(dim=1, keepdim=True)
            next_q = self.q_target(batch.next_obs).gather(1, next_actions).squeeze(1)
        else:
            next_q = self.q_target(batch.next_obs).max(dim=1).values
        target = batch.rewards + self.cfg.gamma * next_q * (1.0 - batch.dones)
    # Huber loss is more robust to the occasional large TD error than MSE.
    return nn.functional.smooth_l1_loss(q_pred, target)
```

Three production details hide in those lines. First, `torch.no_grad()` around the target: gradients must not flow into the bootstrap, or you are differentiating through your own label. Second, `(1.0 - batch.dones)` zeroes the bootstrap at true terminals, and the training loop deliberately stores `terminated`, not `truncated`, so a time-limit cutoff does not get treated as the end of the world. Third, the Huber (`smooth_l1`) loss caps the gradient from rare huge TD errors, a small choice that meaningfully stabilizes training.

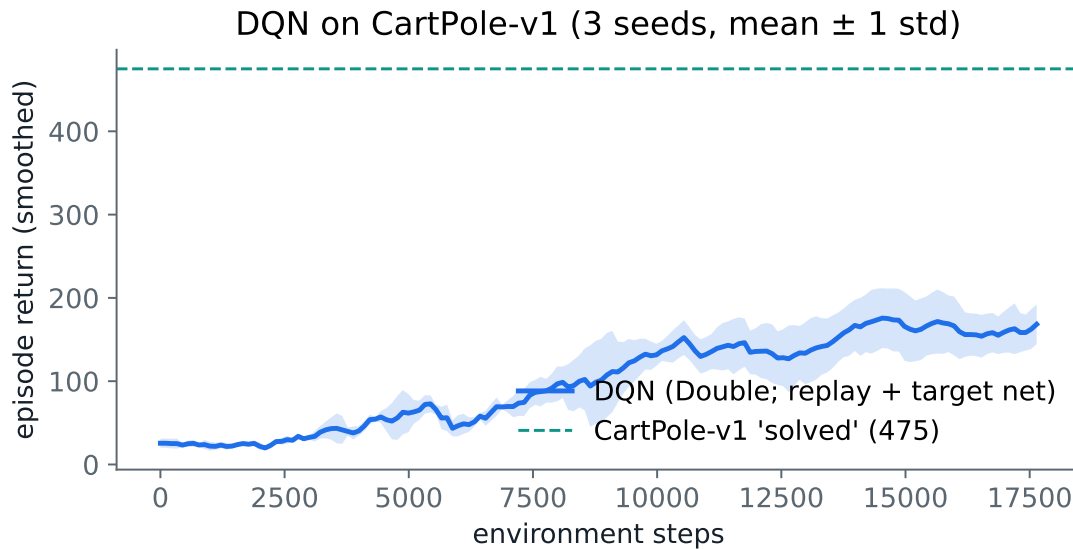


Figure 5.2: DQN on CartPole-v1 over 3 seeds (band = ± 1 std), from `scripts/make_plots.py`. Return climbs steadily but has not yet reached the “solved” line in this short, CPU-budget run; train longer (or add prioritized replay and multi-step returns) to close the gap. Showing the curve with a seed band, rather than one final number, is the honest way to report deep-RL results (Henderson et al., 2018).

! Pitfall / Warning

The single most common DQN bug in interviews and in practice: forgetting to detach the target, so the optimizer “cheats” by moving the target toward the prediction. Symptoms are loss that falls beautifully while returns go nowhere. Always confirm the target is under `no_grad` or `.detach()`.

5.5 Comparison: Vanilla DQN vs. Rainbow Components

Component	Failure mode it fixes	Relative impact (Rainbow ablation)
Double DQN	overestimation from max	moderate
Prioritized replay	rare important transitions under-sampled	large
Dueling net	many actions of similar value	small–moderate
Multi-step returns	slow credit assignment	large
Distributional (C51)	mean target loses signal	moderate
Noisy Nets	blind ϵ -greedy exploration	small

Table 5.1: If asked to improve a DQN baseline, propose prioritized replay and multi-step returns first, highest leverage, lowest complexity.

5.6 Mock Interview Questions

Q1

Why does naive Q-learning diverge when combined with neural networks, and how does DQN fix it?

Two problems compound. First, consecutive states in a trajectory are highly correlated, violating the i.i.d. assumption SGD relies on and causing the net to overfit recent, narrow experience. Second, the target $r + \gamma \max_{a'} Q(s', a'; \theta)$ uses the same rapidly-changing weights being updated, so it moves every gradient step, you are chasing a target that chases you back. Experience replay stores transitions and samples mini-batches uniformly at random, breaking temporal correlation and restoring something close to i.i.d. supervised learning. Target networks use a separate, periodically-synced copy θ^- to compute the target, holding it still for C steps so the network chases a stationary goal.

Q2

What problem does Double DQN solve, and how, concretely?

It removes the overestimation bias from the max. Vanilla DQN both selects and evaluates the next action with the target net, so noise that makes one action's value high gets propagated. Double DQN selects the action with the *online* net ($\arg \max_{a'} Q(s', a'; \theta)$) and evaluates it with the *target* net ($Q(s', \cdot; \theta^-)$). Because selection and evaluation use independently-noisy networks, the systematic upward bias cancels. It is the deep-learning instantiation of Double Q-learning, and crucially it is one line of code, exactly the change shown in the project's `.compute_loss`.

Q3

Explain Rainbow DQN. Which components matter most empirically?

Rainbow combines six extensions: Double DQN (overestimation), Prioritized Experience Replay (sample by TD-error magnitude), Dueling networks (separate $V(s)$ and $A(s, a)$ streams), multi-step returns (n -step targets), Distributional RL / C51 (learn the full return distribution, not just the mean), and Noisy Nets (state-dependent learned exploration). The ablation found prioritized replay and multi-step returns contributed the largest individual gains, with distributional close behind; dueling and noisy nets were smaller but positive. Practical take-away: when asked to improve a DQN baseline, suggest prioritized replay and multi-step returns first.

Q4

Compare ϵ -greedy, UCB, and Thompson sampling for exploration.

ϵ -greedy takes a uniformly random action with probability ϵ , simple but blind, wasting exploration on known-bad actions. UCB picks $\arg \max_a [Q(a) + c\sqrt{\ln t / N(a)}]$, optimism under uncertainty that explores high-uncertainty (low-count) actions and self-anneals as counts grow, no schedule needed. Thompson sampling maintains a posterior over each action's value, samples once from each, and acts greedily on the samples; wider posteriors get explored more. In production, UCB and Thompson (especially contextual-bandit variants) dominate naive ϵ -greedy because they explore proportionally to actual uncertainty. Noisy

Nets bring a learned version of this idea into DQN.

Q5

How do you size the replay buffer and the target-sync interval, and what goes wrong at the extremes?

The buffer trades recency against decorrelation: too small and samples stay correlated and recent (instability, forgetting); too large and you train heavily on stale, off-distribution data from much weaker past policies (slow, biased). Atari-scale defaults are around 10^6 . The target-sync interval C trades stability against staleness: too frequent and the target moves like the online net (the instability you were avoiding); too infrequent and you bootstrap toward badly outdated values, slowing learning. Hard sync every few hundred to few thousand steps is typical; soft (Polyak) updates $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$ are a smoother alternative.

Q6

Why use the Huber loss instead of MSE for the TD error?

MSE's gradient grows linearly with the error, so a single large TD error, common early in training or after a reward spike, produces a huge gradient that can destabilize the network. Huber (smooth-L1) is quadratic for small errors and linear beyond a threshold, so it behaves like MSE near zero but caps the gradient magnitude for outliers. This makes training robust to the occasional large bootstrap error without sacrificing precision on the common small ones. It is a cheap, standard choice in the original DQN and in our implementation.

Q7

In a Gymnasium environment, episodes end by either termination or truncation. Why must DQN treat them differently?

Termination is a true MDP terminal (the pole fell): there is no future, so the bootstrap must be zeroed, target = r . Truncation is an artificial time limit (CartPole's 500-step cap): the process did not actually end, so zeroing the bootstrap would teach the agent that good states near the time limit have no future value, injecting a systematic bias. The correct handling is to store `terminated` as the done flag for the bootstrap and let truncation only reset the episode. Our training loop does exactly this, and getting it wrong is a subtle but real performance bug.

Q8

Your DQN's loss is decreasing but episode return is flat. Walk me through your debugging.

A falling loss with flat return is the classic "learning the wrong thing" signature. I would check, in order: (1) is the target detached? An attached target lets the net minimize loss by chasing itself; (2) is exploration adequate, ϵ not decayed to near zero before Q is good; (3) terminal/truncation handling, am I bootstrapping past terminals; (4) reward scale and clipping, exploding or vanishing targets; (5) target-sync interval too short (instability) or too long (stale). I would also plot Q -value magnitudes: runaway Q indicates the deadly triad biting, suggesting Double DQN, smaller learning rate, or more frequent but smaller target updates.

Q9

When is DQN the wrong choice, and what would you reach for instead?

DQN requires a discrete action space (it maxes over actions) and is sample-hungry. For continuous control (robot torques), DQN does not apply directly, use DDPG/TD3/SAC or PPO (Chapter 8). For very large discrete action spaces (millions of catalog items), the max is intractable, use a policy-gradient or factorized approach. When you have only logged data and cannot explore online, plain DQN's bootstrapped max over-extrapolates, use offline-RL methods with conservatism. And if the problem is a contextual bandit (no state transitions), full DQN is over-engineering. The senior signal is knowing DQN's assumptions and naming the right tool when they break.

Q10

Prioritized experience replay samples high-TD-error transitions more often. What bias does that introduce and how is it corrected?

Sampling non-uniformly changes the distribution the network trains on, which biases the gradient toward high-error transitions, no longer an unbiased estimate of the expected update. PER corrects this with importance-sampling weights $w_i = (1/(N P(i)))^\beta$ applied to each sample's loss, where $P(i)$ is its sampling probability and β is annealed from a small value toward 1 over training so the correction becomes exact late, when it matters most for convergence. The priority itself is usually $|\delta| + \epsilon$ (proportional) or rank-based. The trade-off: PER adds a sum-tree data structure and two hyperparameters (α for how much to prioritize, β for correction) in exchange for often-large sample-efficiency gains.

References

- Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., ... Silver, D. (2018). Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence* (pp. 3215–3222).
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., ... Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2016). Prioritized experience replay. In *International Conference on Learning Representations*.
- van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence* (pp. 2094–2100).

Policy Gradients — REINFORCE

Subscribe → aiengineeringinsider.substack.com/subscribe

Why learn a value function at all, when you could optimize the policy directly?

6.1 Overview

Value-based methods like DQN learn $Q(s, a)$ and act greedily. Policy-gradient methods take a different route: parameterize the policy directly as $\pi_\theta(a | s)$ and optimize θ by gradient ascent on expected return. This is the family underlying actor-critic, PPO, and RLHF, essentially the entire second half of the book, so deriving it cleanly is worth the space.

The core concept is the **policy gradient theorem**: you can estimate the gradient of expected return with respect to policy parameters using only sampled trajectories and the gradient of $\log \pi_\theta$, with no model and no differentiable reward. REINFORCE (Williams, 1992) is the simplest instance, plugging in the Monte-Carlo return G_t as the weight. Its defining weakness, high variance, is what every later policy-gradient method is engineered to reduce.

Log-Derivative Trick

$\nabla_\theta \log f_\theta(x) = \frac{\nabla_\theta f_\theta(x)}{f_\theta(x)}$, rearranged to $\nabla_\theta f_\theta(x) = f_\theta(x) \nabla_\theta \log f_\theta(x)$. This single identity is the hinge on which the entire policy-gradient family turns. Derive it on a whiteboard and you can derive REINFORCE, and from there, with one variance-reduction step, actor-critic.

6.2 Deriving the Policy Gradient

We maximize $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)]$. The trajectory distribution depends on θ , so we cannot differentiate through the expectation directly. Using the log-derivative trick on the trajectory probability and noting that the (unknown) transition dynamics do not depend on θ , the dynamics terms drop out, leaving

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) G_t \right].$$

The intuition: increase the log-probability of actions that led to high return, decrease it for low return, weighted by how good the outcome was. This needs no model, works for discrete or continuous actions, and is exactly why it generalizes beyond DQN.

6.3 Architecture: the REINFORCE Loop

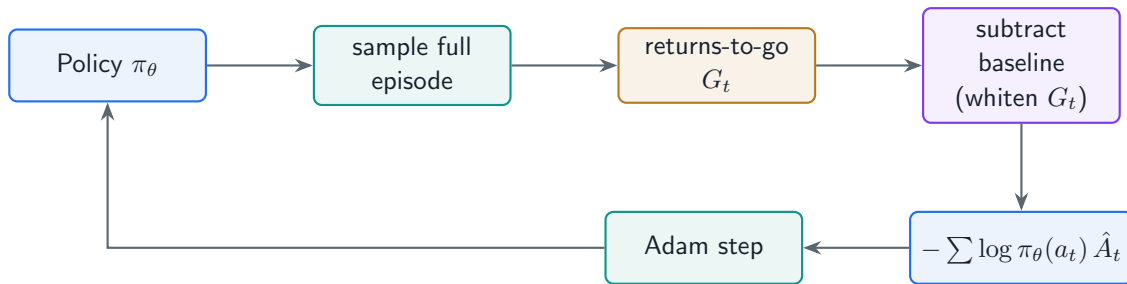


Figure 6.1: REINFORCE: roll out, compute returns, weight log-probabilities, ascend. The baseline box is the only thing standing between this and unusable variance.

6.4 The Variance Problem and the Baseline

REINFORCE’s estimator is unbiased but high-variance, because G_t is a Monte-Carlo sum over the whole trajectory (Chapter 3’s variance, now inside a gradient). The fix that preserves unbiasedness is subtracting a state-dependent **baseline** $b(s_t)$:

$$\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - b(s_t))],$$

valid because $\mathbb{E}_a[\nabla_{\theta} \log \pi_{\theta}(a | s)] = 0$ for any baseline that does not depend on the action. This is the single idea that becomes actor-critic in the next chapter.

! Pitfall / Warning

The most common interview slip: claiming the baseline must be a constant, or that any baseline biases the gradient. The baseline may depend on the *state* (even be a learned $V(s)$); it must only be independent of the *action*. Getting this distinction backwards under pressure is a frequent tell.

6.5 The Project: REINFORCE with a Whitened-Return Baseline

`rl_interview_lab/deep/reinforce.py` uses the simplest possible baseline, standardizing the batch of returns, which is a state-agnostic but effective variance reducer and a clean lead-in to a learned critic.

reinforce.py — the update with a return baseline

```

def train(self):
    for ep in range(self.cfg.episodes):
        log_probs, rewards = self._run_episode(seed=self.cfg.seed + ep)
        returns = discounted_returns(rewards, self.cfg.gamma)
        if self.cfg.use_baseline:
            # Whitening leaves the gradient unbiased (baseline is action-free)
            # but sharply cuts variance, the smallest useful baseline.
            returns = (returns - returns.mean()) / (returns.std() + 1e-8)

```

```

loss = -(log_probs * returns).sum()    # ascend => minimise negative
self.opt.zero_grad()
loss.backward()
self.opt.step()

```

CartPole, 200 episodes — variance reduction visibly helps

REINFORCE first20=42.0 last20=217.4

The negative sign converts gradient *ascent* on return into the *descent* that PyTorch optimizers perform. `discounted_returns` computes every G_t in one backward pass. Even this crude baseline lifts late-training return from the low tens to the low hundreds; a learned critic (next chapter) does substantially better.

6.6 Comparison: Value-Based vs. Policy-Gradient

Dimension	Value-based (DQN)	Policy-gradient (REINFORCE)
What is learned	$Q(s, a)$, policy implicit	$\pi_\theta(a s)$ directly
Action spaces	discrete only	discrete <i>and</i> continuous
Stochastic policies	no (greedy)	yes, natively
Sample efficiency	higher (off-policy replay)	lower (on-policy, MC)
Stability	deadly-triad fragile	unbiased but high variance
Convergence target	can diverge	local optimum of J

Table 6.1: The two families trade off along orthogonal axes; modern methods (actor-critic, PPO) borrow from both.

6.7 Mock Interview Questions

Q1

Explain the policy gradient theorem in your own words, derive it if you can.

We maximize $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)]$, but τ 's distribution depends on θ , so we cannot differentiate through it as in supervised learning. The log-derivative trick, $\nabla_\theta \pi = \pi \nabla_\theta \log \pi$, rewrites the gradient of an expectation as an expectation of a gradient. Expanding the trajectory probability, the environment dynamics terms are independent of θ and vanish, leaving $\nabla_\theta J = \mathbb{E}[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) G_t]$. Intuitively it pushes up the probability of actions that preceded high returns. It needs no model and handles continuous actions, which is why it underpins everything after DQN.

Q2

Why does subtracting a baseline reduce variance without adding bias, and what makes a good baseline?

Unbiasedness holds because $\mathbb{E}_a[\nabla_\theta \log \pi_\theta(a | s) b(s)] = b(s) \mathbb{E}_a[\nabla_\theta \log \pi_\theta] = 0$ for any action-independent $b(s)$, so subtracting it changes the variance but not the mean of the estimator. Variance drops because the baseline centers the weights: actions are reinforced by how much better than average they were, not by the raw (possibly large and noisy) return. The variance-minimizing baseline is roughly the expected return $V(s)$, which is precisely why actor-critic learns $V_\phi(s)$ as the baseline. A poor baseline (e.g. a wildly miscalibrated constant) reduces variance less but still does no harm to bias.

Q3

REINFORCE is unbiased; DQN is biased. Why is REINFORCE still often harder to train?

Because unbiasedness is not the binding constraint, variance is. REINFORCE's full-return weights have variance that grows with horizon and stochasticity, so gradients are noisy and require many episodes and small steps to average out, making it sample-inefficient and slow. DQN's bias (from bootstrapping) is a real cost, but its variance is far lower and it reuses data off-policy via replay, so in practice it learns faster on many tasks. This is the bias-variance trade-off from Chapter 3 reappearing at the level of gradient estimators: the unbiased method is not automatically the better one.

Q4

How would you scale REINFORCE to an action space of millions, like a recommender?

A softmax over millions of items is intractable, and logged data comes from an older policy, so you face off-policy bias too. The known production answer (Chen et al., 2019, at YouTube scale) combines REINFORCE with off-policy correction, importance-weighting each logged action by the ratio of current-policy to logging-policy probability, and a factorized/sampled-softmax policy so training and serving stay tractable. The framing to convey: scaling policy gradients is rarely about the core theorem changing; it is about treating off-policy correction and action-space scale as separate, additive engineering problems on top of the same gradient.

Q5

Walk through how REINFORCE handles a continuous action space.

You replace the categorical policy with a parameterized continuous distribution, typically a diagonal Gaussian whose mean (and optionally log-std) are network outputs: $a \sim \mathcal{N}(\mu_\theta(s), \sigma_\theta(s))$. The update is identical, $\nabla_\theta \log \pi_\theta(a | s)$ is now the Gaussian log-density gradient. Practical caveats: bound or squash actions (e.g. $\tanh$) to respect actuator limits and correct the log-prob for the squashing; and state-independent σ is a common simplification. This is exactly why policy gradients dominate continuous control while DQN cannot apply, there is no tractable $\max_a$ over a continuum.

Q6

What is the credit-assignment problem in REINFORCE, and how does the reward-to-go formulation help?

Naive REINFORCE weights $\nabla \log \pi(a_t)$ by the *total* episode return, which incorrectly credits action a_t with rewards earned *before* it, pure noise from a_t 's perspective. The reward-to-go (causality) trick weights each action only by $G_t = \sum_{k \geq t} \gamma^{k-t} r_{k+1}$, the return from that step onward, since earlier rewards are unaffected by later actions. This is unbiased and lower-variance, and it is what our `discounted_returns` computes. It is a small change with a real effect and a common follow-up once you have stated the basic theorem.

Q7

Why is REINFORCE on-policy, and what does that cost you?

The gradient is an expectation under the *current* policy π_θ , so the data must be generated by that same policy. The moment you update θ , previously collected trajectories are off-distribution and, strictly, can no longer be used without importance-sampling correction. The cost is sample efficiency: you throw away data after each update, unlike DQN's replay. You can partially recover reuse with importance weights, but their variance grows as policies diverge, the same product-of-ratios blow-up from Chapter 3. PPO's clipping (Chapter 8) is essentially a controlled way to reuse recent on-policy data for a few epochs without that blow-up.

Q8

Your REINFORCE training is extremely noisy and barely improving. What concrete levers do you pull?

Diagnosis is variance. Levers: (1) add a baseline, ideally a learned $V(s)$ (i.e. move to actor-critic); (2) use reward-to-go rather than full return; (3) increase batch size, average gradients over more episodes per update; (4) add an entropy bonus to prevent premature determinism that collapses exploration; (5) normalize/whiten returns and check reward scaling; (6) reduce γ if the effective horizon exceeds the task's real horizon. "Lower the learning rate" alone treats the symptom; the structural fix is variance reduction, which is the whole reason the next two chapters exist.

Q9

What role does the entropy bonus play, and what is the failure mode if you omit it?

An entropy bonus adds $+\beta \mathcal{H}(\pi_\theta(\cdot | s))$ to the objective, rewarding the policy for staying stochastic. Without it, policy gradients can collapse to a near-deterministic policy early, before the value estimates are reliable, locking in a suboptimal action and killing exploration (premature convergence). The bonus keeps probability mass on alternatives long enough to discover better actions, then is typically annealed down. This is the discrete-action cousin of SAC's maximum-entropy objective (Chapter 8). Our A2C and PPO configs expose an `entropy_coef` for exactly this reason.

Q10

Compare REINFORCE with a learned baseline to one-step actor-critic. Are they the same thing?

Closely related but not identical. REINFORCE-with-baseline still uses the *Monte-Carlo* return G_t as the target and only *subtracts* a learned $V(s)$ to center it, so the weight is $G_t - V(s)$, unbiased but still MC-variance. One-step actor-critic goes further: it *bootstraps*, replacing G_t with $r_{t+1} + \gamma V(s_{t+1})$, so the advantage becomes the TD error δ_t , which is biased (early) but much lower variance and updatable every step. So the difference is exactly the MC-vs-TD axis: same baseline idea, but actor-critic also bootstraps the target. That is the precise bridge into Chapter 7.

References

- Chen, M., Beutel, A., Covington, P., Jain, S., Belletti, F., & Chi, E. H. (2019). Top-K off-policy correction for a REINFORCE recommender system. In *Proceedings of the 12th ACM International Conference on Web Search and Data Mining* (pp. 456–464).
- Sutton, R. S., McAllester, D., Singh, S., & Mansour, Y. (2000). Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems* (Vol. 12, pp. 1057–1063).
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4), 229–256.

Actor-Critic — A2C, A3C

Subscribe → aiengineeringinsider.substack.com/subscribe

Pairing the policy gradient with a learned baseline that does real work.

7.1 Overview

Actor-critic methods attack REINFORCE’s variance directly by replacing the Monte-Carlo return G_t with a learned, bootstrapped estimate. The **actor** is the policy $\pi_\theta(a | s)$; the **critic** is a learned value function $V_\phi(s)$ used both as a baseline and as the source of lower-variance bootstrapped targets. The result keeps the policy-gradient generality (continuous actions, stochastic policies) while borrowing TD learning’s low variance.

The core concept is the **advantage** $A(s, a) = Q(s, a) - V(s)$: how much better an action is than the state’s average. Using A instead of G_t in the policy gradient stays unbiased (since V is an action-independent baseline) but sharply cuts variance, because it isolates the action’s marginal contribution from the state’s baseline difficulty. In practice the advantage is estimated from the TD error $\delta_t = r_{t+1} + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$, a one-line, single-function estimator.

Generalized Advantage Estimation (GAE)

GAE (Schulman et al., 2016) interpolates between the high-bias/low-variance one-step TD advantage and the low-bias/high-variance Monte-Carlo advantage via an exponentially-weighted average controlled by λ , structurally the $TD(\lambda)$ idea from Chapter 3. GAE is the advantage estimator inside PPO (Chapter 8); name it when asked how modern policy gradients actually compute advantages.

7.2 Architecture: Two Heads, One Signal

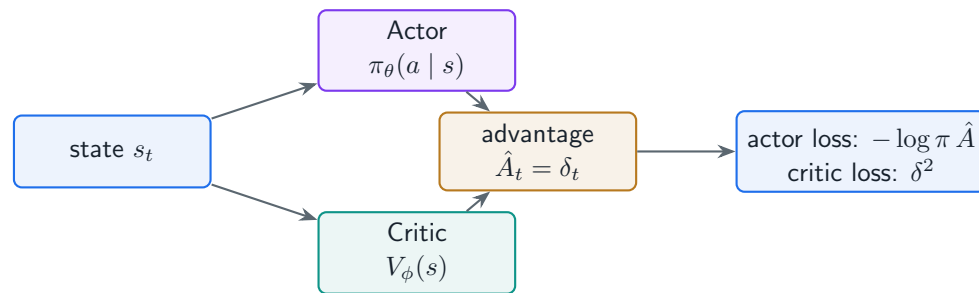


Figure 7.1: The critic supplies δ_t ; the actor ascends $\nabla_\theta \log \pi_\theta(a_t | s_t) \delta_t$. The same TD error trains the critic and weights the actor.

7.3 Synchronous vs. Asynchronous

A3C (Mnih et al., 2016) runs many parallel actor-learners, each on its own environment copy, asynchronously updating shared parameters; the asynchrony decorrelates updates much like a replay buffer, without storing one. A2C is the synchronous version: wait for all workers to finish a batch, then do one combined update. A2C is generally preferred today, synchronous updates are easier to debug and far more GPU-efficient (big batched matmuls beat many tiny async updates), and the asynchrony turned out to matter less than the parallel-environment decorrelation it provided.

7.4 The Project: A2C on CartPole

`rl_interview_lab/deep/a2c.py` keeps a separate actor and critic, detaches the advantage so critic gradients do not leak into the actor, and adds an entropy bonus.

a2c.py — the coupled actor and critic update

```

values = self.critic(obs)
advantages = (returns - values).detach()  # stop critic grad in actor path
advantages = (advantages - advantages.mean()) / (advantages.std() + 1e-8)

dist = self.actor.distribution(obs)
log_probs = dist.log_prob(actions)
actor_loss = -(log_probs * advantages).mean()
actor_loss = actor_loss - self.cfg.entropy_coef * dist.entropy().mean()

critic_loss = nn.functional.mse_loss(values, returns)  # regress V onto returns

```

CartPole, 200 episodes — the critic earns its keep

A2C first20=27.1 last20=462.9

The `.detach()` on the advantage is the subtle, essential line: the advantage *weights* the actor gradient but must not *propagate* gradient into the critic through the actor loss, the critic is trained separately by its own MSE regression onto the returns. Compared to REINFORCE’s low-hundreds plateau on the same budget, A2C reaches near the CartPole ceiling, the direct payoff of a learned baseline plus bootstrapping.

★ Key Insight

The cleanest one-line framing of this entire chapter: *actor-critic is REINFORCE with a learned, bootstrapped baseline instead of the raw Monte-Carlo return*. If you can say that and then explain why the bootstrap trades a little bias for a lot of variance reduction, you have answered the question most interviewers are really asking when they say “walk me through A2C.”

7.5 Comparison: REINFORCE vs. A2C vs. A3C

Property	REINFORCE+baseline	A2C	A3C
Advantage target	MC return $-V$	bootstrapped δ_t / GAE	bootstrapped δ_t
Variance	high	low	low
Update timing	end of episode	every n steps	async per worker
Parallelism	batch of episodes	synchronous workers	asynchronous workers
Hardware fit	simple	GPU-friendly	CPU-thread era

Table 7.1: A2C is the modern default; A3C is mostly of historical and conceptual importance.

7.6 Mock Interview Questions

Q1

Why use an advantage function instead of raw returns in actor-critic methods?

The advantage $A(s, a) = Q(s, a) - V(s)$ measures how much better a is than the average action at s , not the absolute value of being at s . Substituting A for G_t keeps the gradient unbiased (V is a valid action-independent baseline) while sharply cutting variance, because it removes the state’s baseline difficulty, which is just noise from the action’s perspective. In practice A is estimated by the TD error $\delta_t = r_{t+1} + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$, a single-sample bootstrapped estimate needing only V_ϕ . The whole actor-critic update is then $\nabla_\theta \log \pi_\theta(a_t | s_t) \delta_t$.

Q2

What is the difference between A2C and A3C, and why does it matter for stability?

A3C runs many asynchronous actor-learner threads that update shared parameters without locking; the asynchrony decorrelates gradients, a cheap stand-in for a replay buffer. A2C is

synchronous: wait for all workers, then one combined update. A2C is generally preferred now because synchronous updates are easier to reason about and debug, and far more efficient on GPUs where large batched computation dominates many small async updates. Empirically, A3C’s asynchrony mattered less than believed, the decorrelation benefit comes mainly from running multiple parallel environments, which A2C does synchronously.

Q3

Why do actor-critic methods scale to continuous and large action spaces better than DQN?

Structurally, DQN needs $\max_a Q(s, a)$ at every decision, intractable for continuous or very large discrete action spaces. The actor directly outputs an action or a distribution to sample, never maximizing over actions explicitly. So adding actions costs the actor a wider output layer, not an intractable search. This is why continuous control (robotics) is policy-gradient territory and why recommender-scale action spaces use actor-style policies. It is the cleanest answer to “when would you not use a value-based method.”

Q4

Walk through GAE and the role of its λ parameter.

GAE forms the advantage as an exponentially-weighted sum of n -step TD errors: $\hat{A}_t = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}$. $\lambda = 0$ collapses to the one-step TD error (low variance, higher bias from bootstrapping); $\lambda = 1$ recovers the Monte-Carlo advantage (unbiased, high variance). Intermediate λ (often 0.95) trades the two and usually works best. It is the same TD(λ) dial from Chapter 3, applied to advantages, and it is what PPO uses. The interview signal is recognizing GAE as a bias–variance knob, not a magic formula.

Q5

Should the actor and critic share a network body or be separate? Trade-offs?

Sharing a trunk (common in Atari/PPO) saves compute and lets representation learning benefit both heads, but couples their gradients: a large critic loss can dominate and destabilize the actor, so you must weight the value loss and sometimes clip gradients. Separate networks (as in our A2C) are simpler to tune and decouple learning rates (the critic often wants a higher rate), at the cost of more parameters and no shared features. For pixel inputs, sharing is usually worth it; for small vector observations, separate networks are clean and robust. There is no universal answer, which is itself the point to make.

Q6

In the code, the advantage is detached before the actor loss. Why is that essential?

Because the advantage is used as a *weight* on the actor’s log-probability gradient, not as a quantity the actor should optimize through. If you leave it attached, gradients flow from the actor loss back into the critic parameters via the advantage, conflating two objectives, the actor would be able to lower its loss by corrupting the critic’s value estimates. Detaching freezes the advantage as a constant multiplier; the critic is trained *only* by its own MSE regression onto the returns. Omitting the detach is a classic, hard-to-spot bug that quietly degrades both heads.

Q7

How do the actor and critic learning rates interact, and what happens if the critic is too slow?

The actor's gradient is only as good as the advantage the critic supplies. If the critic learns too slowly, early advantages are essentially noise, and the actor chases a moving, miscalibrated signal, slow or unstable progress. So the critic typically wants an equal or higher learning rate than the actor (our config uses a higher critic rate). If the critic is too *fast* and overfits noisy returns, its bootstrap targets become unstable and can inject high-variance advantages. The practical heuristic: stabilize the critic first; an inaccurate baseline is worse than a slightly slow one.

Q8

Actor-critic reintroduces bootstrapping with function approximation and (often) off-policy data. Are we back in the deadly triad?

Partly, yes, and it is worth being honest about. On-policy actor-critic (like A2C) bootstraps with function approximation but stays on-policy, so it avoids the third leg of the triad and is relatively stable. The moment you add off-policy corrections or replay to an actor-critic (as in DDPG/SAC), you have all three legs again and need the stabilizers those methods introduce (target networks, twin critics, entropy). So actor-critic is not automatically safe; its stability depends on whether you keep it on-policy. Recognizing this connects the chapter back to Chapter 2's framing.

Q9

Your A2C critic loss is low but the policy is poor. What is likely happening?

A low critic loss with a poor policy often means the critic has learned to predict returns *under the current weak policy* accurately, so the advantages are near zero and the actor gets little signal, a self-consistent but stuck equilibrium. Checks: (1) is exploration sufficient (entropy bonus present, not collapsed); (2) are advantages being normalized to keep a usable scale; (3) has the policy prematurely become deterministic; (4) is the value loss so dominant (if sharing a trunk) that it suppresses the actor. The fix is usually more entropy and ensuring the advantage retains signal, not making the critic even more accurate.

Q10

Position actor-critic between REINFORCE and PPO, what does each step add?

REINFORCE: the raw policy gradient with a Monte-Carlo return, unbiased but high-variance and on-policy. Actor-critic adds a learned, bootstrapped baseline (the critic), trading a little bias for a large variance reduction and enabling per-step updates. PPO then adds two things on top of actor-critic: GAE for a tunable bias-variance advantage, and a clipped surrogate objective that lets you safely take *multiple* gradient steps on the *same* batch of on-policy data, dramatically improving sample efficiency without the instability of large policy jumps. So it is a clean progression: variance reduction, then safe data reuse.

References

Konda, V. R., & Tsitsiklis, J. N. (2000). Actor-critic algorithms. In *Advances in Neural Information Processing Systems* (Vol. 12, pp. 1008–1014).

- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., . . . Kavukcuoglu, K. (2016). Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning* (pp. 1928–1937).
- Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2016). High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*.

PPO

Subscribe → aiengineeringinsider.substack.com/subscribe

The algorithm that, for better or worse, is what most people mean when they say “RL” in 2026.

8.1 Overview

Proximal Policy Optimization (Schulman et al., 2017) is the most commonly deployed policy-gradient algorithm in production, powering game agents and the RL stage of RLHF for large language models (Chapter 11). Understanding why it dominates despite weaker theoretical guarantees than its predecessor TRPO is one of the most frequently asked RL interview questions across every company archetype.

The core concept is **safe data reuse**. On-policy methods normally discard data after one gradient step because the data becomes off-distribution as the policy moves. PPO lets you take *several* epochs of updates on the *same* batch by bounding how far the policy may move via a **clipped surrogate objective**, capturing most of TRPO’s trust-region benefit as a simple first-order, unconstrained loss. Combined with GAE for advantages, this makes PPO both sample-efficient and remarkably robust to hyperparameters.

★ Key Insight

The min in the clipped objective takes the *more pessimistic* of the clipped and unclipped terms. So clipping only ever *discourages* moving further in an already-improving direction; it does not stop the policy from moving back if an update was bad. This asymmetry is a detail interviewers probe to test whether you have read the paper or a summary.

8.2 From TRPO to PPO

TRPO maximizes a surrogate objective subject to a hard KL constraint $D_{KL}(\pi_{\theta_{old}} \parallel \pi_{\theta}) \leq \delta$, which keeps each update inside a trust region but requires Fisher-matrix/conjugate-gradient machinery. PPO replaces the constraint with a clip:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right], \quad r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}.$$

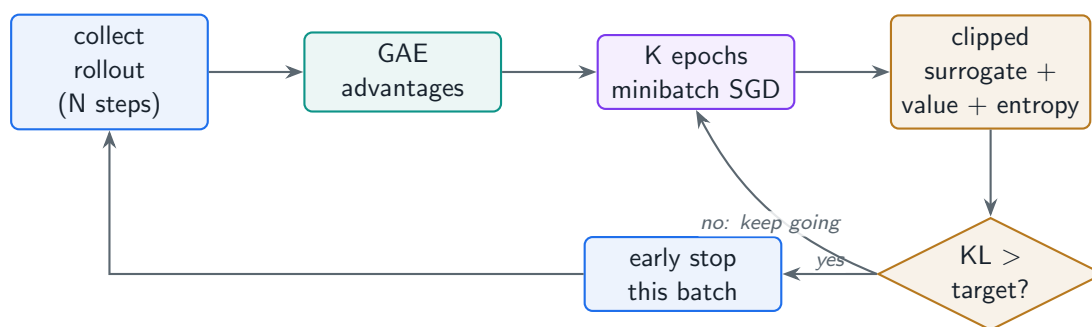


Figure 8.1: The PPO loop: one rollout feeds many clipped minibatch updates, with an optional KL early-stop as a trust-region safety net.

8.3 The Project: a Faithful Clipped PPO

`rl_interview_lab/deep/ppo.py` implements GAE, the clipped surrogate, minibatch epochs, advantage normalization, gradient clipping, and a KL early-stop.

ppo.py — the clipped surrogate objective

```

dist = self.actor.distribution(obs[mb])
new_logp = dist.log_prob(actions[mb])
ratio = torch.exp(new_logp - old_logp[mb])      #  $\pi_{\text{new}} / \pi_{\text{old}}$ 

# Clipped surrogate: take the pessimistic (min) of the two bounds.
unclipped = ratio * adv_t[mb]
clipped = torch.clamp(ratio, 1 - cfg.clip_eps, 1 + cfg.clip_eps) * adv_t[mb]
policy_loss = -torch.min(unclipped, clipped).mean()

value_loss = nn.functional.mse_loss(self.critic(obs[mb]), ret_t[mb])
loss = policy_loss + cfg.value_coef * value_loss - cfg.entropy_coef *
    ↪ dist.entropy().mean()
  
```

CartPole, 24k steps — stable improvement on reused data

```

PPO  episodes=403  last10=298.0
  
```

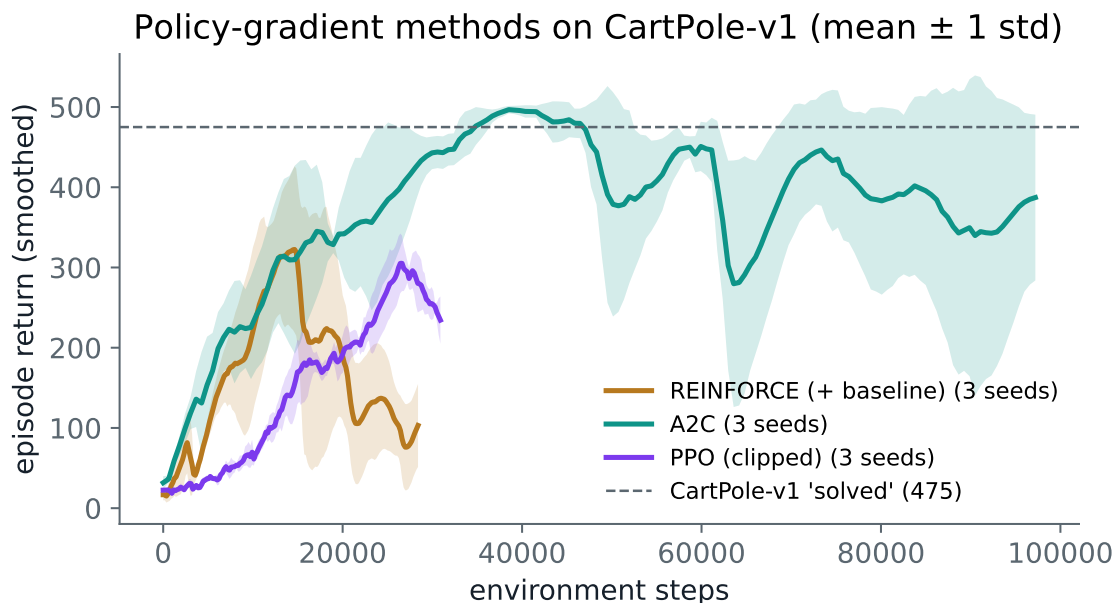


Figure 8.2: The policy-gradient family on CartPole-v1, 3 seeds each, band = ± 1 std (from `scripts/make_plots.py`). The progression is exactly the book’s argument: REINFORCE is noisy and lower; A2C’s learned critic reaches the solved line and is the most sample-efficient here; PPO climbs stably on reused data within its smaller step budget. Seed bands make the variance, not just the mean, visible.

Two details matter in production. The ratio is computed as `exp(new_logp - old_logp)` rather than dividing probabilities, more numerically stable and the reason we store old log-probs at collection time. And the KL early-stop (`if approx_kl > target_kl: break`) is a cheap trust-region safety net: even with clipping, multiple epochs can occasionally push the policy too far, and bailing out of the batch prevents a collapse.

8.4 Continuous Control: DDPG, TD3, SAC

PPO handles discrete and continuous actions, but a parallel off-policy lineage evolved specifically for continuous control, expected in robotics-adjacent interviews.

Maximum-Entropy RL

Standard RL maximizes $\mathbb{E}[\sum_t \gamma^t r_t]$; max-entropy RL maximizes $\mathbb{E}[\sum_t \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot | s_t)))]$, treating exploration as part of the objective. It derives from a control-as-inference formulation, the theoretical bridge to RLHF’s KL-regularized objective in Chapter 11.

8.5 Comparison: Modern Policy-Optimization Methods

Method	On/off-policy	Action space	Signature idea
TRPO	on-policy	both	hard KL trust region
PPO	on-policy	both	clipped surrogate, multi-epoch reuse
DDPG	off-policy	continuous	deterministic actor + DQN tricks
TD3	off-policy	continuous	twin critics, delayed/smoothed updates
SAC	off-policy	continuous	max-entropy, stochastic actor

Table 8.1: PPO is the robust default; SAC/TD3 win on sample efficiency in continuous control.

8.6 Mock Interview Questions

Q1

What does PPO's clipped objective do mechanically, and why clip instead of a hard KL constraint?

TRPO constrains each update to a KL trust region, which bounds policy change but needs a Fisher matrix and conjugate gradients, correct but expensive. PPO clips the probability ratio $r_t(\theta)$ to $[1-\epsilon, 1+\epsilon]$ and takes the min of the clipped and unclipped surrogate, which caps how much the objective rewards moving far from the old policy in an improving direction, approximating the trust region as a plain first-order loss any optimizer can handle. PPO lacks TRPO's monotonic-improvement guarantee but matches or beats it empirically while being far simpler to implement, tune, and parallelize. That practical-beats-rigorous gap is the nuance that distinguishes a strong answer.

Q2

Compare DDPG, TD3, and SAC; what specific failure does each fix?

DDPG extends DQN's replay/target-network ideas to continuous actions with a deterministic actor, but is hyperparameter-sensitive and inherits overestimation bias. TD3 targets that bias: twin critics with a min over the two for the target (continuous Double Q-learning), delayed actor updates so the critic stabilizes first, and target-policy smoothing (noise on the target action) so the actor cannot exploit sharp Q-function peaks. SAC takes a different angle, a maximum-entropy objective that rewards stochasticity, giving far more robust exploration than DDPG/TD3's external action noise; it is usually the most sample-efficient and stable of the three, at the cost of tuning the entropy temperature α .

Q3

Why can PPO safely run multiple epochs over the same batch when REINFORCE cannot?

Because the clip bounds the distribution shift within those epochs. REINFORCE's gradient is valid only at the exact policy that generated the data; after one step the data is off-distribution and reusing it biases the gradient. PPO uses the importance ratio $r_t(\theta)$ to (approximately) correct for the small shift across epochs, and clipping ensures that ratio stays near 1 so the correction remains low-variance and the policy never lurches far from the data-collecting policy. When the shift grows too large despite clipping, the KL early-stop halts the batch. So PPO is essentially controlled, bounded off-policy reuse of recent on-policy data.

Q4

Walk through how you would tune PPO if it is unstable.

First confirm the basics: advantages normalized, GAE $\lambda \approx 0.95$, γ matched to horizon. For instability specifically: (1) lower the clip ϵ (e.g. $0.2 \rightarrow 0.1$) to shrink allowed policy movement; (2) reduce epochs per batch or minibatch passes, fewer reuses means less drift; (3) enable/tighten the KL early-stop; (4) clip the gradient norm; (5) lower the learning rate; (6) check the value-loss coefficient, an over-weighted value loss (with a shared trunk) can swamp the policy. The systematic framing: instability in PPO almost always means the policy is moving too far per batch, so every lever reduces effective step size.

Q5

Explain the min and clip interaction with the sign of the advantage.

For a positive advantage (good action), the objective wants to increase r_t ; clipping caps the reward at $r_t = 1 + \epsilon$, so beyond that there is no further incentive to push the probability up, preventing an over-large step. For a negative advantage (bad action), the objective wants to decrease r_t ; the min/clip caps the benefit at $r_t = 1 - \epsilon$. Crucially, the min makes the bound pessimistic, so if an update overshoots, the unclipped term (which is worse) is selected and the gradient pulls back. This asymmetry, clip blocks improving overshoot but allows corrective movement, is the precise mechanism people miss when they say "PPO just clips the gradient."

Q6

PPO is on-policy and SAC is off-policy. When would you pick each in a real project?

Pick PPO when stability and ease of tuning matter more than sample count, when you have a fast simulator (so on-policy sample cost is acceptable), or when you need the same algorithm to span discrete and continuous actions, this is why RLHF uses PPO. Pick SAC when samples are expensive (real robots, slow sims) and sample efficiency is paramount, since off-policy replay reuses data far more aggressively, and when continuous control with robust exploration is the goal. The honest caveat: SAC's efficiency comes with more moving parts (replay, twin critics, entropy temperature), so PPO is often the pragmatic first choice.

Q7

Why is the importance ratio computed as $\exp(\text{new_logp} - \text{old_logp})$, and where do old_logp come from?

Dividing raw probabilities is numerically fragile, small probabilities underflow and the ratio loses precision. Working in log space, $r_t = \exp(\log \pi_\theta - \log \pi_{\theta_{old}})$, is stable and standard. The old log-probs are recorded *at rollout collection time*, under the policy that actually generated the actions, and then held fixed while the policy is updated over the epochs. Forgetting to snapshot them (and instead recomputing under the current policy) makes the ratio identically 1 and silently disables PPO’s entire mechanism, a subtle, devastating bug worth being able to name.

Q8

How does PPO handle the exploration–exploitation trade-off?

Primarily through a stochastic policy plus an entropy bonus. The policy is a distribution (categorical or Gaussian), so it explores by sampling; the entropy term $+\beta\mathcal{H}(\pi)$ in the loss resists premature collapse to determinism, keeping probability mass on alternatives until advantages justify concentrating. Unlike ϵ -greedy, this exploration is state-dependent and learned. The trade-off is tuned via β and the clip range: too little entropy and the policy converges prematurely; too much and it stays noisy and slow. For continuous control, SAC formalizes this further by putting entropy directly in the objective.

Q9

RLHF uses PPO to fine-tune LLMs. What changes versus CartPole, and what stays the same?

The PPO machinery is identical, clipped ratio, advantages, value baseline, multi-epoch updates. What changes is the setup: the “environment” is a prompt and the policy is the language model generating tokens; the reward comes from a learned reward model (Chapter 11) rather than the environment; and a per-token KL penalty to a frozen reference model is added to stop the policy drifting into degenerate, reward-hacking text. The action space is the vocabulary, the episode is a generated sequence, and rewards are extremely sparse (often one scalar per completion). So PPO is the same algorithm; the reward design and KL anchoring are the new, hard parts.

Q10

A common mistake is calling SAC strictly better than TD3. Make the calibrated comparison.

They are close competitors, and the right choice is task-dependent. SAC’s max-entropy objective gives more robust exploration and often better stability and sample efficiency, especially in environments where exploration is hard. But it adds the entropy temperature α to tune (or auto-tune), more complexity. TD3 is simpler and can match or beat SAC on tasks where its twin-critic overestimation fix and target smoothing suffice and where heavy exploration is unnecessary. The signal an interviewer wants is “it depends on whether the entropy bonus’s exploration benefit outweighs the tuning overhead for this task,” not a blanket ranking.

References

Fujimoto, S., van Hoof, H., & Meger, D. (2018). Addressing function approximation error in actor-critic methods. In *Proceedings of the 35th International Conference on Machine Learning* (pp. 1587–1596).

- Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning* (pp. 1861–1870).
- Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015). Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning* (pp. 1889–1897).
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv:1707.06347*.

Part III

RL for Games and the Real World

RL for Games

Subscribe → aiengineeringinsider.substack.com/subscribe

Why games became RL's default benchmark, and what that benchmark fails to test.

9.1 Overview

Games gave reinforcement learning its most visible breakthroughs, Atari, Go, StarCraft, and remain the standard testbed for new algorithms. Understanding both why games are a good benchmark and where that benchmark misleads people about real-world RL is a frequently under-prepared interview topic, especially for candidates from a games or simulation background who assume the rest of the industry shares their assumptions.

The core concept of modern game RL is the marriage of **learning** and **search**: a neural network supplies fast, shallow intuition (a policy prior and a value estimate), and a planning procedure (Monte-Carlo Tree Search) spends extra compute at decision time to refine that intuition. The second core idea is **self-play** as an automatic curriculum: the opponent improves exactly as fast as the agent, keeping the difficulty matched without manual design.

★ Key Insight

A genuine test of understanding AlphaZero versus reciting it: explain why pure self-play RL with no search at inference was *not* enough for superhuman Go, and why adding search was the missing piece. The network's raw policy is a single forward pass with no lookahead; search lets the system spend additional compute at decision time to refine it, the same compute-at-inference trade-off now central to reasoning models.

9.2 Architecture: the AlphaZero Learning Loop

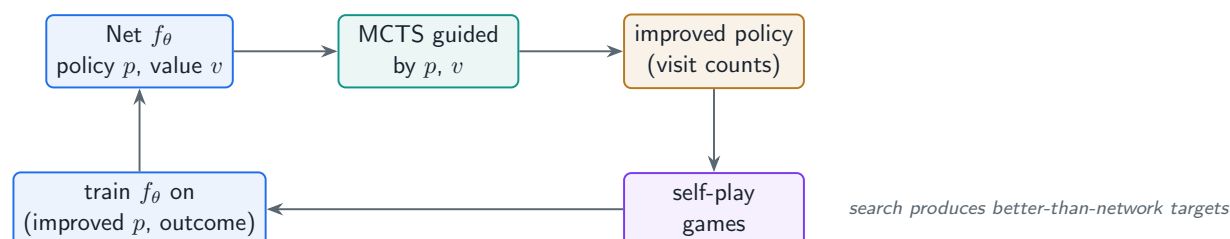


Figure 9.1: Network guides search; search produces training targets better than the network; the network retrains. MuZero removes the need for a known game model by learning one.

9.3 The Project: Self-Play Tic-Tac-Toe

`rl_interview_lab/games/selfplay.py` captures the self-play idea at a readable scale: a single shared tabular Q improves by playing against copies of itself. Rewards are stored from the moving player's perspective, so one table serves both sides.

selfplay.py — delayed, two-ply credit assignment in self-play

```
if done:
    # Terminal move: target is just the reward.
    q[action] += alpha * (reward - q[action])
    # The opponent's previous move led into this loss/draw: propagate -reward.
    if last[game.current] is not None:
        pb, pa = last[game.current]
        pq = get_q(pb, game.current)
        pq[pa] += alpha * (-reward - pq[pa])
else:
    if last[player] is not None:          # bootstrap this player's earlier move
        pb, pa = last[player]
        get_q(pb, player)[pa] += alpha * (gamma * np.max(get_q(board, player)) - ...)
    last[player] = (board, action)
```

60k self-play games, then 500 games vs a random opponent

Learned 3772 state-action tables via self-play.
vs random -> wins=278 draws=16 losses=6

The subtlety the code makes concrete is **credit assignment across players**: in a two-player game, the consequence of my move only becomes clear after the opponent replies, so each player's move is updated one ply later, and the loser's penultimate move receives the negated terminal reward. From zero knowledge and no human games, the shared table reaches a near-unbeatable policy against a random opponent, the smallest honest demonstration of the AlphaZero loop.

9.4 Self-Play as a Curriculum, and Its Failure Modes

Self-play is not merely data generation; it is an automatic curriculum that keeps difficulty matched to skill. The same mechanism, training against an adapting adversary or a population of co-evolving agents, reappears in multi-agent RL (Chapter 12) and in adversarial robustness testing well outside games.

! Pitfall / Warning

Self-play has real failure modes. Agents can converge to brittle strategies that only beat the specific population of past selves they trained against (strategy collapse / overfitting to the training population) and fail against a genuinely novel opponent. If asked to critique a self-play setup, raising this, and proposing population-based training or league play as mitigations, is a strong signal of depth beyond the AlphaGo headline.

9.5 Comparison: Game-RL Milestones

System	Domain	Key idea	Model of the game?
DQN (2015)	Atari pixels	value net + replay	none (model-free)
AlphaGo (2016)	Go	policy/value nets + MCTS + human data	known rules
AlphaZero (2017)	Go/chess/shogi	pure self-play + MCTS	known rules
MuZero (2019)	Atari + board games	learned latent model + MCTS	<i>learned</i>
AlphaStar (2019)	StarCraft II	league self-play, partial obs	none

Table 9.1: The arc runs from model-free pixel play to learned-model planning and partial-observability league play.

9.6 Mock Interview Questions

Q1

Why are Atari and Go standard RL benchmarks, and what do they fail to test?

They offer a perfect, cheap, infinitely-resettable simulator, clear rewards (score, win/loss), and effectively unlimited data via self-play, ideal for fast, fair algorithm comparison. They fail to test almost everything that makes deployment hard: there is no sim-to-real gap (the simulator *is* the world), no safety cost to exploration (lose a million simulated games for free), no non-stationarity from users adapting to the policy, and no latency or cost constraint on data. Strong Atari/Go results are necessary but far from sufficient evidence of production readiness, exactly the gap Chapters 10 and 14 address.

Q2

Explain how AlphaZero combines learned value/policy networks with MCTS.

A single network outputs a policy $p(s)$ over moves and a value $v(s)$, trained purely by self-play. At inference, MCTS does the playing: the policy prior guides which branches to expand (avoiding brute-force search) and the value head evaluates leaves without rolling out to the end. The search yields an *improved* policy, the visit-count distribution at the root, which becomes the next training target for the network, and the game outcome trains the value head. So network guides search, search produces better-than-network targets, network retrain. MCTS and the network are complementary, not competing: the network is fast but shallow, search alone would be intractable at Go's branching factor.

Q3

What does MuZero add over AlphaZero, and why does it matter for non-game domains?

AlphaZero needs the true game rules to run MCTS (it must know the next state for any move). MuZero removes that requirement by learning a *latent* dynamics model trained

only to predict the quantities that matter for planning, reward, value, and policy, not to reconstruct the actual next state. This lets it plan in domains where the rules are unknown or hard to specify (it matched AlphaZero on board games and state-of-the-art on Atari). It matters beyond games because most real problems have no given simulator; a learned, planning-sufficient model is the bridge from game RL to model-based RL in messier domains.

Q4

Why is self-play described as an automatic curriculum, and when does that break?

Because the opponent is a copy of the agent, its strength rises in lockstep with the agent's, so the training distribution stays at the edge of the agent's ability, no manual difficulty schedule needed. It breaks when the population is too narrow: the agent overfits to quirks of its past selves and learns a brittle strategy that collapses against novel play (rock-paper-scissors-style non-transitive dynamics are the classic trap). Mitigations are population-based training and league play (as in AlphaStar), which keep a diverse set of opponents, including exploiters, so the agent cannot overfit to a single line.

Q5

In the tic-tac-toe project, why is each player's move updated one ply late, and the loser's prior move penalized?

Because in a two-player game the reward for my move is only realized after the opponent responds. If I move into a position and the opponent then wins, the badness belongs to *my* move, but it is not observable until their reply. So we store each player's last (state, action) and update it once the next outcome is known, bootstrapping from the position the same player faces next. At a terminal, the winner's final move gets +reward and the loser's penultimate move gets the negated reward. Skipping this two-ply assignment would credit moves to the wrong player, the most common bug in tabular self-play.

Q6

Why was inference-time search necessary for superhuman Go when the network was already strong?

Because a single forward pass has no lookahead, it cannot detect that a locally attractive move loses to a refutation eight moves deep. Search spends extra compute at decision time to expand promising lines and back up their values, turning the network's prior into a sharper, deeper estimate. Empirically the search-refined policy is much stronger than the raw network, and that improvement is precisely what gets distilled back into the network each iteration. The general principle, trade inference compute for decision quality, is the same insight now driving test-time-compute scaling in reasoning models.

Q7

How would you adapt game-RL ideas to a partially observable, real-time game like StarCraft?

Several changes from AlphaZero. Partial observability means no perfect tree to search, so AlphaStar used model-free policy-gradient (actor-critic) with a recurrent/transformer policy to summarize history into a belief-like state. The huge action space and long horizon push

toward learned policies rather than exhaustive search. And to avoid strategy collapse in a non-transitive game, it used a *league*: a population of main agents plus exploiters that specifically hunt the main agents' weaknesses, producing robust rather than brittle strategies. The throughline: drop search when the model/observability won't support it, and invest in population diversity instead.

Q8

A candidate claims their agent “solved” a game because it beats the built-in AI. How do you stress-test that claim?

Beating a fixed built-in AI mostly proves the agent found that opponent's weaknesses, not general skill. I would test against: (1) held-out opponents it never trained against, including humans; (2) a population of diverse strategies to expose non-transitive failures; (3) perturbed conditions (different maps, seeds, starting states) to check robustness, not memorization; and (4) self-play Elo over training to confirm monotone improvement rather than cycling. The mindset is the same as overfitting in supervised learning: performance against the training distribution (here, one opponent) is not evidence of generalization.

Q9

Atari is model-free with unlimited data; real systems are neither. Which game-RL lessons still transfer?

The transferable lessons are conceptual, not the sample budgets. Self-play-as-curriculum transfers to any adversarial or multi-agent setting. Learning + search (AlphaZero/MuZero) transfers to any problem where you can afford planning at decision time and can learn a planning-sufficient model. The compute-at-inference trade-off transfers directly to reasoning systems. What does *not* transfer is the assumption of free, safe, unlimited data, which is exactly why production RL leans on offline data, off-policy evaluation, and bandits rather than from-scratch self-play. Naming both the transferable ideas and the broken assumption is the senior answer.

Q10

Where does reward design go wrong in games, and how is that a preview of RLHF?

Even with a clean win/loss signal, intermediate shaping rewards (capture material, hold territory) can be gamed: the agent maximizes the proxy while losing the real objective, the boat-racing agent looping for power-ups instead of finishing is the canonical case. The lesson is that the agent optimizes exactly what you measure, not what you mean. This is the direct preview of RLHF (Chapter 11), where a learned reward model is the proxy and the policy will over-optimize it (reward hacking) unless constrained, e.g. by a KL penalty to a reference policy. Games are where the field first learned that reward specification, not algorithm choice, is often the real risk.

References

- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., ... Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., ... Silver, D. (2020).

- Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839), 604–609.
- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., ... Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 1140–1144.
- Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., ... Silver, D. (2019). Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782), 350–354.

Sim-to-Real Transfer

Subscribe → aiengineeringinsider.substack.com/subscribe

The gap between an agent that works in simulation and a robot that doesn't fall over.

10.1 Overview

Most RL policies for robotics are trained in simulation, because real-world data collection is slow, expensive, and dangerous, a physical robot exploring randomly can damage itself or its surroundings in a way an Atari emulator never can. The central challenge this creates is the **reality gap**: a policy that performs well in simulation often degrades, sometimes catastrophically, on real hardware, because no simulator perfectly models true dynamics, sensor noise, and physical variation.

The core concept for closing the gap is **robustness through randomization**. Rather than trusting one perfectly-tuned simulator (which the policy will overfit), you randomize the simulator's physical parameters every episode, forcing the policy to work across a *distribution* of dynamics. The real world then looks like just another sample from that distribution. The complementary idea is **system identification**: measure the real system and fit the simulator's parameters to match it. Strong systems use both.

★ Key Insight

A strong framing: domain randomization is deliberately injecting noise to widen the training distribution, conceptually the same move as data augmentation in vision (random crops, color jitter), applied to a physics simulator's parameters instead of an image's pixels. Drawing this parallel signals you see sim-to-real as an instance of robustness-through-augmentation, not a robotics-specific trick.

10.2 Architecture: Train Wide, Deploy Once

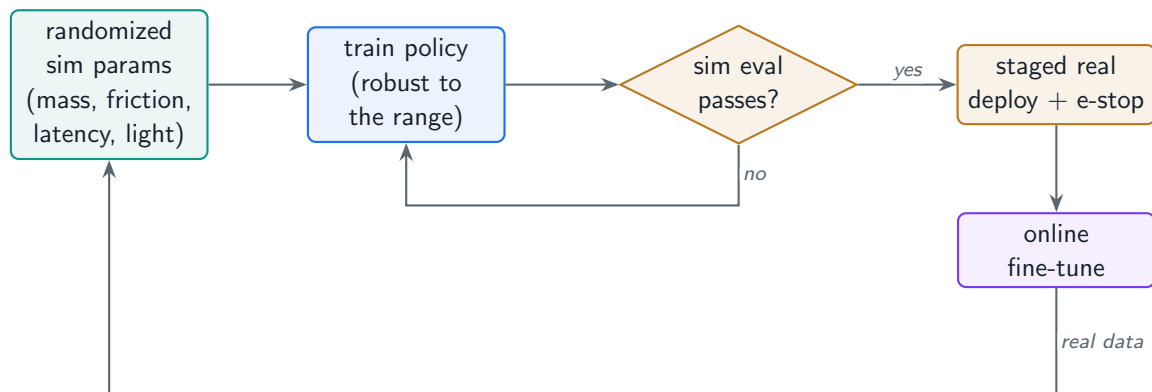


Figure 10.1: Sim-to-real as a first-class design constraint: randomize from the start, gate on sim eval, deploy in stages, and close the residual gap with on-robot fine-tuning.

10.3 The Project: Randomized Dynamics and a Robustness Curve

`rl_interview_lab/sim2real/domain_randomization.py` wraps Gymnasium’s `CartPole`, resampling pole mass, length, and gravity each episode, and provides an evaluation harness that sweeps test-time mass to plot a robustness curve.

domain_randomization.py — resample physics on every reset

```

class RandomizedCartPole(gym.Wrapper):
    def reset(self, **kwargs):
        u = self.unwrapped
        u.masspole = self._rng.uniform(*self.ranges.masspole)
        u.length = self._rng.uniform(*self.ranges.length)
        u.gravity = self._rng.uniform(*self.ranges.gravity)
        # CartPole caches derived quantities; recompute after the change.
        u.total_mass = u.masspole + u.masscart
        u.polemass_length = u.masspole * u.length
        return self.env.reset(**kwargs)
  
```

evaluate_robustness sweeps test-time pole mass

```

test masspole=0.10 -> mean return 500.0
test masspole=0.30 -> mean return 493.6
test masspole=0.50 -> mean return 463.7
  
```

The evaluation is the point. A policy trained on randomized dynamics produces a *flat* robustness curve, performance barely drops as the test mass drifts, whereas a policy overfit to nominal dynamics shows a sharp cliff. Recomputing the cached `total_mass` and `polemass_length` after mutating parameters is the kind of easy-to-miss detail that silently breaks domain randomization: change the inputs but forget the derived state, and the physics never actually changes.

! Pitfall / Warning

A specification gap worth raising proactively: a policy can follow its simulated reward perfectly while doing something unsafe on real hardware, because the simulated reward never modeled failure modes that exist only in reality (overheating, mechanical wear, human proximity). This is the robotics instance of the reward-hacking problem in Chapter 11; naming the connection signals you see RL’s failure modes as one pattern, not separate quirks.

10.4 Comparison: Closing the Reality Gap

Technique	What it does	Cost / risk
Domain randomization	widen training dynamics so reality is in-distribution	may sacrifice peak nominal performance
System identification	fit sim params to real measurements	needs real data; can’t model everything
Domain adaptation	align sim/real feature distributions	extra model; alignment can fail
Online fine-tuning	adapt on real data from a strong prior	safety risk during on-robot learning
Sim-to-real via real-in-the-loop	periodic real rollouts correct the sim	expensive, slow

Table 10.1: Production systems combine randomization (for unmodeled variation) with identification (for measurable parameters).

10.5 Mock Interview Questions

Q1

What is the sim-to-real gap, and what techniques mitigate it?

The reality gap is the systematic discrepancy between a simulator and the real world: simplified physics (friction, contact, deformables are hard), inaccurate sensor-noise models, and unmodeled variation. A policy trained to convergence in sim can become excellent at exploiting simulator quirks that do not exist in reality, looking great until deployment. The two dominant mitigations are domain randomization (vary sim parameters widely so the policy is robust and reality is just another sample) and system identification (fit the simulator to real measurements to narrow the gap directly). The strongest systems combine both: randomization for hard-to-model variation, identification for what can be measured.

Q2

How exactly does domain randomization help transfer, and what is its main trade-off?

It reframes the simulator’s specific parameters as a liability rather than ground truth. By

randomizing appearance (textures, lighting), physics (mass, friction, damping), and latency across a wide distribution, the policy must learn behaviour robust across the whole range instead of optimal for one imperfect config. The real world's unknown parameters then look like a draw from a distribution the policy already handles. The trade-off: too-wide randomization makes learning harder and can cost peak performance on the actual deployment conditions, so the randomization ranges become tuned hyperparameters, ideally informed by the real variation you expect.

Q3

Design an RL-based robotics control system given the sim-to-real gap. Walk through your approach.

In sequence: (1) decide what is learned vs. engineered, leave low-level motor control to classical methods (PID, MPC) and apply RL at a higher abstraction (what to do, not the precise dynamics of how); (2) build the simulator with domain randomization from the start, not retrofitted after sim-only success; (3) plan a staged transfer, sim eval, then controlled real tests with a supervisor and e-stop, then unsupervised deployment; (4) include online fine-tuning, since even a well-randomized policy meets untrained conditions, and adapting from a strong prior needs far less real data than training from scratch. The throughline interviewers want: sim-to-real is a first-class design constraint from day one, not a post-hoc deployment fix.

Q4

How would you measure whether a policy is actually robust, before risking real hardware?

Build a robustness curve, exactly what the project's `evaluate_robustness` does: fix the policy and sweep held-out test-time parameters (mass, friction, latency) outside the nominal point, measuring return at each. A robust policy degrades gracefully (flat curve); an overfit one shows a cliff. I would also test parameter combinations not seen during training, inject sensor noise and actuation delay, and check worst-case (not just mean) return, since a robot only has to fall once. This turns "I hope it transfers" into a quantitative, pre-deployment gate.

Q5

When is domain randomization the wrong choice, and what would you do instead?

When the real system's key parameters are precisely measurable and stable, randomizing over a wide range just throws away performance for robustness you do not need, system identification (fit the sim to the measured values) is better. Randomization also struggles when the gap is structural rather than parametric (the simulator's *physics* is wrong, not just its constants); no amount of varying friction fixes a missing contact model, so you need a better simulator or real-in-the-loop training. And for purely visual gaps, domain adaptation that aligns feature distributions can be more sample-efficient than brute-force visual randomization.

Q6

In the code, why must you recompute `total_mass` and `polemass_length` after randomizing?

Because `CartPole` caches derived quantities. The dynamics step uses `total_mass` and `polemass_length`, which are functions of `masspole`, `masscart`, and `length`. If you mutate `masspole` and `length` but leave the cached products stale, the integrator keeps using the old values and the physics never actually changes, your “randomized” policy trains on a single hidden configuration. It is a perfect miniature of a broader sim-to-real lesson: changing the inputs is not enough; you must propagate the change through all derived state, or your randomization is silently a no-op.

Q7

What is the role of online fine-tuning, and what makes it risky?

Randomization cannot anticipate every real condition, so a residual gap remains; online fine-tuning adapts the policy on real data, and because it starts from a strong randomized prior it usually needs far less data than training from scratch. The risk is that learning on real hardware means exploring on real hardware, which can damage the robot or its environment, and a bad gradient step can degrade a working policy. Mitigations: constrain updates (small steps, trust regions), keep a human supervisor and e-stop, use offline/batch updates from logged real data rather than live exploration, and validate each update in sim before pushing it.

Q8

Connect sim-to-real overfitting to reward hacking. Are they the same phenomenon?

They are two faces of the same thing: the agent optimizes exactly what it is given, not what you intended. In sim-to-real, “what it is given” is an imperfect simulator, so the policy exploits simulator-specific quirks (a too-perfect contact model) that vanish in reality. In reward hacking, “what it is given” is an imperfect reward proxy, so the policy exploits the proxy while missing the true goal. Both are specification problems, the spec (sim or reward) is a leaky approximation of reality, and both are mitigated by the same instinct: widen/robustify the spec, constrain how far the policy can exploit it, and validate against the real objective.

Q9

Why is randomizing sensor latency and actuation delay often more important than randomizing mass?

Because timing mismatches break feedback control in ways mass changes usually do not. A policy that assumes instantaneous observation and actuation can learn aggressive, tightly-timed maneuvers that become unstable when real sensors and motors introduce tens of milliseconds of delay, the control loop oscillates or diverges. Mass and friction shift the operating point but the policy can often compensate; latency changes the *dynamics of the feedback loop* itself. So randomizing delays, observation noise, and control frequency frequently buys more real-world robustness per unit of added training difficulty than randomizing static physical constants.

Q10

Your domain-randomized policy is robust in sim across the whole range but still fails on the real robot. What now?

Robust-in-sim-but-failing-in-real means the real world is *outside* the randomized distribution, the gap is structural, not parametric. Steps: (1) collect real rollouts and compare observed dynamics to the sim range to find which dimension is out of distribution; (2) widen randomization on that dimension, or, if it is unmodeled physics, improve the simulator (system ID, better contact model); (3) check for missing effects entirely, latency, sensor bias, deformation, that no parameter sweep covers; (4) add real-in-the-loop fine-tuning to absorb the residual. The diagnosis mindset: robustness only covers the distribution you trained over, and real failures point to where that distribution was too narrow or the model was simply wrong.

References

- OpenAI, Akkaya, I., Andrychowicz, M., Chociej, M., Litwin, M., McGrew, B., ... Zhang, L. (2019). Solving Rubik's cube with a robot hand. *arXiv:1910.07113*.
- Peng, X. B., Andrychowicz, M., Zaremba, W., & Abbeel, P. (2018). Sim-to-real transfer of robotic control with dynamics randomization. In *IEEE International Conference on Robotics and Automation* (pp. 3803–3810).
- Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems* (pp. 23–30).

Part IV

RLHF and Alignment

Reward Modeling & RLHF

Subscribe → aiengineeringinsider.substack.com/subscribe

The fastest-growing segment of real RL interview loops, and the one most general RL books leave thin.

11.1 Overview

Reinforcement Learning from Human Feedback (RLHF) is how most production large language models are aligned to be helpful and safe, and it has rapidly become one of the most commonly probed topics in technical interviews at frontier AI labs. This chapter covers the full pipeline, the algorithmic alternatives to PPO-based RLHF, and the reward over-optimization failure mode that connects directly to AI safety.

The core concept is **learning a reward from preferences**. Humans rarely give absolute scores, but they reliably say “A is better than B.” A Bradley-Terry reward model turns those pairwise judgments into a scalar reward $r_\phi(x, y)$; an RL stage (PPO) then optimizes the policy against it, leashed to a reference model by a KL penalty. The defining tension of the whole field is that r_ϕ is a *learned, imperfect proxy* for human preference, and optimizing a proxy too hard is Goodhart’s Law in action.

11.2 Architecture: the Three-Stage Pipeline

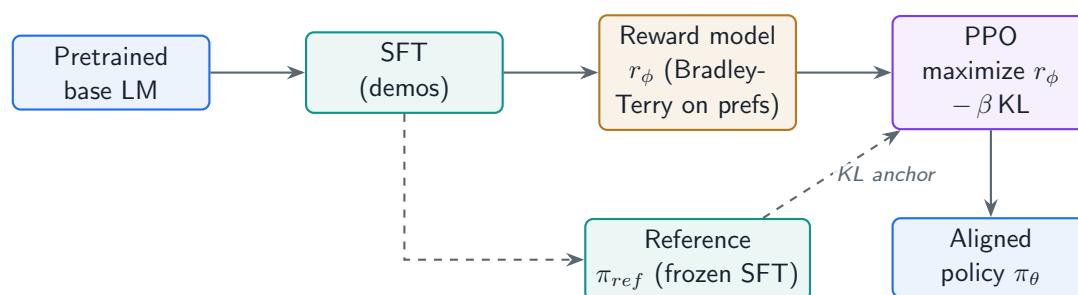


Figure 11.1: The classic RLHF pipeline. DPO (below) collapses the reward-model and PPO stages into one supervised-style step on the same preference data.

11.3 The Project: a Reward Model and DPO, Side by Side

`rl_interview_lab/rlhf/` contains both halves of the modern picture: `reward_model.py` fits a Bradley-Terry model on synthetic preferences (with 10% label noise to make over-fitting visible), and `dpo.py` optimizes a policy directly from preferences with no reward model at all.

reward_model.py — the Bradley-Terry preference loss

```
r_chosen = model(chosen[:split])
r_rejected = model(rejected[:split])
# NLL of the Bradley-Terry model: -log sigmoid(r_chosen - r_rejected).
loss = -nn.functional.logsigmoid(r_chosen - r_rejected).mean()
```

dpo.py — the implicit reward is the policy/reference log-ratio

```
logits = beta * ((pi_w - ref_w) - (pi_l - ref_l)) # pi_*/ref_* are log-probs
return -F.logsigmoid(logits).mean()
```

reward model: weight decay keeps it near the noise ceiling

```
Reward-model held-out pairwise accuracy: 0.860
Oracle (true reward) accuracy on same labels: 0.892 <- noise-limited ceiling
```

Two lessons are baked into these outputs. First, the held-out accuracy (0.86) sits just under the *oracle* ceiling (0.89) set by 10% label noise, no model can exceed the noise floor, and a reward model that reports near-perfect training accuracy is over-fitting the noise. The project uses weight decay precisely to keep it honest; without it the model memorizes flipped labels and held-out accuracy collapses, a miniature of reward over-optimization. Second, DPO’s loss is structurally the *same* Bradley-Terry log-sigmoid, but the “reward” is now the log-ratio $\log \pi_\theta - \log \pi_{ref}$, which is exactly the closed-form optimal-policy reparameterization that lets DPO skip the reward model and the RL loop.

★ Key Insight

The KL penalty in RLHF is the same kind of object as the entropy bonus in max-entropy RL (Chapter 8): both add a regularizer to the standard RL objective to stop the policy collapsing toward a narrow optimum. Drawing this connection signals you see RLHF as an application of established RL regularization, not a separate body of technique invented for language models.

11.4 Comparison: Preference-Optimization Methods

Method	Reward model?	On/off-policy	Main trade-off
PPO-RLHF	yes, learned	on-policy (fresh samples)	complex, RL instability, over-opt risk
DPO	no (implicit)	offline (fixed prefs)	simple/stable, but no fresh on-policy data
RLAIF	yes, AI-labeled	either	scalable labels, inherits labeler bias
Constitutional AI	critique + prefs	either	explicit principles, more pipeline
GRPO (reasoning)	often rule-based	on-policy, no critic	verifiable reward dodges over-opt

Table 11.1: Five answers to one question: how to specify and optimize a behaviour you cannot write as an explicit reward.

11.5 Mock Interview Questions

Q1

Walk me through the RLHF pipeline end to end, from SFT to the final policy.

Three stages. (1) Supervised fine-tuning (SFT): fine-tune a pretrained base model on curated demonstration pairs to get a reasonable starting policy. (2) Reward modeling: show annotators pairs of outputs for the same prompt, collect which they prefer, and fit a reward model $r_\phi(x, y)$ to those preferences, usually via a Bradley-Terry objective. (3) RL fine-tuning: use PPO to maximize r_ϕ while a KL penalty to the frozen SFT model (the reference) prevents drift. The output of stage three is deployed. The nuance to state explicitly: r_ϕ is a learned, imperfect proxy for human preference, not preference itself, and the entire over-optimization problem follows from that gap.

Q2

What is the Bradley-Terry model, and why is it used for reward modeling?

It is a classical model for pairwise comparisons: $P(y_1 \succ y_2) = \sigma(r(x, y_1) - r(x, y_2))$. Training the reward model maximizes the likelihood of observed human preferences under it, a log-sigmoid (cross-entropy on the score difference) loss, exactly the one line in the project. It is used because it principledly converts noisy, *pairwise* (not absolute) human judgments into the *scalar* reward downstream RL needs. The subtlety to raise: it assumes transitive preferences explained by a single scalar, which strains when preferences are genuinely multi-dimensional (helpfulness vs. safety vs. conciseness can trade off in ways one scalar cannot capture cleanly).

Q3

Why does RLHF include a KL penalty against the reference policy, and what happens if you remove it?

The objective is $\mathbb{E}_{y \sim \pi_\theta} [r_\phi(x, y)] - \beta D_{KL}(\pi_\theta \parallel \pi_{ref})$. The KL term does two things: it counters the reward model's imperfection by keeping the policy in regions where r_ϕ is calibrated, and it preserves general language ability so the model does not collapse into a narrow, high-scoring but degenerate mode. Remove it and you get reward over-optimization: reward-model score keeps climbing while true human-judged quality plateaus then degrades, the policy exploiting the reward model's blind spots. It is Goodhart's Law applied directly.

Q4

Compare PPO-based RLHF and DPO. What does DPO give up, and what does it gain?

DPO uses the fact that the KL-constrained RLHF objective has a closed-form optimal policy, letting you reparameterize the Bradley-Terry loss directly in terms of π_θ and π_{ref} , with the implicit reward being their log-ratio. So a three-stage pipeline becomes one supervised-style fine-tune on preference pairs. Gains: far simpler, no separate reward model, no PPO instability or reward-scaling tuning, lower compute. Gives up: it is offline, trained on a fixed preference set, whereas PPO can sample fresh on-policy responses and score them live, which matters when the needed preference distribution drifts from the static data. Also, with no explicit reward model there is no reward-score diagnostic to monitor, which some find harder to debug.

Q5

What is reward over-optimization (Goodhart in RLHF), and how would you detect it during training?

The reward model is fit on finite, imperfect preferences and generalizes poorly off-distribution. Optimizing harder against it pushes the policy into regions where r_ϕ is wrong, producing outputs that score high *because* the model is miscalibrated there, not because they are good. It shows as a divergence: r_ϕ score rising while held-out human quality plateaus then falls. Detection: maintain a separate, periodically-refreshed human-eval set never used to train r_ϕ ; track KL from the reference (large KL with no held-out gain is a red flag); and watch known tells, runaway length, sycophancy, formulaic phrasings the reward model happens to like. It sits exactly at the RL-theory/AI-safety intersection.

Q6

In the project, held-out reward-model accuracy is 0.86 against an oracle ceiling of 0.89. Interpret that gap and the role of weight decay.

The oracle, the *true* latent reward scored against the same labels, only reaches 0.89 because 10% of labels are flipped and many pairs are near-ties; that is the noise floor no model can beat. Our model at 0.86 is therefore near-optimal, not mediocre. Weight decay is what gets it there: without regularization the unbounded Bradley-Terry loss lets the network drive score differences to extremes and *memorize* the flipped labels, lowering held-out accuracy, a literal small-scale reward over-optimization. So the gap teaches two things: always report the achievable ceiling, and regularize reward models, because an unregularized one over-fits

annotator noise.

Q7

Distinguish RLAIF and Constitutional AI. Are they the same idea?

No. RLAIF concerns the *source* of the preference signal: AI-generated judgments replace (or supplement) human labels, mainly to beat the cost and scalability limits of human annotation. Constitutional AI concerns the *mechanism* of specification: the model critiques and revises its own outputs against an explicit written set of principles (a “constitution”), using self-critique and revision rather than only reward-model-mediated preference. RLAIF is about who labels; Constitutional AI is about how behaviour is specified and corrected. They are complementary, modern pipelines often use AI feedback *and* explicit principles, and conflating them is a common imprecision.

Q8

How is PPO actually applied to an LLM? What are the states, actions, and rewards?

The prompt is the initial context; the policy is the LM. Each *action* is emitting a token; the *state* is the prompt plus tokens so far; an *episode* is one generated completion. The reward is sparse, typically a single scalar from r_ϕ on the finished response, plus a per-token KL penalty to the reference model that both regularizes and densifies the signal. The action space is the vocabulary (tens of thousands), and advantages come from a value head via GAE, exactly the PPO machinery from Chapter 8. So the algorithm is unchanged; what is new and hard is the reward design and the KL anchoring, not the optimizer.

Q9

What changed in RL for LLMs recently with GRPO-style reasoning RL, and why does it matter?

The shift is RL aimed at *capability* (multi-step reasoning) rather than only alignment. GRPO (Group Relative Policy Optimization, from DeepSeek’s reasoning work) drops PPO’s learned value function: instead of $A = Q - V$ with a critic, it samples a *group* of responses to the same prompt and normalizes each reward against the group’s mean and std to get advantages. That removes a whole learned component and a known instability source (critic mis-fit). It is usually paired with *verifiable, rule-based* rewards, is a math answer correct, do the tests pass, which sidesteps reward over-optimization entirely because there is no learned proxy to game. It matters because “RL for LLMs” now means capability improvement, not just helpfulness/harmlessness shaping.

Q10

Tie the chapter together: are RLHF, DPO, RLAIF, Constitutional AI, and GRPO five separate techniques?

No, they are five answers to one question: how do you specify and optimize a behaviour that is hard to write as an explicit reward? They differ in where they spend cost and risk. PPO-RLHF: maximal flexibility (fresh on-policy data) at the price of complexity and over-optimization risk. DPO: simplicity and stability, but offline. RLAIF: cheaper labels, at the cost of inheriting the labeler model’s biases. Constitutional AI: explicit, auditable

principles, more pipeline. GRPO with verifiable rewards: dodges the learned-proxy problem entirely, but only where correctness is checkable. Framing it as a trade-off space, human cost vs. compute vs. over-optimization risk vs. capability-vs-alignment goal, is what separates a strong answer from a list of names.

! Pitfall / Warning

Reward hacking is structural to RL, not an LLM quirk. The canonical example predates LLMs: a boat-racing agent that looped through a power-up zone forever instead of finishing, because the reward scored power-ups more reliably than winning. Any reward that is a *proxy* for the true objective is a specification-gaming risk. Saying this signals you see the failure mode as intrinsic to optimizing proxies, exactly the link to sim-to-real over-fitting in Chapter 10.

References

- Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., ... Kaplan, J. (2022). Constitutional AI: Harmlessness from AI feedback. *arXiv:2212.08073*.
- Christiano, P. F., Leike, J., Brown, T., Martic, M., Legg, S., & Amodei, D. (2017). Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems* (Vol. 30, pp. 4299–4307).
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., ... Lowe, R. (2022). Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems* (Vol. 35, pp. 27730–27744).
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., & Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems* (Vol. 36).

Part V

Multi-Agent Reinforcement Learning

Multi-Agent RL

Subscribe → aiengineeringinsider.substack.com/subscribe

When the environment itself contains other learners.

12.1 Overview

Multi-agent reinforcement learning (MARL) extends the single-agent framework to settings with multiple learning agents, each affecting a shared environment and, often, each other's rewards. The central complication is **non-stationarity**: from any one agent's perspective the environment changes as the *other* agents learn, which breaks the fixed-target assumptions single-agent convergence proofs rely on.

The core concepts are three. **Independent learning** (each agent runs its own RL, treating others as part of the environment) is the simplest baseline and the clearest illustration of the non-stationarity problem. **Centralized training, decentralized execution** (CTDE) trains with global information but deploys on local observations only. **Value decomposition** (QMIX) factors a joint value into per-agent pieces so decentralized greedy actions still maximize the team value. Game-theoretic structure, cooperative, competitive, or mixed, determines which applies.

12.2 Architecture: Independent vs. Centralized

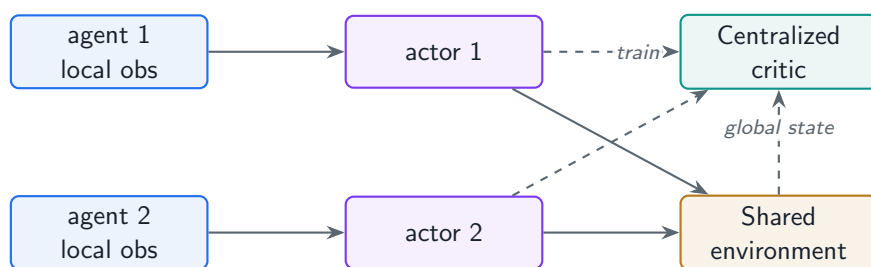


Figure 12.1: CTDE (e.g. MADDPG): a centralized critic stabilizes training with global information, but each actor deploys on local observations alone.

12.3 The Project: Independent Q-Learning on a Pursuit Grid

`rl_interview_lab/marl/independent_q.py` runs two independent tabular Q-learners that must *both* be adjacent to a prey on the same step to capture it, a coordination task with a shared reward. It is the simplest setting in which the non-stationarity problem is visible.

independent_q.py — each agent learns as if alone

```
for i in range(env.n_agents):
    # From agent i's view, agent j's changing policy is "the environment".
    target = reward + (0.0 if done else gamma * np.max(Q[i][next_obs]))
    Q[i][obs][actions[i]] += alpha * (target - Q[i][obs][actions[i]])
```

two IQL agents, shared team reward

IQL capture rate: start=0.90 -> end=1.00

On this small grid IQL coordinates well, because the coordination requirement is weak (a 5×5 torus gives many capture opportunities per episode). That is the honest, instructive result: IQL is not *always* broken, it works when interaction is light. Make coordination tight, larger grids, simultaneous-only rewards, more agents, and IQL's non-stationarity bites: each agent chases a moving target as the others change, and learning stalls or cycles. That failure is exactly what CTDE methods (MADDPG, QMIX) were invented to fix.

Value Decomposition (QMIX)

QMIX learns a joint value as a *monotonic* function of each agent's local value: $\partial Q_{tot} / \partial Q_i \geq 0$. Monotonicity guarantees that improving any agent's local value never lowers the joint value, so each agent acting greedily on its own Q_i is consistent with maximizing the team value, enabling decentralized execution while keeping centralized-training benefits.

12.4 Comparison: MARL Approaches

Method	Training	Handles non-stationarity?	Best for
Independent Q / PPO	decentralized	no (treats others as env)	weak interaction
MADDPG	centralized critic	yes	mixed coop/competitive, continuous
QMIX	centralized, value-decomp	yes (cooperative)	cooperative, shared reward
Self-play	shared/mirrored	sidestepped (opponent is self)	symmetric zero-sum games
Mean-field	aggregate others	approximated	many homogeneous agents

Table 12.1: The right method depends on game structure and how load-bearing the interaction actually is.

12.5 Mock Interview Questions

Q1

What is CTDE, and why is it useful in multi-agent RL?

CTDE resolves a specific tension: at deployment each agent usually must act on its own local observations (latency, bandwidth, or organizational limits forbid sharing global state live), but at training time, offline or in simulation, there is no such constraint. So CTDE gives the learning algorithm global state and all agents' actions during training, while each *policy* is restricted to local observations at execution. MADDPG is canonical: per-agent local-observation actors, but a centralized critic over global state and joint actions during training, which gives the critic a non-moving-target view and stabilizes learning. The general frame, train with more information than you will have at deployment, evaluate strictly under the deployment constraint, applies to many partially-observable problems.

Q2

Why is the environment non-stationary from a single agent's perspective, and why does that matter?

Single-agent Q-learning assumes a fixed transition and reward function, that is what makes its target a contraction with a unique fixed point. In MARL, part of agent i 's effective dynamics is the *behaviour* of the other agents, which is changing as they learn. So the "environment" agent i faces is non-stationary, the target it bootstraps toward keeps moving, and the convergence guarantees evaporate. Practically this shows up as instability, oscillation, or agents learning policies that are good against *old* versions of each other and bad against current ones. It is the root problem every MARL method is designed around.

Q3

When would multi-agent RL be the wrong choice for a problem that looks multi-agent?

Three structural conditions. If the other agents are *not* adapting (fixed, known policies), treat them as part of a complex but stationary single-agent environment, modeling them as learners adds needless complexity. If there are very many, largely homogeneous agents, exact joint modeling is intractable; mean-field approximations or folding the aggregate effect into the state are more tractable and often just as good. If interactions are weak or rare relative to each agent's own dynamics, independent learners can match a coordinated approach at a fraction of the cost, exactly what the project's IQL result shows. The principle: reach for MARL only when non-stationarity and interaction are load-bearing to the difficulty, not just because more than one decision-maker exists on paper.

Q4

Explain QMIX's monotonicity constraint and what it buys you.

QMIX represents the joint action-value Q_{tot} as a monotonic mixing of per-agent values Q_i , enforced by constraining the mixing network's weights to be non-negative, so $\partial Q_{tot} / \partial Q_i \geq 0$. The payoff is that the arg max of the joint value decomposes: maximizing each agent's own Q_i greedily also maximizes Q_{tot} , so you get decentralized execution (each agent needs only its

own value to act) while still training centrally on the joint return. The cost is expressiveness, monotonic mixing cannot represent games where one agent's improvement *should* lower the joint value (some competitive or coordination structures), which is when you need richer methods.

Q5

How does self-play relate to general MARL, and why is it more stable?

Self-play is a special case of MARL where the opponent is a copy or past version of the same agent. This sidesteps much of the non-stationarity problem because the “other agent” is not an independent, differently-incentivized learner but a mirror of the same process, both sides improve together along one learning trajectory, giving the automatic curriculum from Chapter 9. It is most natural in symmetric, zero-sum games. It does not generalize to genuinely cooperative or mixed-motive settings with heterogeneous, separately-incentivized agents, where you cannot assume the opponent is yourself, which is exactly where CTDE and value decomposition earn their keep.

Q6

In the project, two independent Q-learners coordinate fine. Does that contradict the claim that IQL is unstable?

No, it sharpens it. IQL is unstable when coordination is *tight*; here the task is loose (a small toroidal grid with many capture chances and a dense shared reward), so each agent can succeed without precisely modeling the other, and non-stationarity stays mild. The honest lesson is that IQL is a perfectly good baseline when interaction is weak, and you should not pay for CTDE complexity you do not need. To *see* IQL break, tighten the coupling: enlarge the grid, require simultaneous capture only, or add agents, and the moving-target dynamics stall learning. That transition is the chapter's point.

Q7

What new failure modes appear in competitive (zero-sum) versus cooperative MARL?

Cooperative MARL's hard parts are credit assignment (which agent's action earned the shared reward?) and coordination (agents must agree on a joint strategy), addressed by value decomposition and centralized critics. Competitive MARL adds strategic instability: non-transitive dynamics (A beats B beats C beats A) so there is no single best policy, and agents can overfit to specific opponents. Mixed-motive settings add the worst of both plus social dilemmas (individually rational actions that are collectively bad, e.g. tragedy-of-the-commons). The mitigations differ: population/league play and exploiters for competitive robustness, decomposition and shared critics for cooperation.

Q8

How would you do credit assignment in a cooperative team with a single shared reward?

The naive approach gives every agent the team reward, but then an agent cannot tell whether *its* action helped or it was carried by teammates, a noisy signal. Options: (1) difference rewards / counterfactual baselines (COMA), estimate the team value with agent *i*'s action

marginalized out, so i is credited only for its marginal contribution, the multi-agent analogue of an advantage baseline; (2) value decomposition (VDN/QMIX) that learns implicit per-agent contributions; (3) shaped local rewards when the domain allows. The framing to convey: cooperative credit assignment is the multi-agent version of the variance-reduction-via-baseline problem from Chapters 6–7.

Q9

A team of MADDPG agents trains well in simulation but coordination collapses when one agent’s policy is updated at deployment. What happened?

This is non-stationarity returning at deployment. MADDPG’s stability comes from a *centralized critic that saw all agents’ actions during training*; the learned actors are implicitly tuned to each other’s training-time behaviour. Change one agent’s policy after training and the others are now facing an off-distribution partner their decentralized actors never adapted to, coordination degrades. Fixes: retrain/fine-tune jointly after any policy change, train against a *population* of partner policies so actors are robust to partner variation (ad-hoc teamwork), or freeze the team together. It is the multi-agent face of the train/deploy distribution-shift problem.

Q10

Where is genuinely multi-agent RL actually deployed, and why is it rarer than the literature suggests?

Genuine MARL deployments are uncommon. Real-time ad bidding is a clear case: advertisers bid against each other, so each one’s optimal strategy is a direct function of the others’, not just the auction mechanism, the interaction is load-bearing. Most other production RL systems (recommendation, notification timing, dispatch, Chapter 14) are deployed as *single-agent* even though they could be framed as multi-agent (other users, other drivers), because the single-agent framing is more tractable and the interaction effects, while real, are not strong enough to justify the added complexity and instability. The senior judgment is matching the modelling complexity to how much the interaction actually drives the problem.

References

- Foerster, J., Farquhar, G., Afouras, T., Nardelli, N., & Whiteson, S. (2018). Counterfactual multi-agent policy gradients. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence* (pp. 2974–2982).
- Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., & Mordatch, I. (2017). Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems* (Vol. 30, pp. 6379–6390).
- Rashid, T., Samvelyan, M., de Witt, C. S., Farquhar, G., Foerster, J., & Whiteson, S. (2018). QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning* (pp. 4295–4304).

Part VI

Advanced Topics

Offline RL, Model-Based RL & Exploration

Subscribe → aiengineeringinsider.substack.com/subscribe

The three topics interviewers probe once you have proven you know the basics, and that most interview guides skip.

13.1 Overview

This chapter closes three gaps that separate a competent candidate from a strong one. **Offline RL** learns a policy from a fixed, previously-collected dataset with *no* further environment interaction, the setting of most real products, where online exploration is expensive or unsafe. **Model-based RL** learns a model of the environment and plans or generates synthetic experience with it, trading sample efficiency for model bias. **Exploration** beyond ϵ -greedy, count-based bonuses, curiosity, and random network distillation, is what makes hard-exploration tasks solvable at all.

The unifying thread is **what to do when data is scarce or its distribution is wrong**. Offline RL must not trust value estimates for actions absent from the data; model-based RL must not trust a learned model far from its training distribution; exploration must decide where to spend scarce interaction. All three are distribution-awareness problems wearing different clothes.

13.2 Offline RL and Distributional Shift

The core difficulty is **distributional (distribution) shift**. A bootstrapped target $r + \gamma \max_{a'} Q(s', a')$ queries Q at actions a' the learned policy prefers, but if those actions never appear in the dataset, $Q(s', a')$ is an *extrapolation* the data cannot correct. Online RL self-corrects by trying the action and observing the truth; offline RL cannot. The result is systematic overestimation of out-of-distribution (OOD) actions, and the policy chases that phantom value.

Why naive off-policy RL fails offline

Running DQN or SAC on a fixed dataset usually *diverges or degrades*, even though both are off-policy. Off-policy means the algorithm *can* learn from other policies' data; it does not mean it is safe when it can never collect corrective samples. The max (or actor) seeks high-value actions, precisely the OOD region where the value estimate is least trustworthy.

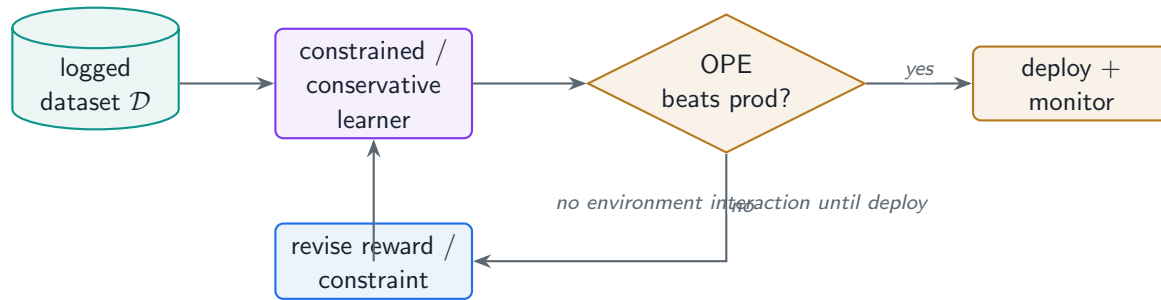


Figure 13.1: Offline RL: learn from a fixed dataset, validate by off-policy evaluation (OPE), and only *then* deploy. The agent never explores during training, which is the entire source of its difficulty.

13.2.1 The three families of offline algorithms

Policy constraint (BCQ, BRAC). Force the learned policy to stay close to the behaviour policy that generated the data, so it only ever evaluates in-distribution actions. BCQ (Fujimoto et al., 2019) generates candidate actions from a learned generative model of the data and only maximizes over those.

Conservative value (CQL). Conservative Q-Learning (Kumar et al., 2020) adds a penalty that *pushes down* Q on OOD actions while keeping it accurate on dataset actions, yielding a lower bound on the true value so the policy cannot exploit overestimation. Schematically it augments the Bellman loss with

$$\alpha \left(\mathbb{E}_{s \sim \mathcal{D}, a \sim \mu} [Q(s, a)] - \mathbb{E}_{(s, a) \sim \mathcal{D}} [Q(s, a)] \right),$$

raising in-data Q and lowering Q under a broad action distribution μ .

Implicit / in-sample (IQL). Implicit Q-Learning (Kostrikov et al., 2022) never queries an OOD action at all: it fits the value with *expectile* regression over actions *present in the data*, then extracts a policy by advantage-weighted regression. This sidesteps the OOD-max entirely and is a strong, stable default.

13.2.2 Off-policy evaluation (OPE)

You cannot A/B test offline, so before deployment you estimate the new policy's value from logged data. The estimators trade bias against variance: **importance sampling** (unbiased, but variance explodes with horizon as a product of ratios), **weighted IS** (lower variance, slight bias), **doubly robust** (combines a model and IS to reduce variance while staying low-bias), and **fitted Q-evaluation** (FQE, regression-based, low variance, model-biased). Reliable OPE is what makes offline RL deployable, and is exactly what LinkedIn and Microsoft did before launch (Chapter 14).

13.3 Model-Based RL

Model-based RL learns the dynamics $\hat{P}(s' | s, a)$ and reward $\hat{R}$, then uses them to plan or to generate synthetic transitions, slashing the amount of real interaction needed. The cost is **model bias**: errors compound over a rollout, so a long imagined trajectory drifts away from reality.

Dyna interleaves real steps with planning updates on the learned model. **MBPO** (Janner et al., 2019) keeps model rollouts *short* (a few steps, branched from real states) precisely to bound compounding error, then trains an off-policy agent on the mixture. **World models / Dreamer** (Ha & Schmidhuber, 2018; Hafner et al., 2020) learn a *latent* dynamics model and train the policy entirely “in imagination” inside that latent space, achieving strong sample efficiency. **MuZero** (Chapter 9) is model-based planning with a model learned only to predict planning-relevant quantities.

★ Key Insight

The model-based bet is “a learned model is cheaper to query than the real world.” It pays off when real samples are expensive (robotics) and the dynamics are learnable; it backfires when the model is wrong in exactly the high-value region the policy wants to exploit, the same OOD failure as offline RL, now inside the model.

13.4 Exploration Beyond ϵ -Greedy

ϵ -greedy explores blindly; on **hard-exploration** tasks with sparse, delayed reward (the canonical example is Atari’s *Montezuma’s Revenge*), it essentially never stumbles onto reward. Directed exploration adds an **intrinsic** reward for novelty:

Count-based bonuses add $1/\sqrt{N(s)}$ for rarely-visited states; **pseudo-counts** extend this to high-dimensional states via a density model (Bellemare et al., 2016). **Curiosity / ICM** (Pathak et al., 2017) rewards states where a learned forward model predicts poorly, novelty as prediction error, though it can be fooled by inherent randomness (the “noisy-TV problem”). **Random Network Distillation** (Burda et al., 2019) rewards prediction error against a *fixed random* target network, which sidesteps the noisy-TV failure and was the method that first made meaningful progress on Montezuma’s Revenge.

13.5 Comparison: Offline RL Families

Method	How it handles OOD actions	Trade-off
BCQ / BRAC	constrain policy to the behaviour distribution	safe, but capped by data quality
CQL	penalize / lower-bound Q on OOD actions	one extra coefficient α to tune
IQL	never query OOD actions (in-sample expectile)	simple, stable; mild conservatism
Model-based offline (MOPO/COMBO)	penalize model uncertainty	needs a calibrated uncertainty estimate

Table 13.1: All four answer one question, “how do you avoid trusting value estimates the data cannot support.”

13.6 Mock Interview Questions

Q1

What is distributional shift in offline RL, and why does it break off-policy methods that work online?

The learned policy prefers high-value actions, and bootstrapping queries Q at those actions in the next state. Offline, many of those actions never appear in the dataset, so Q there is an uncorrectable extrapolation that tends to be *overestimated*, and the policy chases it. Online algorithms are “off-policy” in that they can reuse old data, but they still periodically *collect* the action and observe the real return, correcting the estimate. Offline RL removes that corrective loop, so the same DQN/SAC that work online degrade or diverge on a fixed dataset. The fix is to stop trusting OOD value: constrain the policy, lower-bound the value, or only ever evaluate in-sample actions.

Q2

Compare BCQ, CQL, and IQL.

All three combat OOD overestimation, differently. BCQ is a *policy constraint*: it generates candidate actions from a learned model of the data and maximizes only over those, so it never strays far from the behaviour policy. CQL is a *conservative value* method: it adds a penalty that pushes Q down on a broad action distribution while keeping it high on dataset actions, giving a value lower bound the policy cannot exploit. IQL is *in-sample*: it fits values by expectile regression over actions actually in the data and extracts a policy by advantage-weighted regression, never querying an OOD action at all. IQL is often the most stable and simplest default; CQL is strong but adds a tuning coefficient; BCQ’s ceiling is set by data quality.

Q3

You have one year of logged production data and cannot run live experiments yet. How do you build and validate an offline policy?

Confirm the data has the actions and reward signal needed and that propensities were logged. Pick an in-sample method (IQL) or a conservative one (CQL) to avoid OOD overestimation. Validate with off-policy evaluation, doubly-robust or FQE estimates of the new policy’s value, checking action overlap with the logging policy and reporting confidence intervals, not point estimates. Only after OPE clears it do you run a small, guard-railed online pilot, expanding if online tracks the offline estimate. This is exactly the offline-then-pilot pattern Microsoft and LinkedIn used (Chapter 14).

Q4

Why is off-policy evaluation hard, and which estimator would you reach for?

Because correcting for the mismatch between the logging and target policies needs importance weights, and for sequential decisions those weights are a *product* of per-step ratios whose variance grows exponentially with horizon, ordinary IS becomes useless on long horizons. Weighted IS reduces variance at the cost of slight bias; doubly-robust estimators combine a learned value model with IS so that if either the model or the weights are good,

the estimate is reliable; FQE regresses the value directly and is low-variance but model-biased. In practice I would start with FQE or a doubly-robust estimator, check overlap, and treat any estimate as a confidence interval.

Q5

When is model-based RL worth the added complexity, and what is its central failure mode?

It is worth it when real samples are expensive or slow (robotics, slow simulators) and the dynamics are learnable, then a learned model amortizes exploration by generating cheap synthetic experience (Dreamer trains entirely in imagination; MBPO uses short model rollouts). The central failure mode is *compounding model error*: small per-step prediction errors accumulate over a rollout, so long imagined trajectories drift into fiction, and the policy exploits model errors in exactly the high-value region. Mitigations are short rollouts (MBPO), uncertainty penalties (MOPO), and latent models that predict only planning-relevant quantities (MuZero).

Q6

Explain MBPO's short-rollout idea. Why not roll the model out for the full episode?

MBPO branches *short* (k -step) model rollouts from states sampled out of the real replay buffer, then trains an off-policy agent on the mixture of real and model data. Short rollouts bound compounding error: a learned model is accurate for a few steps but drifts over many, so a full-episode imagined rollout would be dominated by model fiction and would teach the agent to exploit those errors. Starting from *real* states keeps the rollouts anchored to the true state distribution. The knob k trades sample efficiency (longer = more synthetic data) against model bias (longer = more drift), and is often annealed as the model improves.

Q7

Why does ϵ -greedy fail on Montezuma's Revenge, and how does RND help?

Reward in Montezuma's Revenge is extremely sparse and deep: you must execute a long, specific sequence before any reward appears. ϵ -greedy explores by uniform random actions, so the probability of stumbling onto that sequence is astronomically small, the agent essentially never sees reward to learn from. RND adds an *intrinsic* reward equal to the prediction error of a trained network against a fixed, randomly-initialized target network: novel states have high error (the predictor has not seen them), familiar states low error. This rewards reaching new states, driving directed exploration deep into the game. Crucially, because the target is deterministic, RND avoids the "noisy-TV" trap that fools curiosity methods based on forward-dynamics prediction error.

Q8

What is the noisy-TV problem in curiosity-driven exploration?

Curiosity methods (ICM) reward states where a learned *forward dynamics* model predicts the next observation poorly, treating prediction error as novelty. But some transitions are inherently *stochastic* (a TV showing random static, a leaf blowing randomly): they are un-

predictable forever, so they generate permanent high intrinsic reward, and the agent gets transfixed, “watching the noisy TV” instead of exploring. The fix is to base novelty on something that does not reward inherent randomness, e.g. RND’s error against a fixed deterministic target, or measuring novelty in a learned feature space that discards uncontrollable noise.

Q9

How are offline RL, model-based RL, and exploration the same problem underneath?

All three are about *not trusting estimates where the data does not support them*. Offline RL must not trust Q at OOD actions; model-based RL must not trust the learned model far from its training data; exploration is the active version, deciding where the data is thin and going to collect it. Offline RL and model-based offline methods both add conservatism/uncertainty penalties for the same reason, and exploration bonuses are the inverse sign of the same uncertainty signal (seek high-uncertainty states to learn, avoid high-uncertainty actions to stay safe). Seeing this shared “distribution-awareness” structure is the senior-level synthesis.

Q10

An offline policy scores far higher under your value model than the behaviour policy. Are you happy?

Cautious, not happy. A large estimated improvement is exactly the signature of OOD overestimation: the value model is most optimistic precisely where it is least supported by data. I would check (1) action overlap, is the new policy choosing actions the logging policy rarely took? (2) the OPE estimator’s variance and confidence interval, not just its mean; (3) agreement across independent estimators (IS vs. FQE vs. doubly-robust), divergence is a red flag; and (4) whether a conservative method (CQL/IQL) gives a similar policy. A huge offline “win” with thin support is a reason to add conservatism and run a small pilot, not to ship.

References

- Bellemare, M. G., Srinivasan, S., Ostrovski, G., Schaul, T., Saxton, D., & Munos, R. (2016). Unifying count-based exploration and intrinsic motivation. In *Advances in Neural Information Processing Systems* (Vol. 29).
- Burda, Y., Edwards, H., Storkey, A., & Klimov, O. (2019). Exploration by random network distillation. In *International Conference on Learning Representations*.
- Fujimoto, S., Meger, D., & Precup, D. (2019). Off-policy deep reinforcement learning without exploration. In *Proceedings of the 36th International Conference on Machine Learning* (pp. 2052–2062).
- Hafner, D., Lillicrap, T., Ba, J., & Norouzi, M. (2020). Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*.
- Janner, M., Fu, J., Zhang, M., & Levine, S. (2019). When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems* (Vol. 32).
- Kostrikov, I., Nair, A., & Levine, S. (2022). Offline reinforcement learning with implicit Q-learning. In *International Conference on Learning Representations*.
- Kumar, A., Zhou, A., Tucker, G., & Levine, S. (2020). Conservative Q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems* (Vol. 33, pp. 1179–1191).
- Levine, S., Kumar, A., Tucker, G., & Fu, J. (2020). Offline reinforcement learning: Tutorial, review, and

perspectives on open problems. *arXiv:2005.01643*.

Pathak, D., Agrawal, P., Efros, A. A., & Darrell, T. (2017). Curiosity-driven exploration by self-supervised prediction. In *Proceedings of the 34th International Conference on Machine Learning* (pp. 2778–2787).

Part VII

Interview Preparation

RL System Design Case Studies from Industry

Subscribe → aiengineeringinsider.substack.com/subscribe

Real, published, attributed production systems — not invented interview prompts.

Genuinely RL-specific system design questions are rarely documented as standalone interview transcripts; companies test RL system design judgment by asking you to reason through real production patterns. Rather than fabricate “Company X asked me to design Y” scenarios, this chapter compiles real, published engineering case studies of RL running in production at named companies, each cited to its original source. Treat each one as the pattern an interviewer is actually drawing on when they ask you to design an RL system, not a guess at the question itself.

Every case study below is sourced from a peer-reviewed paper, an official company engineering blog, or a named-author conference presentation. None are reconstructed from secondhand summaries without a verifiable primary source. **Each case is delivered the way you would answer a senior system-design round:** a one-line architecture diagram, then the same ten-step checklist worked through explicitly. Where a primary source does not state a detail (exploration scheme, safety bounds), the step gives the design choice a senior engineer would defend, clearly phrased as reasoning rather than a claim about the paper.

14.1 The Ten-Step RL System-Design Checklist

Every case below is answered against this exact sequence. Memorize it; it is the spine of a strong whiteboard answer.

1. **Clarify objective** — the true business goal, not a proxy metric.
2. **Formulate MDP** — is it sequential, a bandit, or supervised?
3. **Define state / action / reward** — concretely, with units.
4. **Choose RL type & algorithm** — value / policy / bandit; on/off-policy.
5. **Design simulator** — or the logged-data substitute when none exists.
6. **Exploration strategy** — how to gather data without harming users.
7. **Training pipeline** — offline vs. online, off-policy correction, cadence.
8. **Evaluation metrics** — offline estimator, then online business KPIs.
9. **Safety constraints** — guardrails, fallback, bounded actions.
10. **Deployment + monitoring** — rollout plan, logging, drift alarms.

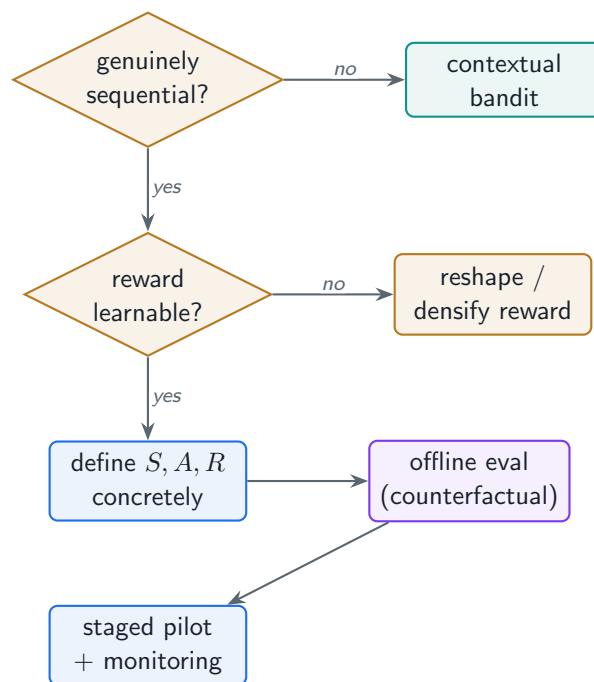
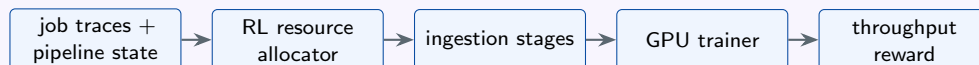


Figure 14.1: Steps 1–3 of the checklist as a decision flow: is it sequential, is the reward learnable, then formulate $S/A/R$, validate offline, and deploy in stages.

Case 1 — Netflix: InTune, RL for Data-Pipeline Optimization

Source. Nagrecha et al., ACM RecSys 2023; extended as InTuneX, ACM TORS 2024.

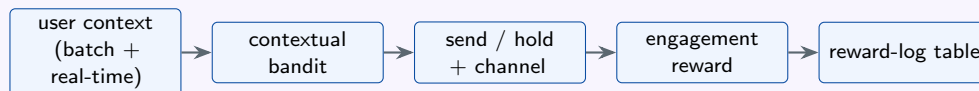


1. **Clarify objective.** Maximize DLRM *training* throughput by removing the CPU data-ingestion bottleneck, without changing the model or forcing a cluster reorganization.
2. **Formulate MDP.** Sequential resource-allocation MDP over the life of a training job: as queue depths and utilization shift, the allocation must be re-decided, so earlier choices affect later state.
3. **State / action / reward.** *State*: per-stage CPU/memory utilization and queue depths. *Action*: allocate CPU/memory across ingestion stages. *Reward*: ingestion throughput (records/s), dense and directly measurable.
4. **RL type & algorithm.** Value-based RL acting as a drop-in replacement for heuristic autotuners (`tf.data AUTOTUNE`), framework-agnostic across pipeline backends.
5. **Design simulator.** No physics simulator; historical job traces are the offline substitute, the agent is pretrained on them before touching a live job.
6. **Exploration strategy.** Bounded perturbations around the current allocation so exploration can never stall a real, expensive training run.
7. **Training pipeline.** Pretrain on logged traces, deploy, then continue tuning online; one agent generalizes across jobs and frameworks.
8. **Evaluation metrics.** Throughput up to **2.29×** the strongest baseline, CPU/GPU utilization, and multi-node cost-efficiency (15–25% in InTuneX).

9. **Safety constraints.** Allocations clamped to feasible cluster limits; the agent must never crash or starve the job, the exact failure mode of prior tools.
10. **Deployment + monitoring.** Drop-in across clusters; monitor throughput and utilization, with a fallback to AUTOTUNE on regression.

Case 2 — Wayfair: Griffin, Contextual Bandits for Notifications

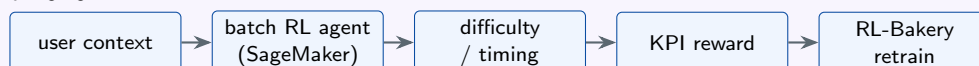
Source. Tony Cheng, Wayfair Tech Blog, August 2023.



1. **Clarify objective.** Maximize customer engagement from notifications while avoiding fatigue, coordinated across channels in real time.
2. **Formulate MDP.** Modeled as a *contextual bandit*, not a full MDP: long-horizon credit assignment is not load-bearing, so each decision is treated as single-step given context.
3. **State / action / reward.** *State*: batch features (e.g. emails clicked in 30 days) fused with real-time features (emails received in the last hour). *Action*: send/hold and which channel. *Reward*: engagement, penalized for fatigue.
4. **RL type & algorithm.** Contextual bandit balancing exploration vs. exploitation, the right tool when actions barely change future state.
5. **Design simulator.** No simulator; learns online from real customer responses, with logged data enabling replay/offline checks.
6. **Exploration strategy.** Bandit exploration (e.g. Thompson/ ϵ -style) over send decisions, with propensities logged for later off-policy evaluation.
7. **Training pipeline.** Online learning fed by a dedicated reward-logging table; features fused from batch and streaming sources.
8. **Evaluation metrics.** Engineering KPIs (latency, error rate) plus business KPIs (send rate, click-through rate).
9. **Safety constraints.** Fatigue caps and frequency limits prevent over-notifying; channel rules enforce business constraints.
10. **Deployment + monitoring.** Real-time serving with continuous KPI monitoring; the reference design for “build an RL notification system.”

Case 3 — Zynga: RL-Bakery and Game Personalization at Scale

Source. Halina & Pangler (Zynga ML Engineering); Toronto ML Summit 2019, Spark+AI Summit 2020.



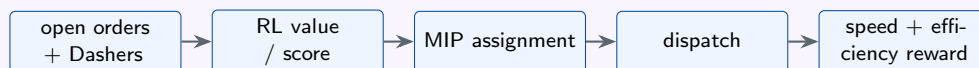
1. **Clarify objective.** Lift retention and engagement by personalizing difficulty, modes, and message timing across 60M+ MAU.
2. **Formulate MDP.** Sequential per-user MDP, but only *after* confirming the problem is

genuinely sequential and the reward learnable (Zynga’s own pre-flight check).

3. **State / action / reward.** *State*: time played, current level, recent performance. *Action*: the controlled element (next difficulty, message timing). *Reward*: a chosen KPI (levels played, session length).
4. **RL type & algorithm.** Batch (offline) RL via the open-sourced RL-Bakery on TF-Agents, fitting Zynga’s existing Spark/batch infrastructure.
5. **Design simulator.** No simulator; trains on batches of logged player trajectories.
6. **Exploration strategy.** Logged exploration in production plus periodic policy refresh; state-space kept small (compress with autoencoding) so exploration is tractable.
7. **Training pipeline.** Spark batch training → model artifact → serving layer; a game client calls the serving layer, which invokes the agent on AWS SageMaker.
8. **Evaluation metrics.** The targeted KPI (levels played, session length, retention), compared against the prior personalization baseline.
9. **Safety constraints.** Difficulty bounded to playable ranges; guard against frustrating or trivializing the game.
10. **Deployment + monitoring.** Productionized serving with periodic retraining; doubles as the canonical “*when not to use RL*” diagnostic.

Case 4 — DoorDash: RL Feeding a MIP for Dispatch

Source. DoorDash Engineering Blog (2018 hackathon; production engine “DeepRed”).

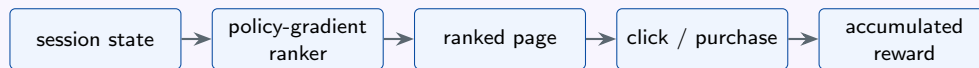


1. **Clarify objective.** Assign Dashers to deliveries to improve speed *and* courier efficiency in a three-sided marketplace (merchants, Dashers, consumers).
2. **Formulate MDP.** Sequential assignment problem with long-term marketplace effects, but the final decision is handed to a constrained optimizer.
3. **State / action / reward.** *State*: open orders, available Dashers, ETAs. *Action*: an RL-learned score/value per candidate assignment. *Reward*: delivery speed and efficiency, net of estimation variance.
4. **RL type & algorithm.** Hybrid: RL learns a scoring/value function that feeds a mixed-integer program (MIP) which performs the actual ranked assignment.
5. **Design simulator.** A marketplace simulator used to compare the RL assignment against the production algorithm before any rollout.
6. **Exploration strategy.** Exploration confined to the simulator and to learned scores; the MIP guarantees feasible assignments at serving time.
7. **Training pipeline.** Train/evaluate the scorer in simulation, then integrate it as one input to the production MIP.
8. **Evaluation metrics.** Simulated delivery speed and Dasher efficiency (RL beat the production algorithm on both).

9. **Safety constraints.** The MIP enforces hard business and feasibility constraints, RL only influences, never overrides, them.
10. **Deployment + monitoring.** RL score as a component of DeepRed; monitor speed/efficiency and the gap between estimated and realized ETAs.

Case 5 — Alibaba: RL-to-Rank in Taobao Search

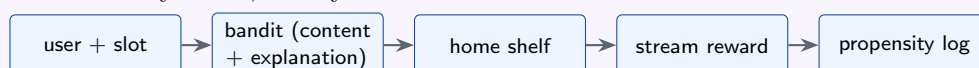
Source. Hu, Da, Zeng, Yu, & Xu, KDD 2018.



1. **Clarify objective.** Maximize session-level transaction value, not per-step click probability, across a multi-step search session.
2. **Formulate MDP.** The paper defines a Search-Session MDP (SSMDP) and *proves* you must maximize accumulated, not per-step, reward, the formal justification for going sequential.
3. **State / action / reward.** *State*: session context (queries, prior results, user actions). *Action*: the ranking emitted at each step. *Reward*: purchase value, accumulated over the session.
4. **RL type & algorithm.** A custom policy-gradient method designed to handle the high, unbalanced reward variance of the SSMDP.
5. **Design simulator.** A Taobao search simulator for offline training and comparison before live tests.
6. **Exploration strategy.** Policy-gradient stochastic exploration over rankings, with variance-reduction tailored to sparse purchase rewards.
7. **Training pipeline.** Train in simulation → validate → live deployment, with retraining on fresh session logs.
8. **Evaluation metrics.** >40% transaction-amount growth in simulation, >30% in live Taobao, including a 30% GMV lift during Double 11.
9. **Safety constraints.** Ranking must respect relevance/quality floors so revenue optimization does not degrade result quality.
10. **Deployment + monitoring.** Live ranking with GMV and session-conversion monitoring; the clean example of recasting “single-step” ranking as sequential.

Case 6 — Spotify: BaRT, Bandits for Explainable Recommendations

Source. McInerney et al., RecSys 2018.

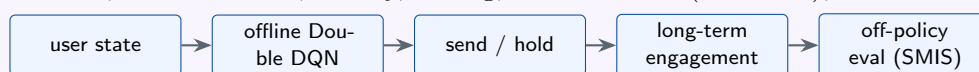


1. **Clarify objective.** Increase stream rate by jointly choosing *which* content to recommend and *how* to explain it (“recsplanations”).
2. **Formulate MDP.** A (contextual) multi-armed bandit: the decision is single-step, so no long-horizon MDP is needed despite the multi-objective payoff.

3. **State / action / reward.** *State*: user context and Home-page slot. *Action*: the joint (content, explanation) choice. *Reward*: stream rate.
4. **RL type & algorithm.** Multi-armed bandit (BaRT, “Bandits for Recommendations as Treatments”) balancing explore/exploit.
5. **Design simulator.** No simulator; instead, propensity-based counterfactual estimation from logged data plays the validation role.
6. **Exploration strategy.** Randomized exploration with logged propensities, the prerequisite for unbiased offline evaluation.
7. **Training pipeline.** Learn from logged interactions; counterfactually evaluate candidate policies before online tests.
8. **Evaluation metrics.** Offline counterfactual estimates first, then a significant online stream-rate lift over non-bandit baselines.
9. **Safety constraints.** Explanations must be truthful and on-brand; exploration capped to protect Home-page experience.
10. **Deployment + monitoring.** Home-page serving with stream-rate monitoring; a model example of rigorous offline-before-online discipline.

Case 7 — LinkedIn: Offline RL for Mobile Notifications

Source. Yuan, Muralidharan, Nandy, Cheng, & Prabhakar (LinkedIn), 2022.

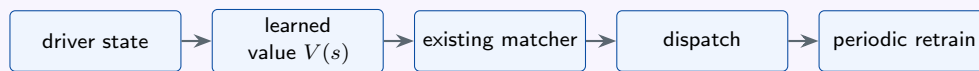


1. **Clarify objective.** Optimize *sequential* notification decisions for long-term engagement, which per-notification response models structurally cannot attribute.
2. **Formulate MDP.** A genuine sequential MDP over a user’s notification stream, solved entirely offline (no live exploration on members).
3. **State / action / reward.** *State*: user engagement state. *Action*: send/hold. *Reward*: long-term engagement.
4. **RL type & algorithm.** Offline (batch) RL with a Double Deep Q-Network, chosen to curb overestimation in the off-policy setting.
5. **Design simulator.** No simulator; trained on logged data from *online exploration* already running in production.
6. **Exploration strategy.** Exploration happened earlier in the production system; the offline learner reuses that logged exploratory data.
7. **Training pipeline.** Offline Double DQN training on logged exploration data, then a single vetted launch.
8. **Evaluation metrics.** State-marginalized importance sampling (SMIS) for off-policy evaluation; the policy then beat and replaced the prior survival model online.
9. **Safety constraints.** Conservative offline learning to avoid extrapolating to never-seen actions; frequency limits on sends.

10. Deployment + monitoring. Launched after offline validation; monitor long-term engagement and distribution shift versus the training logs.

Case 8 — Uber: Learned Value Signal for Marketplace Matching

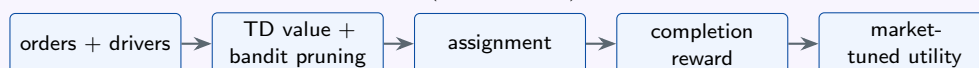
Source. Uber Engineering Blog.



- 1. Clarify objective.** Match drivers to riders accounting for the *long-term* value of keeping a driver engaged, not just immediate match value.
- 2. Formulate MDP.** An infinite-horizon MDP over driver states, with a periodic evaluation-improvement loop.
- 3. State / action / reward.** *State*: driver state (location, supply/demand context). *Action*: matching influenced by a learned value. *Reward*: long-term marketplace value.
- 4. RL type & algorithm.** DQN-inspired value learning: learn $V(\text{driver state})$ used as a *signal* inside the existing matcher, not an end-to-end policy.
- 5. Design simulator.** A marketplace simulator/offline evaluation for the value function before injection.
- 6. Exploration strategy.** Minimal live exploration; the value signal nudges a proven matcher, limiting risky deviations.
- 7. Training pipeline.** Periodic re-training (policy evaluation $\rightarrow$ improvement) on fresh marketplace data.
- 8. Evaluation metrics.** Marketplace efficiency/match outcomes; deployed in 400+ cities, described as the largest production RL matching system in ridesharing.
- 9. Safety constraints.** The matcher retains hard constraints; the learned value can only re-weight, not violate, them.
- 10. Deployment + monitoring.** Value signal feeds the live matcher; monitor supply/demand balance and re-train on cadence.

Case 9 — DiDi: Standalone RL Dispatching at Scale

Source. Sadeghi Eshkevari, Tang, Qin (DiDi Labs) et al.



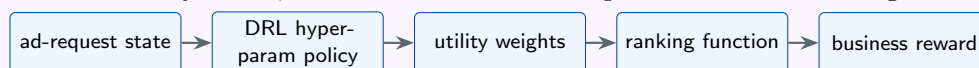
- 1. Clarify objective.** Replace myopic/rule-based dispatch with a forward-looking policy that improves completion and reduces cancellations.
- 2. Formulate MDP.** A sequential dispatching MDP tailored to real-time, large-scale uncertainty in the marketplace.
- 3. State / action / reward.** *State*: outstanding orders and driver positions. *Action*: driver-order assignment. *Reward*: a market-statistics-tuned utility for completion value.
- 4. RL type & algorithm.** Temporal-difference value learning plus a multi-armed-bandit

adaptive graph-pruning step to cut post-assignment cancellations, a more *standalone* RL dispatcher than Uber’s signal-injection design.

5. **Design simulator.** A city-scale dispatch simulator for training and tuning the utility function.
6. **Exploration strategy.** TD-based exploration with bandit pruning that adaptively trims risky assignment edges.
7. **Training pipeline.** Simulator training with market-tuned reward, then staged real-world rollout.
8. **Evaluation metrics.** Completion rate, cancellation reduction, and dispatch efficiency versus rule-based baselines.
9. **Safety constraints.** Pruning bounds cancellation risk; assignment respects operational feasibility.
10. **Deployment + monitoring.** Large-scale deployment; the direct architectural contrast to Uber (more decision authority, more risk and upside).

Case 10 — Pinterest: Deep RL for Ad-Ranking Utility Tuning

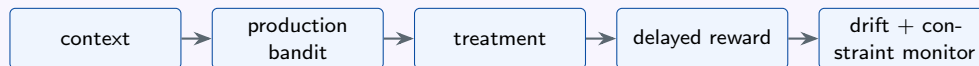
Source. ACM RecSys 2025, validated on Pinterest’s production ads ranking.



1. **Clarify objective.** Replace unprincipled manual tuning of the ad ranker’s utility weights with personalized, seasonality-aware tuning.
2. **Formulate MDP.** A per-request decision MDP where the agent sets *hyperparameters* of an existing ranker rather than picking ads directly.
3. **State / action / reward.** *State*: the ad request. *Action*: the optimal utility hyperparameters. *Reward*: a predefined business objective.
4. **RL type & algorithm.** DRL-PUT, a deep-RL framework for personalized utility tuning (RL tunes a function, it does not choose the action).
5. **Design simulator.** Offline replay over logged ad requests stands in for a simulator.
6. **Exploration strategy.** Exploration over hyperparameter settings, bounded so served rankings stay sensible.
7. **Training pipeline.** Train on logged requests/outcomes; deploy the tuner alongside the existing ranking stack.
8. **Evaluation metrics.** The predefined business reward; validated in Pinterest’s production ranking system.
9. **Safety constraints.** Hyperparameters constrained to safe ranges so the ranker never degrades catastrophically.
10. **Deployment + monitoring.** Low-risk, auditable placement (tunes weights); monitor business KPIs and weight drift.

Case 11 — Booking.com: What Breaks Bandits in Production

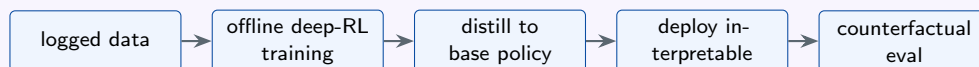
Source. van den Akker, Weber, Moraes, & Goldenberg (Booking.com).



- 1. Clarify objective.** Run bandit-driven personalization at e-commerce scale while surviving the realities textbooks ignore.
- 2. Formulate MDP.** Contextual bandit, but the paper's contribution is everything that complicates that clean formulation in production.
- 3. State / action / reward.** *State*: customer/session context. *Action*: a treatment (variant). *Reward*: conversion, which arrives *delayed*.
- 4. RL type & algorithm.** Production bandits (on- and off-policy), with the choice between them itself a documented challenge.
- 5. Design simulator.** They extended the open-source Open Bandit Pipeline with simulation components for delayed reward, drift, and constraints.
- 6. Exploration strategy.** Logged-only exploration in many cases; off-policy methods reused historical data rather than exploring freely.
- 7. Training pipeline.** Offline learning/evaluation with the simulator add-ons, handling concept drift via refresh.
- 8. Evaluation metrics.** Realistic reward definitions and off-policy estimates that account for delay and a changing arm set.
- 9. Safety constraints.** External business rules constrain which arms are available, and the set changes constantly.
- 10. Deployment + monitoring.** Continuous monitoring for drift and delayed-reward attribution, the five named production challenges.

Case 12 — Meta: Horizon, and Train-Complex / Deploy-Interpretable Bidding

Source. Gauci et al. (Horizon), arXiv 2018; Korenkevych et al. (production bidding), AI at Meta.

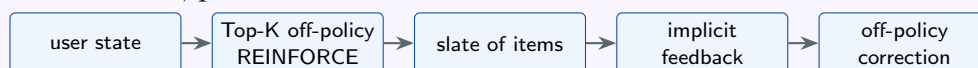


- 1. Clarify objective.** Optimize advertiser bidding (and other surfaces) at scale while keeping deployed decisions *explainable* to advertisers.
- 2. Formulate MDP.** Sequential bidding/decision MDPs with slow real feedback and no usable simulator.
- 3. State / action / reward.** *State*: campaign/auction context. *Action*: bid (or notification/stream decision). *Reward*: advertiser value / engagement.
- 4. RL type & algorithm.** Offline deep RL during training; a hybrid that pairs an *interpretable base policy* with a deep net, then discards the net and ships only the optimized base-policy parameters.

5. **Design simulator.** No simulator; Horizon emphasizes counterfactual policy evaluation as the simulator substitute.
6. **Exploration strategy.** Exploration via logged data; careful experimentation because there is no cheap simulator to fall back on.
7. **Training pipeline.** Horizon, an end-to-end applied-RL platform (datasets of millions–billions of records, CPE, optimized serving).
8. **Evaluation metrics.** Counterfactual policy evaluation, then statistically significant gains in simulation and at-scale production; RL replaced supervised systems for notifications and video streaming.
9. **Safety constraints.** Shipping only an interpretable policy bounds and audits behavior; advertisers can reason about decisions.
10. **Deployment + monitoring.** Optimized serving of the distilled policy; the model answer for “deploy RL where decisions must be explained.”

Case 13 — Google/YouTube: Top-K Off-Policy REINFORCE

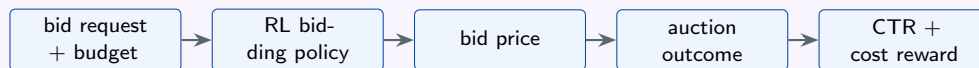
Source. Chen et al., production recommender at YouTube.



1. **Clarify objective.** Recommend from a catalog of *millions* of items to maximize long-term user satisfaction.
2. **Formulate MDP.** Sequential recommendation MDP learned off-policy from logged, biased implicit feedback collected under many past policies.
3. **State / action / reward.** *State*: user history/context. *Action*: an item, extended to a Top-K slate. *Reward*: implicit engagement.
4. **RL type & algorithm.** Policy-gradient REINFORCE with off-policy correction and a Top-K (slate) extension; a factorized/sampled softmax keeps the million-action policy tractable.
5. **Design simulator.** No simulator; the logged interaction stream is the data source.
6. **Exploration strategy.** Exploration is implicit in the diversity of past serving policies; importance weights correct the resulting bias.
7. **Training pipeline.** Off-policy REINFORCE on logged feedback, importance-weighting by current/logging-policy ratio.
8. **Evaluation metrics.** Engagement lift; the standard reference for production-scale policy gradients (extends Chapter 6).
9. **Safety constraints.** Content-policy filtering on candidates; off-policy correction prevents over-trusting rare logged actions.
10. **Deployment + monitoring.** Live recommendation with propensity logging; monitor engagement and off-policy variance.

Case 14 — Yahoo!: Real-Time Bidding by RL

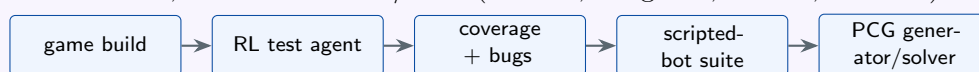
Source. Cai, Ren, Zhang, Malialis, Wang, Yu, & Guo, WSDM 2017.



- 1. Clarify objective.** Maximize clicks/value within a budget in real-time display-ad auctions, jointly improving CTR estimation and bidding.
- 2. Formulate MDP.** A budget-constrained sequential bidding MDP: each bid consumes budget and changes the feasible set of future bids.
- 3. State / action / reward.** *State*: request features plus remaining budget. *Action*: bid price. *Reward*: clicks/value net of cost.
- 4. RL type & algorithm.** Value-based RL bidding combined with a CTR-estimation model.
- 5. Design simulator.** An auction simulator built from historical bid logs for training the bidding policy.
- 6. Exploration strategy.** Exploration over bid levels in simulation, then conservative online tuning under the budget constraint.
- 7. Training pipeline.** Train policy + CTR model offline on logs, validate in the auction simulator, then live A/B.
- 8. Evaluation metrics.** Online A/B: **2.97% AUC** lift for CTR estimation and a **9.30% eCPC** drop for bid optimization.
- 9. Safety constraints.** Hard budget caps and bid ceilings prevent overspend.
- 10. Deployment + monitoring.** Deployed on Yahoo!'s RTB platform; monitor spend pacing, eCPC, and CTR calibration, one of the few cases with named A/B business metrics.

Case 15 — Electronic Arts (SEED): RL for AAA Testing and Gameplay

Source. EA SEED, IEEE CoG 2020/2023 (Gisslén, Bergdahl, Sestini, Tollmar).



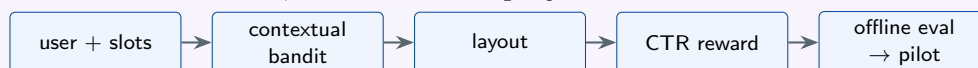
- 1. Clarify objective.** Raise automated test coverage in large AAA titles, and ship more human-like in-game behavior, where scripted bots and human testers each fall short.
- 2. Formulate MDP.** Sequential control MDPs inside the game (navigate/play to explore states and find bugs).
- 3. State / action / reward.** *State*: game observations. *Action*: in-game controls. *Reward*: coverage/exploration for testing, or play quality for gameplay features.
- 4. RL type & algorithm.** An RL test agent integrated into a scripted-bot suite; a separate adversarial line trains Generator and Solver agents jointly for procedural content/difficulty.
- 5. Design simulator.** The game build *is* the simulator, but it differs from the shipped

product, the core research-to-production gap.

- 6. Exploration strategy.** Coverage-driven exploration to reach rare states and surface bugs.
- 7. Training pipeline.** Train against game builds; integrate with existing automated QA tooling.
- 8. Evaluation metrics.** Test coverage and bugs found (deployed on Battlefield 2042 and Dead Space 2023); EA SPORTS FC 26 ships RL goalkeeper behavior.
- 9. Safety constraints.** Shipped gameplay agents must stay believable and bounded; test agents must not corrupt builds.
- 10. Deployment + monitoring.** RL in both QA tooling and a player-facing feature; the CoG 2023 “Lessons Learned” centers on the dev-vs-shipped-environment gap.

Case 16 — Microsoft: Azure Personalizer / Xbox Top-of-Home Pilot

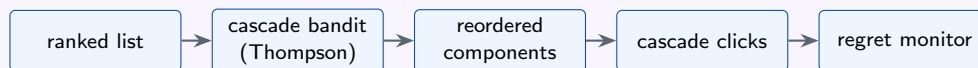
Source. Microsoft Research, Real-World RL project documentation.



- 1. Clarify objective.** Personalize which content fills high-traffic Xbox “Top of Home” slots, where static A/B testing cannot adapt per user.
- 2. Formulate MDP.** Contextual bandit, each slot-fill is a single-step contextual decision.
- 3. State / action / reward.** *State*: user context plus the slot. *Action*: content to place. *Reward*: click-through.
- 4. RL type & algorithm.** The Real-World RL platform behind Azure Personalizer (contextual bandit), personalizing two of three slots.
- 5. Design simulator.** Counterfactual evaluation on logged data plays the simulator role before any live exposure.
- 6. Exploration strategy.** Logged exploration with propensities, enabling unbiased counterfactual estimates.
- 7. Training pipeline.** Two-phase: offline counterfactual evaluation, then a time-boxed online pilot.
- 8. Evaluation metrics.** Counterfactual eval on two weeks of June 2018 data, then a November 2018 pilot with a **60% CTR** lift over random.
- 9. Safety constraints.** Time-boxed, slot-limited rollout caps blast radius; baseline fallback always available.
- 10. Deployment + monitoring.** The clean low-risk rollout template: validate offline, pilot, then scale, presenting a deployment plan, not just an algorithm.

Case 17 — Expedia: Cascade Bandits for Ranking

Source. Andrea Marchini, Expedia Group Technology engineering blog.



1. **Clarify objective.** Improve home-page component ranking where supervised learning-to-rank on noisy implicit signals converges to suboptimal orders.
2. **Formulate MDP.** A cascade bandit: model the *cascade click* process (users scan top-down) rather than treating positions independently.
3. **State / action / reward.** *State*: ranking context. *Action*: the ordered list of components. *Reward*: clicks under the cascade model.
4. **RL type & algorithm.** CascadeLinTS, a Thompson-sampling cascade bandit, exploring alternative rankings while protecting UX.
5. **Design simulator.** A click-model simulator (cascade) for measuring cumulative regret offline.
6. **Exploration strategy.** Thompson sampling for principled exploration of orderings.
7. **Training pipeline.** Online bandit learning with the cascade assumption baked into the reward model.
8. **Evaluation metrics.** Cumulative regret, where cascade bandits beat a greedy no-exploration baseline after 100k interactions.
9. **Safety constraints.** Exploration bounded to maintain an acceptable user experience on the live page.
10. **Deployment + monitoring.** Live ranking with regret/engagement monitoring; choosing the click model is itself a modeling decision interviewers probe.

14.2 Worked Senior System-Design Walkthroughs

The case boxes above show the ten-step checklist applied to real systems; this section shows how to *deliver* a full design end to end in a 45-minute senior round at a company like Google, Meta, or Apple. Senior interviewers grade structure, scale reasoning, offline-before-online discipline, and explicit trade-offs.

14.2.1 Walkthrough 1 — RL Feed Recommendation (Meta / Google scale)

1. **Clarify.** The objective is long-term engagement (e.g. 28-day retention or session depth), *not* immediate click-through, this is why it is RL, not a one-shot classifier. Non-functional: p99 ranking latency under ~ 100 ms, billions of daily requests, a catalog of 10^8+ items, and a hard guardrail against integrity/policy violations.
2. **Scale.** At, say, 10^6 QPS and 10^8 items, scoring every item per request is impossible, so the system is two-stage: cheap **candidate generation** ($10^8 \rightarrow 10^3$) then expensive **ranking** ($10^3 \rightarrow$ ordered slate). The RL policy lives in ranking; candidate generation stays a fast retrieval model.

3. Formulate. *State*: user features (long-term embedding + real-time session context). *Action*: the ranked slate selected from candidates (Top-K). *Reward*: a long-horizon engagement signal with a KL/diversity regularizer and hard integrity filters. Because logs come from past serving policies, this is an *off-policy* problem (YouTube’s Top-K off-policy REINFORCE, Case 13).

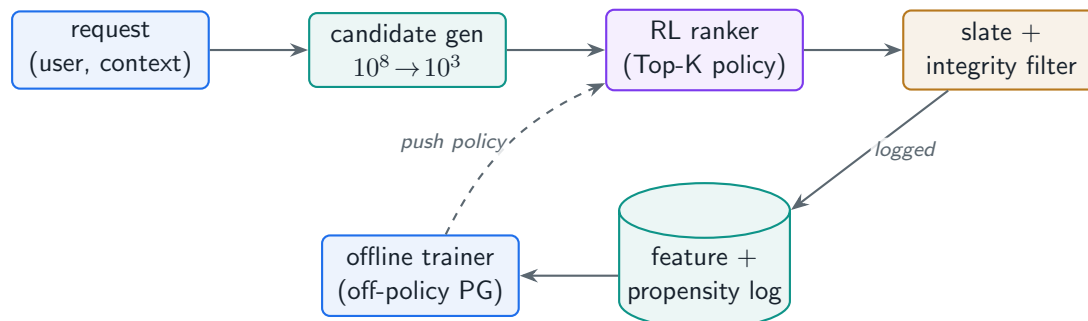


Figure 14.2: Two-stage recommender: retrieval narrows the catalog, the RL policy ranks, and propensity-logged outcomes feed an offline off-policy trainer that periodically pushes a new policy.

5. Training. Train offline from logged data with **off-policy correction** (importance weighting by the ratio of current-policy to logging-policy propensity, hence propensity must be logged at serving time). Refresh the policy on a daily/hourly cadence; never explore live on the full population.

6. Evaluation. Counterfactual/doubly-robust offline policy evaluation first, check the new policy’s estimated value beats production with confidence intervals and adequate log overlap, then a small-traffic online A/B with engagement and integrity guardrail metrics, expanding only if online tracks offline.

7. Trade-offs. Off-policy variance explodes if the new policy drifts far from the logging policy (clip/constrain). Long-horizon reward risks feedback loops and filter bubbles (add diversity/exploration). Reward is a proxy, so monitor for engagement-bait over-optimization, the recsys face of reward hacking (Chapter 11).

14.2.2 Walkthrough 2 — RLHF Pipeline for an LLM (frontier-lab scale)

1. Clarify. Objective: make a pretrained LLM more helpful and harmless per human preference, while *preserving* base capabilities. Non-functional: reproducible, auditable, and safe, no degenerate or unsafe policy ships.

2. Scale. Reward-model training on 10^5 – 10^6 preference pairs; PPO fine-tuning generates millions of on-policy completions; serving must meet token-latency SLAs. The expensive resource is human preference labels, hence RLAIIF/Constitutional methods to scale labeling (Chapter 11).

3. Formulate. *State*: prompt + tokens generated so far. *Action*: next token. *Reward*: a scalar from the learned reward model on the completed response, minus a per-token KL penalty to a frozen reference (SFT) model that both regularizes and densifies the signal.

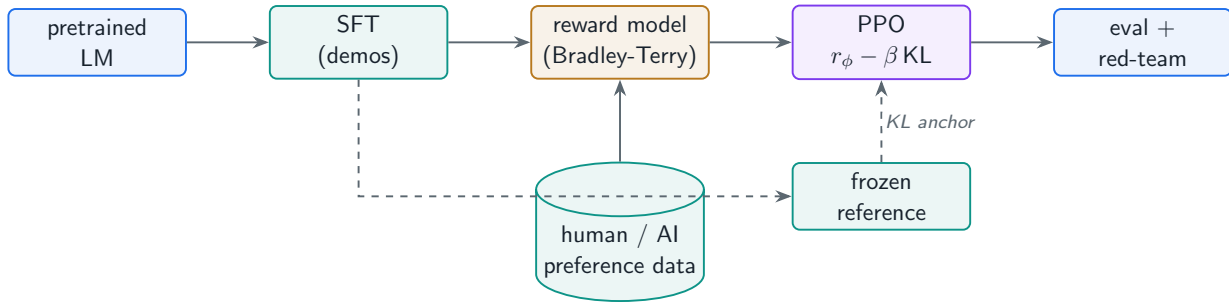


Figure 14.3: RLHF pipeline: SFT initializes the policy and the frozen reference; a Bradley-Terry reward model scores completions; PPO optimizes reward minus a KL leash; an eval/red-team gate precedes any release.

5. Training. Three stages, SFT $\rightarrow$ reward model $\rightarrow$ PPO, with the KL penalty preventing drift and capability loss. Consider DPO to remove the reward-model and RL loop where a fixed preference set suffices, or GRPO with verifiable rewards for reasoning, which sidesteps reward over-optimization entirely (Chapter 11).

6. Evaluation. A held-out human-eval set never used for reward-model training; track win-rate vs. the SFT baseline, plus the KL from reference as an early-warning of over-optimization (large KL with no held-out gain is the red flag). Add automated safety evals and adversarial red-teaming as a release gate.

7. Trade-offs. The reward model is a proxy, optimize it too hard and quality plateaus then degrades (Goodhart). Mitigate with the KL leash, reward-model regularization, periodic reward-model refresh, and ensembling. Be explicit that “higher reward-model score” is not the goal; held-out human quality is.

14.3 Cross-Case Summary

Company	Domain	RL flavour	Design lesson
Netflix	data-pipeline tuning	online RL (resource alloc)	RL for infrastructure; dense learnable reward
Wayfair	notifications	contextual bandit	bandit when no long credit horizon
Zynga	game personalization	batch RL	“when not to use RL” diagnostic
DoorDash	dispatch	RL score $\rightarrow$ MIP	RL feeds classical optimization
Alibaba	search ranking	session MDP + PG	recast ranking as sequential
Spotify	rec + explanation	multi-armed bandit	multi-objective + offline counterfactual
LinkedIn	notifications	offline Double DQN	solves attribution; offline eval first
Uber	marketplace matching	learned value signal	value function feeds matcher

Company	Domain	RL flavour	Design lesson
DiDi	dispatch	TD value + bandit pruning	contrast to Uber's choice
Pinterest	ad-utility tuning	DRL hyperparameter tuning	RL tunes a function, not the action
Booking.com	e-commerce bandits	production bandits	the gap textbooks ignore
Meta	notifications, bidding	Horizon; offline RL	train complex, deploy interpretable
YouTube	recommendation	Top-K off-policy REINFORCE	PG at million-action scale
Yahoo!	real-time bidding	RL bidding	named A/B business metrics
EA SEED	game testing/NPCs	RL + adversarial PCG	research-to-production gap
Microsoft	Xbox layout	contextual bandit (Personalizer)	offline-then-pilot rollout
Expedia	ranking	cascade bandit (Thompson)	cascade click model

14.4 How to Use These Case Studies in an Interview

When asked to design an RL system for a given domain, the strongest answers borrow structure from real systems like these rather than reciting algorithm names in the abstract. A reliable sequence: first, establish whether the problem is genuinely sequential and whether the reward is learnable, Zynga's own diagnostic from Case 3. Second, decide between full RL and a lighter contextual-bandit framing, as Wayfair and Spotify did, based on whether the action space and credit-assignment horizon actually require full sequential RL. Third, describe the state, action, and reward formulation concretely, the way Zynga's and LinkedIn's teams did, rather than abstractly. Fourth, address the production realities interviewers will probe: feature freshness (batch versus real-time, as in Griffin), reward logging and monitoring infrastructure, and how the system is validated before full production rollout, whether via offline counterfactual evaluation (Microsoft, LinkedIn) or a staged pilot (Microsoft). Finally, be ready to discuss what's hard to get right, sparse rewards, state-space sensitivity, and the gap between offline and online performance, since this is precisely the "respect how difficult RL is in practice" signal interviewers are listening for throughout this entire book.

14.5 Mock Interview Questions

Q1

Design an RL system to optimize push notifications. Walk me through it end to end.

I would first ask whether the problem is genuinely sequential: a single send affects fatigue and future opens, so yes, but I would check whether a contextual bandit suffices (Wayfair's Griffin) before committing to full RL (LinkedIn's offline DQN). State: user engagement history (recency, frequency, recent opens), fused batch and real-time features. Action: send / hold, channel, timing. Reward: long-term engagement, which is why response-prediction models fail, they cannot attribute it across a sequence. I would train offline from logged exploration data with off-policy evaluation (state-marginalized importance sampling), validate with held-out counterfactual eval, then run a time-boxed online pilot with reward logging and KPI monitoring before full rollout.

Q2

When would you use a contextual bandit instead of full RL in a production recommender?

When actions do not meaningfully change future state, or the credit-assignment horizon is short. Spotify's BaRT and Wayfair's Griffin use bandits because the decision is essentially "which item/explanation now," with exploration the main challenge, not long-horizon planning. Bandits are simpler, far easier to evaluate offline (propensity/counterfactual methods are well-developed), and avoid RL's instability. I would escalate to full RL only when session-level correlation is load-bearing, as Alibaba showed for Taobao search, where treating ranking steps independently left more than 30% GMV on the table.

Q3

A stakeholder needs to explain the agent's decisions. How do you deploy RL while keeping it interpretable?

Meta's production bidding work is the template: train with a complex model (an interpretable base policy plus a deep network) but *discard the network after training*, deploying only the optimized, explainable base-policy parameters. You get the optimization benefit of deep RL during training and a transparent, auditable policy at serving time. Alternatively, use RL to *tune* an existing interpretable function rather than replace it, Pinterest's DRL-PUT tunes ranking-utility hyperparameters; Uber uses a learned value function as one signal into a separately-engineered matcher. The pattern: keep the deployed decision surface simple, push the complexity into training.

Q4

How would you validate an RL policy offline before risking online traffic?

Counterfactual / off-policy evaluation on logged data, the approach Microsoft used for Xbox (two weeks of June 2018 data before the November pilot) and LinkedIn used before launching its DQN. Concretely: importance-sampling or doubly-robust estimators to estimate the new policy's value under the logging policy, with attention to variance (state-marginalized IS for

long horizons). I would also check overlap (does the logging policy cover the new policy's actions?) and report confidence intervals, not point estimates. Then a staged rollout: small traffic slice, monitor business and engineering KPIs, expand only if the online numbers track the offline estimate.

Q5

Uber and DiDi both solve dispatch with RL but differ architecturally. Compare the choices.

Uber learns a *value function* over driver states (DQN-inspired) and injects it as a signal into its existing online matching algorithm, an infinite-horizon MDP with periodic re-training, deployed in 400+ cities. DiDi builds a more *standalone* RL dispatcher with TD value updating, a market-tuned utility function, and a bandit-based graph-pruning step to cut post-assignment cancellations. The lesson is that there is no single correct RL architecture for a problem class: Uber minimizes risk by augmenting a proven matcher, while DiDi takes more of the decision into the RL system. Which is right depends on how much you trust the existing system and your tolerance for end-to-end RL risk.

Q6

What production realities break textbook bandit algorithms, per Booking.com?

Five, from their paper: (1) on-policy vs. off-policy choice when you can only log, not freely explore; (2) delayed rewards, the outcome (a booking) arrives long after the action; (3) concept drift, user behaviour and inventory shift, so a converged policy goes stale; (4) defining a realistic reward that captures the true objective rather than a convenient proxy; and (5) optimizing under business-rule constraints with a constantly changing set of available arms. The meta-point for an interview: textbook regret bounds assume stationarity, immediate reward, and a fixed arm set, exactly the assumptions production violates, so much of the real work is engineering around those gaps.

Q7

How do you scale policy gradients to a recommender with millions of items?

YouTube's Top-K off-policy REINFORCE is the reference. Three pieces: (1) scale REINFORCE to a million-action softmax via factorized/sampled-softmax structure; (2) off-policy correction, importance-weight logged actions by current-policy/logging-policy ratio, because the data comes from many past behaviour policies; and (3) a Top-K extension for slates, since you show multiple items, not one. The framing: the core policy-gradient theorem does not change; you are layering off-policy correction and action-space scaling as separate engineering problems on top of it, exactly the lesson from Chapter 6.

Q8

When is RL the wrong tool, and how do you tell early? Use a real example.

Zynga's own diagnostic: ask first whether the problem is *genuinely sequential* (does one action change future ones) and whether the *reward is learnable* (sparse rewards are very hard). If not sequential, it is a bandit or even a supervised problem; if the reward is unlearnable, fix the reward before touching the algorithm. They also warn that agents are sensitive to overly large state spaces and recommend compression (autoencoding). The

strongest interview signal is treating RL as a fragile, high-leverage instrument you justify, not a default, candidates who can say “this should not be RL, here is why” rate higher than those who reach for it reflexively.

Q9

Design the monitoring and reward-logging infrastructure for a deployed RL system.

Wayfair’s Griffin is the model: a dedicated reward-logging table plus a monitoring system spanning engineering metrics (latency, error rates, serving availability) and business metrics (send rates, click-through, downstream conversion). Concretely I would log, per decision: the context/features, the action taken, the policy’s propensity for that action (essential for later off-policy evaluation), and the eventual reward with its delay. I would alert on distribution shift in features, drops in propensity coverage, and divergence between predicted and realized reward, the early-warning sign of drift or reward over-optimization. Without propensity logging from day one, you cannot do principled offline evaluation later.

Q10

Why do so many production systems use RL as one component feeding a classical system, rather than end-to-end?

Because it bounds risk and exploits existing, trusted infrastructure. DoorDash uses an RL/learned score inside a mixed-integer program; Uber feeds a learned value into its matcher; Pinterest tunes a ranking function’s hyperparameters. End-to-end RL controlling the final decision is harder to validate, harder to explain, and riskier to ship, while a learned signal into a well-understood optimizer keeps the system debuggable and the failure modes contained. The senior judgment is recognizing that “where does RL sit in the stack” is itself a core design decision, and that the lowest-risk high-value placement is often a signal, not the whole controller.

References

- Cai, H., Ren, K., Zhang, W., Malialis, K., Wang, J., Yu, Y., & Guo, D. (2017). Real-time bidding by reinforcement learning in display advertising. In *Proceedings of the 10th ACM International Conference on Web Search and Data Mining* (pp. 661–670).
- Gauci, J., Conti, E., Liang, Y., Virochsiri, K., He, Y., Kaden, Z., ... Fujimoto, S. (2018). Horizon: Facebook’s open source applied reinforcement learning platform. *arXiv:1811.00260*.
- Hu, Y., Da, Q., Zeng, A., Yu, Y., & Xu, Y. (2018). Reinforcement learning to rank in e-commerce search engine. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 368–377).
- McInerney, J., Lacker, B., Hansen, S., Higley, K., Bouchard, H., Gruson, A., & Mehrotra, R. (2018). Explore, exploit, and explain: Personalizing explainable recommendations with bandits. In *Proceedings of the 12th ACM Conference on Recommender Systems* (pp. 31–39).
- Nagrecha, K., Liu, L., Delgado, P., & Padmanabhan, P. (2023). InTune: Reinforcement learning-based data pipeline optimization for deep recommendation models. In *Proceedings of the 17th ACM Conference on Recommender Systems* (pp. 430–442).

Top 50 Most Asked Questions

Subscribe → aiengineeringinsider.substack.com/subscribe

Section A is sourced and verifiable. Section B is compiled. Both are clearly labeled, and that distinction matters.

A note on how this chapter was built

Section A below contains only questions traceable to a named company, role, and source, drawn from candidate interview reports and named-author accounts. Section B is compiled by the author from confirmed RL topic areas tested at the companies named throughout this book, combined with the standard curriculum this guide has built chapter by chapter. Section B questions are written in the style and at the depth real interviewers use, but they are not verbatim transcripts, and this guide does not claim otherwise.

! Pitfall / Warning

Several otherwise reputable-looking “Top 50 RL interview questions” lists circulating online present compiled, generated content as verified real transcripts with no traceable source. This chapter is built the opposite way deliberately: where a real source exists, it’s cited; where it doesn’t, the question is labeled as compiled. Treat any resource that doesn’t make this distinction with appropriate skepticism.

15.1 How RL Interviews Are Weighted by Company Archetype

The same fifty questions are not asked everywhere. Use this map to bias your preparation toward the archetype you are interviewing for.

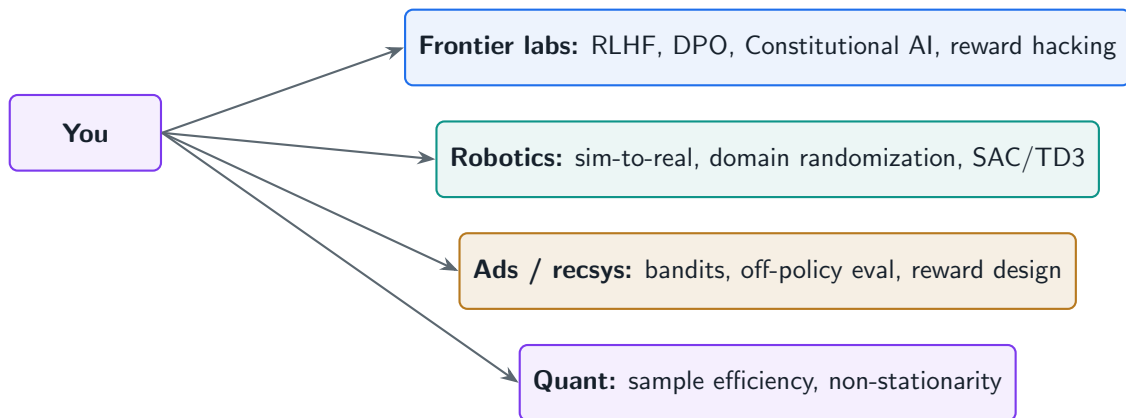


Figure 15.1: Topic weighting by company archetype. The fundamentals (Chapters 1–8) are common to all; the emphasis above is what differs.

Topic cluster	Typical questions	Chapters
Foundations	MDP, Bellman, DP, bias–variance	1–3
Value-based	Q-learning, SARSA, DQN, Rainbow	4–5
Policy gradient	REINFORCE, A2C, PPO, TRPO	6–8
Continuous control	DDPG, TD3, SAC, max-entropy	8
Games / planning	AlphaZero, MCTS, self-play	9
Sim-to-real	reality gap, domain randomization	10
Alignment	RLHF, DPO, RLAIF, GRPO, reward hacking	11
Multi-agent	CTDE, QMIX, non-stationarity	12
Offline / model-based	distribution shift, CQL/IQL, OPE, Dreamer, RND	13
System design	production patterns, offline eval	14

Table 15.1: Use this to locate the full technical answer for any question below in its source chapter.

15.2 Section A — Verified Real Interview Reports

Q1

Explain the difference between PPO and GRPO. *Source: Google DeepMind, Student Researcher role, Cambridge MA, May 2025 (candidate report via Glassdoor).* Asked as part of a technical round following a deep-dive on the candidate’s prior research projects, alongside questions on transformers and LLM reasoning. See Chapter 8 (PPO) and Chapter 11 (GRPO-style reasoning RL) for the full technical answer.

Q2

How does Constitutional AI differ from traditional RLHF? *Source: Anthropic, recurring question across multiple roles (candidate-reported, 2025–2026).* Tests whether candidates can explain Constitutional AI as a self-critique-and-revision mechanism distinct from, and complementary to, RLHF’s preference-based signal. See Chapter 11.

Q3

What are the limitations of Constitutional AI as an alignment approach? *Source: Anthropic (candidate-reported, same source as above).* A follow-up probing whether candidates discuss limitations honestly rather than presenting Constitutional AI as a solved problem.

Q4

If you were writing a constitution for a powerful AI system, what principles would you prioritize and why? *Source: Anthropic (candidate-reported).*

Q5

How would you measure whether an AI system is actually following its stated principles? *Source: Anthropic (candidate-reported).*

Q6

Explain RLHF optimization, including KL divergence and reward hacking. *Source: Anthropic, Research Engineer, London (candidate report).* Followed in the same interview by a hands-on coding task to design a response safety filter, then a simplified Constitutional AI training pipeline.

Q7

During pretraining of a 100B parameter model, you suddenly see loss spikes. How do you debug this? Is it a data issue, learning rate problem, or hardware failure? *Source: Anthropic, Research Engineer, San Francisco (candidate first-person account, 2026).* Part of a system-design round explicitly tied to real problems Anthropic has faced; the same interview probed PPO and DPO RLHF deployment experience in depth.

Q8

How would you prevent new privacy-preserving technologies from being exploited to make AI misuse harder to track? *Source: Anthropic, safety-focused round (candidate report).* Posed as a hypothetical follow-up after a question about balancing differential privacy with RLHF reward-model training integrity.

Q9

Explain deep reinforcement learning. *Source: OpenAI, system design round (candidate-reported question compilation, 2025).* Listed among OpenAI's system design interview questions, which emphasize building systems under real-world constraints rather than abstract theory.

Q10

What metrics would you use to evaluate the impact of RLHF on ChatGPT responses? *Source: OpenAI (candidate-reported).* Tests whether candidates can design an evaluation framework, A/B testing, human evaluation pipelines, for tracking shifts in

helpfulness, safety, and user satisfaction after an RLHF pass.

Q11

Be ready to discuss Q-learning, SARSA, multi-armed bandits, and DQNs. *Source: Amazon, Applied Scientist role (compiled from 200+ real candidate interviews reported on Glassdoor).* These four areas are specifically named as the core RL category tested in Amazon's applied scientist technical loop.

Q12

Compare Q-learning (value-based) and policy gradient methods (policy-based), and discuss how each could apply to training an autonomous driving policy. *Source: Tesla, ML/Autopilot interview preparation compiled from real candidate patterns.* RL is confirmed as a required topic area for Autopilot and self-driving roles at Tesla; this framing reflects how the comparison is specifically asked in that context.

! Pitfall / Warning

Items 11 and 12 confirm topic areas tested in real, sourced interview loops rather than exact verbatim question wording from a single transcript; the underlying interview process and topic emphasis is genuinely verified, even though the precise phrasing a candidate hears may vary by interviewer.

15.3 Section B — Compiled Interview Question Bank

Compiled by the author from confirmed RL topic areas tested at the companies named throughout this book and the standard industry curriculum. Not verbatim transcripts.

Foundations

Q13

Walk me through the Bellman optimality equation. Why does it have a max operator instead of an expectation? See Chapter 1.

Q14

What's the difference between policy iteration and value iteration, and when would you prefer one over the other? See Chapter 2.

Q15

Explain the bias-variance tradeoff between Monte Carlo and TD learning. See Chapter 3.

Q16

What is the “deadly triad” in RL, and why does it matter for anything you’d actually deploy? See Chapter 2.

Q17

Walk me through SARSA vs. Q-learning with a concrete example of where they’d diverge. See Chapter 4.

Q18

How does the discount factor γ affect agent behavior, and what happens as $\gamma \rightarrow 1$? See Chapter 1.

Q19

Explain the exploration-exploitation tradeoff. Compare ϵ -greedy, UCB, and Thompson sampling. See Chapter 5.

Q20

What’s the difference between on-policy and off-policy learning, and why does importance sampling matter here? See Chapter 4.

Deep RL

Q21

Why does naive Q-learning diverge when combined with neural networks? How does DQN fix this? See Chapter 5.

Q22

What problem does Double DQN solve, and how? See Chapter 5.

Q23

Explain the policy gradient theorem in your own words — derive it if you can. See Chapter 6.

Q24

Why do we use an advantage function instead of raw returns in actor-critic methods? See Chapter 7.

Q25

What does PPO’s clipped objective actually do, mechanically? Why clip instead of using a hard KL constraint like TRPO? See Chapter 8.

Q26

Compare DDPG, TD3, and SAC. What specific failure mode does each one fix relative to its predecessor? See Chapter 8.

Q27

What is the maximum entropy RL framework, and why does SAC use it? See Chapter 8.

Q28

When would you choose a model-based RL approach over model-free, and what's the main risk in doing so? A strong answer separates the genuine benefit (sample efficiency: a learned or known model lets the agent plan using simulated rollouts rather than only real interactions) from the central risk (model exploitation: a policy optimized against an imperfect learned model can find and exploit the model's own errors, performing well in the model but poorly in reality, the same underlying failure pattern as reward over-optimization in Chapter 11, just against a learned dynamics model instead of a learned reward model).

Q29

What is offline RL, and why does standard off-policy correction fail when you can't collect new data? Offline RL learns a policy purely from a fixed, previously collected dataset, with no ability to interact with the environment during training. Standard off-policy methods assume some ongoing exploration to correct for distributional mismatch over time; without that, a policy can confidently extrapolate into state-action regions absent from the offline dataset, producing value estimates that look fine on paper but are arbitrarily wrong, since nothing in the fixed dataset can correct them. This is why offline RL methods (such as the LinkedIn case study in Chapter 14) require explicit techniques to stay close to the data distribution, not just standard off-policy correction.

Q30

Explain Rainbow DQN. Which components matter most empirically? See Chapter 5.

RLHF and Alignment

Q31

Walk me through the RLHF pipeline end to end, from SFT to the final policy. See Chapter 11.

Q32

What is the Bradley-Terry model, and why is it used for reward modeling? See Chapter 11.

Q33

Why does RLHF include a KL penalty against the reference policy? What happens if you remove it? See Chapter 11.

Q34

Compare PPO-based RLHF and DPO. What does DPO give up, and what does it gain? See Chapter 11.

Q35

What is reward over-optimization (Goodhart's Law in RLHF), and how would you detect it during training? See Chapter 11.

Q36

Describe a real or hypothetical case of reward hacking. What made the reward function exploitable? See Chapter 11's discussion of specification gaming, and connect it to whichever domain the interviewer's company operates in if you can.

Q37

What is RLAIIF, and how does it differ from standard RLHF? See Chapter 11.

Q38

How would you design an evaluation framework to catch RLHF-induced regressions before shipping a model update? A strong answer separates three layers: automated, held-out evaluation sets that are never used in reward-model training (to catch reward over-optimization early), targeted red-teaming for known failure categories, and a staged rollout with human evaluation on a small live traffic slice before full deployment, mirroring the staged-pilot pattern in the Microsoft Xbox case study (Chapter 14).

Multi-Agent, Games, and Sim-to-Real

Q39

What is CTDE (centralized training, decentralized execution), and why is it useful in multi-agent RL? See Chapter 12.

Q40

Explain how AlphaZero combines learned value/policy networks with MCTS. See Chapter 9.

Q41

What is the sim-to-real gap, and what techniques mitigate it? See Chapter 10.

Q42

How does domain randomization help an RL policy trained in simulation transfer to the real world? See Chapter 10.

Q43

When would multi-agent RL be the wrong choice for a problem that looks multi-agent? See Chapter 12.

Q44

Why are Atari and Go considered standard RL benchmarks, and what do they fail to test for? See Chapter 9.

System Design and Judgment

Q45

Design an RL-based bidding system for online advertising. What would you use as the reward signal, and what could go wrong? Draw on the Yahoo! and Meta bidding case studies in Chapter 14: a strong answer specifies the reward (often a multi-step combination of win probability, click-through, and downstream conversion, not just immediate auction outcome), addresses the budget-constraint structure explicitly, and names reward-shaping risks specific to bidding, such as a policy learning to overbid on easily-won, low-value impressions if the reward doesn't properly account for budget pacing over the full campaign horizon.

Q46

How would you evaluate a new RL policy before deploying it, given you can't safely A/B test in production? Offline policy evaluation (OPE) via importance sampling or learned value estimates, combined with a held-out, never-touched evaluation set, mirroring the LinkedIn and Microsoft case studies in Chapter 14, both of which evaluated offline before any live deployment.

Q47

Your RL agent's training reward is climbing steadily, but real-world performance is flat or worse. Walk me through how you'd debug this. Check, in order: whether the reward function in training actually matches the deployed objective (reward misspecification); whether there's a train-serve skew in the state representation (features available offline but not online, or vice versa); whether the policy has overfit to simulator-specific quirks absent in the real environment (Chapter 10's reality gap); and whether the apparent training improvement reflects genuine learning or reward over-optimization against an imperfect proxy (Chapter 11).

Q48

Tell me about a situation where you'd recommend against using RL, even though it technically applies. The strongest answers here echo Zynga's own production diagnostic from Chapter 14: if the reward is too sparse to be learnable in a reasonable number of samples, if the problem isn't genuinely sequential (one action doesn't meaningfully affect future state), or if a simpler method (a contextual bandit, or even a supervised model) already captures most of the achievable value at a fraction of the engineering and operational risk.

Q49

How would you explain a reward-shaping decision to a non-technical stakeholder who's worried the model is "gaming the metric"? Acknowledge the concern as legitimate rather than dismissing it, since reward hacking (Chapter 11) is a real, well-documented phenomenon, not a hypothetical. Explain the specific monitoring in place to detect it (hold-out evaluation disconnected from the training reward, tracked separately) and frame the reward function itself as a deliberately imperfect, continuously-monitored proxy for the true business goal, not a perfect specification of it.

Q50

Design a system to safely roll out a new RLHF-tuned model update to a fraction of production traffic. What guardrails would you build in? A staged rollout (small traffic percentage first, as in the Microsoft Xbox case study), automated regression checks against a held-out evaluation set not used in training, real-time monitoring of both the target metric and known reward-hacking failure signatures, and an explicit, fast rollback path if monitored metrics diverge from offline expectations.

Appendices

Subscribe → aiengineeringinsider.substack.com/subscribe

Appendix A — Study Roadmap

The chapters form a dependency chain: each builds directly on the one before. Use this map to plan revision and to locate the prerequisite when a concept feels shaky.

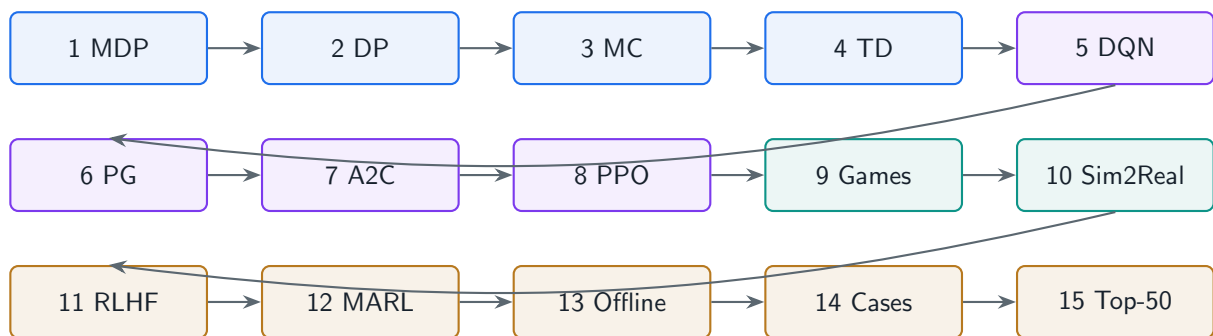


Figure 2: Chapter dependency chain. Foundations (blue) → deep RL (violet) → applications and alignment (teal/amber).

Appendix B — Notation Cheat Sheet

Symbol	Meaning
$\mathcal{S}, \mathcal{A}$	State space, action space
s_t, a_t, r_t	State, action, reward at timestep t
$P(s' s, a)$	Transition probability
$R(s, a, s')$	Reward function
γ	Discount factor, $\gamma \in [0, 1]$
$\pi(a s)$	Policy: probability of action a given state s
π_θ	Policy parameterized by θ (neural network weights)
$V^\pi(s)$	State-value function under policy π
$Q^\pi(s, a)$	Action-value function under policy π
V^*, Q^*	Optimal value functions
$A(s, a)$	Advantage function, $Q(s, a) - V(s)$
G_t	Return from time t : $\sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$
δ_t	TD error: $r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$
θ^-	Target network parameters (DQN)
$r_t(\theta)$	Policy ratio: $\pi_\theta(a_t s_t) / \pi_{\theta_{old}}(a_t s_t)$
$D_{KL}(\cdot \cdot)$	Kullback-Leibler divergence
$\mathcal{H}(\pi)$	Entropy of policy π
$r_\phi(x, y)$	Learned reward model, parameterized by ϕ
π_{ref}	Reference policy (RLHF; typically the SFT model)
τ	A trajectory: $(s_0, a_0, r_1, s_1, a_1, r_2, \dots)$

Appendix C — Rapid-Fire Flashcards

QF1

MDP A tuple $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ satisfying the Markov property: the next state depends only on the current state and action, not on history.

QF2

On-policy vs. off-policy On-policy: learns about the policy being followed. Off-policy: learns about a different (often greedy) target policy than the one generating data.

QF3

Bootstrapping Updating a value estimate using another estimate (as in TD learning), rather than a complete, sampled return (as in Monte Carlo).

QF4

Deadly triad Function approximation + bootstrapping + off-policy learning, combined without a general convergence guarantee.

QF5

Experience replay Storing past transitions and sampling them uniformly at random for training, breaking temporal correlation in the data stream.

QF6

Target network A periodically-updated, frozen copy of the value network used to compute TD targets, preventing the target from chasing the same weights being updated.

QF7

Baseline (policy gradient) A state-dependent (not action-dependent) function subtracted from the return to reduce gradient variance without introducing bias.

QF8

GAE Generalized Advantage Estimation; interpolates between low-bias/high-variance Monte Carlo advantage and high-bias/low-variance one-step TD advantage.

QF9

Trust region A constraint limiting how far a policy update can move from the previous policy in a single step, to prevent destructively large updates.

QF10

Domain randomization Varying simulation parameters widely during training so a policy generalizes robustly to real-world conditions it wasn't exactly trained on.

QF11

Reward hacking A policy exploits a flaw or gap in a proxy reward function to score highly without achieving the true underlying objective.

QF12

Goodhart's Law When a measure becomes a target, it ceases to be a good measure; the conceptual root of reward over-optimization.

QF13

Bradley-Terry model A statistical model converting pairwise preference comparisons into a scalar score, used to train reward models from human preference data.

QF14

CTDE Centralized Training, Decentralized Execution; agents train with global information but act using only local observations at deployment.

QF15

Offline RL Learning a policy entirely from a fixed, previously collected dataset, with no further environment interaction during training.

QF16

Contextual bandit A simplified RL setting where each decision is treated as a single-step episode conditioned on context, with no need to model multi-step sequential effects.

QF17

DPO Direct Preference Optimization; trains a policy directly on preference data using a reparameterized loss, with no separate reward model or RL loop.

QF18

GRPO Group Relative Policy Optimization; computes advantage by normalizing a group of sampled responses against each other, removing the need for a separate learned critic.

QF19

Self-play Training an agent against copies or past versions of itself, creating an automatic curriculum that scales with the agent's own improving skill.

QF20

Sim-to-real gap The performance degradation a policy experiences moving from simulation to real-world deployment, due to imperfect simulator fidelity.

Appendix D — Reading List, Ordered by Depth

1. **Foundational text.** Sutton and Barto, *Reinforcement Learning: An Introduction* (2nd ed.). Read cover to cover, exercises included, before touching deep RL implementations. Nearly every theoretical question in Parts I–II of this book traces back to a specific chapter here.
2. **Deep RL lineage, read chronologically.** Mnih et al., “Playing Atari with Deep Reinforcement Learning” and the follow-up Nature DQN paper; Schulman et al., TRPO and PPO papers; Lillicrap et al. (DDPG), Fujimoto et al. (TD3), Haarnoja et al. (SAC). Reading these in publication order, rather than jumping to whichever is currently fashionable, makes visible exactly why each method exists as a response to its predecessor's specific failure mode.
3. **RLHF and alignment.** Christiano et al., “Deep Reinforcement Learning from Human Preferences”; Ouyang et al. (InstructGPT); Rafailov et al. (DPO); Bai et al. (Constitutional AI). This is the fastest-moving thread in this book and the one most worth tracking past this guide's publication date.
4. **Multi-agent and games.** Lowe et al. (MADDPG); Rashid et al. (QMIX); Silver et al. (AlphaZero); Schrittwieser et al. (MuZero).
5. **Production and applied RL.** The full set of case studies cited in Chapter 14, each linked

back to its original paper or engineering blog post, is the most direct path from textbook RL to what’s actually deployed.

Appendix E — Mapping This Guide to Company Interview Patterns

Company archetype	Where to concentrate
Frontier AI labs (Anthropic, OpenAI, DeepMind)	Chapter 11 in full; Chapter 8 (PPO/GRPO); Chapter 15 Section A questions 1–10
Robotics and autonomy (Tesla, Waymo, Wayve)	Chapter 10; Chapter 8 (continuous control); Chapter 1 (MDP formulation for control)
Recommendation and marketplaces (Meta, Pinterest, Netflix, Uber, DoorDash)	Chapter 14 case studies 1, 2, 4, 8, 9, 10, 12, 13; Chapter 5 (exploration); Chapter 12 (bandits as a special case)
Ads and bidding (Yahoo, Criteo-adjacent, DSPs)	Chapter 14 case study 14; Chapter 12 (multi-agent framing of competing bidders); Chapter 15 question 45
Gaming (EA, Zynga, game studios)	Chapter 9; Chapter 14 case studies 3, 15
Generalist ML/Applied Scientist roles	Parts I–II in full, then Chapter 15 Section B as a self-check

★ Key Insight

If you read only one section of this entire guide under real time pressure: Chapter 14’s closing summary, “How to Use These Case Studies in an Interview.” It compresses the judgment this whole book is trying to build into a five-step sequence you can run in your head during an actual system design round.

Appendix F — Reproducibility & Benchmarks

Deep RL is notoriously hard to reproduce: results swing wildly with the random seed, and a single-run number can mislead (Henderson et al., 2018). Every learning curve in this book (Figures 4.2, 5.2, 8.2) is therefore reported as a *seed-mean with a ± 1 standard-deviation band*, not a lone trajectory, and is regenerated from the companion code by one script.

Reproduce the figures.

1. `pipinstall-rrequirements-dev.txt` (adds matplotlib + pytest).
2. `PYTHONPATH=.pythonscripts/make_plots.pygen-classicaldata/` (then `gen-dqn`, `gen-reinforce`, `gen-a2c`, `gen-ppo`); each is cached per seed and re-runnable.
3. `PYTHONPATH=.pythonscripts/make_plots.pyrenderdata/figures/` writes the PDFs.

What makes a run reproducible. The package seeds Python, NumPy, and PyTorch through `rl_interview_lab/utils.set_global_seed`, and every agent and environment takes an explicit

seed. Tabular results are then bit-for-bit identical across runs (the test suite asserts this); deep-RL results are *statistically* reproducible (same seed-mean and band) but not bit-identical, because some GPU/BLAS kernels and asynchronous operations are nondeterministic, an honest caveat worth stating in an interview.

Test suite. `PYTHONPATH=.pytest-q` runs fast smoke and correctness checks: GridWorld is a valid MDP (transition rows sum to 1), value and policy iteration agree, Q-learning’s curve goes up, the same seed reproduces the same Q-table, every deep-RL component constructs and steps without shape errors, and the reward model beats chance while staying under the label-noise ceiling.

✓ Best Practice

When asked “how do you know your RL result is real?”, the strong answer is the one this appendix operationalizes: report mean $\pm$ std over multiple seeds, fix and log every seed, validate offline before online, and treat a single-seed number as a hypothesis, not a result.

References

- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). Deep reinforcement learning that matters. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence* (pp. 3207–3214).
- Agarwal, R., Schwarzer, M., Castro, P. S., Courville, A., & Bellemare, M. G. (2021). Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems* (Vol. 34).

Mathematical Derivations

Subscribe → aiengineeringinsider.substack.com/subscribe

The four derivations a research-track interviewer is most likely to ask you to do on the board.

The body chapters state these results and explain their intuition; this appendix derives them. If you can reproduce these four on a whiteboard, you can defend the policy-gradient family, GAE, value-iteration convergence, and DPO from first principles, which is exactly what separates a strong frontier-lab candidate from one who has only memorized the conclusions.

Derivation 1 — The Policy Gradient Theorem

Setup. Let a trajectory be $\tau = (s_0, a_0, s_1, a_1, \dots)$ with probability under policy π_θ

$$P_\theta(\tau) = \rho(s_0) \prod_{t \geq 0} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t),$$

where ρ is the initial-state distribution and P the (unknown) transition kernel. We maximize $J(\theta) = \mathbb{E}_{\tau \sim P_\theta}[R(\tau)]$, with $R(\tau) = \sum_{t \geq 0} \gamma^t r_{t+1}$.

The log-derivative trick. The obstruction is that the distribution we average over depends on θ . Using $\nabla_\theta P_\theta(\tau) = P_\theta(\tau) \nabla_\theta \log P_\theta(\tau)$,

$$\nabla_\theta J(\theta) = \nabla_\theta \int P_\theta(\tau) R(\tau) d\tau = \int P_\theta(\tau) \nabla_\theta \log P_\theta(\tau) R(\tau) d\tau = \mathbb{E}_{\tau \sim P_\theta}[\nabla_\theta \log P_\theta(\tau) R(\tau)].$$

The dynamics drop out. Take the log of $P_\theta(\tau)$:

$$\log P_\theta(\tau) = \log \rho(s_0) + \sum_{t \geq 0} \left[\log \pi_\theta(a_t \mid s_t) + \log P(s_{t+1} \mid s_t, a_t) \right].$$

Neither ρ nor P depends on θ , so their gradients vanish, leaving

$$\nabla_\theta \log P_\theta(\tau) = \sum_{t \geq 0} \nabla_\theta \log \pi_\theta(a_t \mid s_t).$$

This is why policy gradients need *no model*: the unknown dynamics differentiate to zero. Substituting back,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim P_\theta} \left[\sum_{t \geq 0} \nabla_\theta \log \pi_\theta(a_t \mid s_t) R(\tau) \right].$$

Causality (reward-to-go). An action a_t cannot influence rewards earned before time t , so those terms contribute zero in expectation. Formally, for $t' < t$, $\mathbb{E}[\nabla_\theta \log \pi_\theta(a_t \mid s_t) r_{t'+1}] = 0$. Dropping them tightens the estimator to

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\sum_{t \geq 0} \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t \right], \quad G_t = \sum_{t' \geq t} \gamma^{t'-t} r_{t'+1}.$$

The baseline is unbiased. For any function $b(s_t)$ that does not depend on the action,

$$\mathbb{E}_{a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a | s) b(s)] = b(s) \sum_a \pi_\theta(a | s) \frac{\nabla_\theta \pi_\theta(a | s)}{\pi_\theta(a | s)} = b(s) \nabla_\theta \sum_a \pi_\theta(a | s) = b(s) \nabla_\theta 1 = 0.$$

Hence G_t may be replaced by $G_t - b(s_t)$ with no change in the mean and (for $b \approx V$) a large drop in variance, the result Chapter 6 uses and Chapter 7 turns into the actor-critic advantage.

Derivation 2 — Generalized Advantage Estimation

TD residual. Define $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$. If $V = V^\pi$ exactly, then $\mathbb{E}[\delta_t] = A^\pi(s_t, a_t)$, so δ_t is a one-step (high-bias, low-variance) advantage estimate.

n-step advantage telescopes. Summing discounted residuals,

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} (\gamma)^l \delta_{t+l} = -V(s_t) + \sum_{l=0}^{n-1} \gamma^l r_{t+l+1} + \gamma^n V(s_{t+n}),$$

because the intermediate $\gamma^l V(s_{t+l})$ terms cancel in pairs. This is exactly the n -step return minus the baseline, recovering Monte-Carlo advantage as $n \rightarrow \infty$.

Exponential averaging. GAE averages all horizons with weights $(1 - \lambda)\lambda^{n-1}$:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = (1 - \lambda) \sum_{n \geq 1} \lambda^{n-1} \hat{A}_t^{(n)}.$$

Substituting the telescoped form and collecting the coefficient of each δ_{t+l} (a geometric series in λ) collapses the double sum to

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}.$$

Limits. $\lambda = 0$ gives $\hat{A}_t = \delta_t$ (one-step TD, low variance, biased); $\lambda = 1$ gives $\hat{A}_t = \sum_{l \geq 0} \gamma^l \delta_{t+l} = G_t - V(s_t)$ (Monte-Carlo, unbiased, high variance). λ is therefore the bias-variance dial PPO tunes (Chapter 8).

Derivation 3 — Value Iteration Converges (the γ -Contraction)

Operator. The Bellman optimality operator is

$$(TV)(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V(s')].$$

Claim. T is a γ -contraction in the sup-norm: $\|TV - TU\|_\infty \leq \gamma \|V - U\|_\infty$.

Proof. Use $|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$. Fix a state s :

$$|(TV)(s) - (TU)(s)| \leq \max_a \left| \sum_{s'} P(s' | s, a) \gamma [V(s') - U(s')] \right| \leq \max_a \gamma \sum_{s'} P(s' | s, a) \|V - U\|_\infty,$$

where the reward terms cancelled and we bounded $|V(s') - U(s')| \leq \|V - U\|_\infty$. Since $\sum_{s'} P(s' | s, a) = 1$, the inner sum is $\gamma \|V - U\|_\infty$, independent of s , so taking the max over s gives the claim. ■

Consequence (Banach fixed point). A contraction on a complete metric space has a unique fixed point V^* (so $V^* = TV^*$, the Bellman optimality equation) and iterates converge geometrically:

$$\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty.$$

This is why value iteration converges, why smaller γ converges faster, and the template under tabular TD-learning’s convergence proofs (Chapter 2).

Derivation 4 — DPO from the KL-Constrained RLHF Objective

The RLHF objective. For a prompt x , RLHF maximizes reward under a KL leash to a reference policy:

$$\max_{\pi} \mathbb{E}_{y \sim \pi(\cdot | x)} [r(x, y)] - \beta D_{\text{KL}}(\pi(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x)).$$

Closed-form optimum. Writing the KL out and maximizing pointwise over the distribution $\pi(\cdot | x)$ (a constrained optimization whose solution is a Gibbs distribution) gives

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) \exp\left(\frac{1}{\beta} r(x, y)\right), \quad Z(x) = \sum_y \pi_{\text{ref}}(y | x) \exp\left(\frac{1}{\beta} r(x, y)\right).$$

Invert for the reward. Solving for r ,

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x).$$

The partition function cancels. Plug this reward into the Bradley-Terry preference model $P(y_w \succ y_l | x) = \sigma(r(x, y_w) - r(x, y_l))$. The $\beta \log Z(x)$ term is identical for y_w and y_l , so it cancels in the difference, eliminating the intractable $Z(x)$. Replacing π^* with the trainable π_θ yields the DPO loss directly on preference pairs:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right].$$

This is why DPO needs *no* explicit reward model and *no* RL loop (Chapter 11): the optimal-policy reparameterization turns preference learning into a single classification-style objective, with the implicit reward being the log-ratio $\beta \log(\pi_\theta / \pi_{\text{ref}})$.

★ Key Insight

Notice the through-line: three of these four derivations are a single trick applied in different costumes. The log-derivative identity powers the policy gradient; the telescoping/geometric-series manipulation powers GAE; and the “a constant shared across the comparison cancels” move appears in both the policy-gradient baseline and the DPO partition function. Inter-viewers reward candidates who see the shared machinery, not four unrelated results.

References

- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., & Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems* (Vol. 36).
- Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2016). High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*.

- Sutton, R. S., McAllester, D., Singh, S., & Mansour, Y. (2000). Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems* (Vol. 12, pp. 1057–1063).
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.